分类号 TP3　　密级

UDC　　编号

中国科学院研究生院

博士学位论文

Java 虚拟机的内存管理及优化

伍鸣

指导教师　胡伟武　研究员

中国科学院计算技术研究所

申请学位级别　工学博士　学科专业名称　计算机系统结构

论文提交日期　2007 年 4 月　论文答辩日期　2007 年 6 月

培养单位　中国科学院计算技术研究所

学位授予单位　中国科学院研究生院

答辩委员会主席

中国科学院计算技术研究所 鄢玺 2025-03-14 14:35:00

中国科学院计算技术研究所 鄢玺 2025-03-14 14:35:00

声　　明

我声明本论文是我本人在导师指导下进行的研究工作及取得的研究成果。尽我所知，除了文中特别加以标注和致谢的地方外，本论文中不包含其他人已经发表或撰写过的研究成果。与我一同工作的同志对本研究所做的任何贡献均已在论文中作了明确的说明并表示了谢意。

作者签名：　　日期：

论文版权使用授权书

本人授权中国科学院计算技术研究所可以保留并向国家有关部门或机构送交本论文的复印件和电子文档，允许本论文被查阅和借阅，可以将本论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存、汇编本论文。

（保密论文在解密后适用本授权书。）

作者签名：　　导师签名：　　日期：

中国科学院计算技术研究所 鄢玺 2025-03-14 14:35:00

中国科学院计算技术研究所 鄢玺 2025-03-14 14:35:00

摘 要

Java 语言是一种面向对象的语言，它运行在 Java 虚拟机的平台之上。Java 语言具有模块化好，跨平台和类型安全等特点。这些特点使得 Java 语言更适合于被用来进行快速安全的大规模的软件开发，因此该语言已经被广泛地使用在服务器，客户端和移动计算等应用领域中。Java 语言的另一个软件工程上的优势来自于 Java 虚拟机中自动内存管理的使用。自动内存管理也被称作垃圾收集（garbage collection）,它能够使程序员避免对显式分配和回收内存的依赖，从而在很大程度上减少了内存泄漏及悬挂指针等内存错误的发生，提高了软件开发的效率。垃圾收集所具有的这种优势也使得它被广泛地部署在除 Java 之外的多种运行时系统之中。

然而，尽管垃圾收集的引入能够提高软件开发的效率，但是垃圾收集系统却有可能会给应用程序的性能带来负面的影响，这主要是由于垃圾收集的运行时间是被包括在程序运行的总时间之内的。另外，一些垃圾收集算法能够移动对象的位置，因此，这些垃圾收集算法能够对应用程序的数据对象的局部性造成影响，从而影响程序的性能。一个良好的垃圾收集的设计和实现应该能够尽量地减少收集所消耗的运行时间，提高收集的效率，减少垃圾收集所带来的暂停时间同时尽可能地获取好的对象局部性。

围绕垃圾收集所带来的问题及其对应用程序性能的影响，本文系统深入地研究了 Java 虚拟机中的垃圾收集机制，其主要的创新和贡献如下：

第一，设计并实现了一个支持灵活的运行时切换垃圾收集算法的内存管理软件框架——RSMTk，并且避免了在收集算法切换过程中由于分配函数的内联而引起的方法重编译。比较了不同的垃圾收集算法和不同的对象拷贝序对应用程序的对象局部性的影响。比较分析了在拷贝垃圾收集中，分代内的不同分区间的拷贝序和不同分代间的拷贝序对对象局部性的影响。结果表明，分代内的不同分区间的对象拷贝序是影响对象局部性的主要因素。构建了一个结合预取和局部性优化技术的软件框架，结果发现基于规则的数组结构的预取更容易取得性能的改进。调查了 on-chip 的 local memory 在 Java 应用中对内存系统性能的影响。结果表明，on-chip local memory 和 cache 相结合的系统能够比 cache-only 的系统具有更优的性能。

第二，分析了服务器端 Java 应用中的对象生命周期行为，发现其中存在着大量长生命周期的对象。设计并实现了一种混合垃圾收集，利用几种经典收集算法的运行时动态切换及其相互的协作来减少垃圾收集对长生命周期对象的重复操作。使用 SPECjbb2005 对该算法进行了评估。实验结果表明，与经典的分代垃圾收集相比，该混合收集算法能够使 SPECjbb2005 的 Throughput 分值提高 3%~25%。

第三，设计并实现了一种新的并行垃圾收集标记算法——task-pushing。该算法利用了 CSP（Communicating Sequential Processes，通信顺序进程）的思想，通过将标记过程

构造成一个处理网络来避免对同步原语的依赖同时获得良好的动态负载平衡。为了支持这种无需同步操作的并行标记算法，设计了一种高性能的通信队列，该队列利用现有硬件所支持的字对齐访问的原子性来避免对同步原语的使用。实验结果表明，与传统的 work-stealing 技术相比，task-pushing 所带来的性能好处会随着处理器个数的增加而变得越来越大。对于 pseudojbb 和 GCOld 这两个服务器端的基准测试程序来说，当处理器的个数为 16 时，task-pushing 的加速比（speedup）比 work-stealing 分别提高了 17.4%和 13.5%。

关键词：Java 语言，Java 虚拟机，自动内存管理，垃圾收集，对象局部性，对象生命周期，并行垃圾收集，通信顺序进程

中国科学院计算技术研究所 鄢銎 2025-03-14 14:35:00

Memory Management and Optimization in Java Virtual Machine

Wu Ming (Computer Architecture)
Directed By Hu Weiwu

Java language is an object-oriented language running on the Java virtual machine. It has features of better modularity, platform independence and type safety and so on. These features make Java language more suitable for fast and safe development of many large scale softwares. Therefore, Java language has already been extensively used in fields of server, client and mobile computing. By using automatic memory management (garbage collection), Java runtime system has more advantages in software engineering. Garbage collection (GC) can free programmers from the efforts of explicitly allocating and reclaiming memory units and this can, to large extent, decrease the possibility of the happening of memory mistakes such as memory leak and dangling pointer, etc. These benefits of garbage collection also make it be extensively employed in many kinds of runtime systems besides Java.

Although garbage collection can improve the efficiency of software development, it also can bring some negative effects on performance of applications since the running time of garbage collection is counted into the total running time of the applications. In addition, because some garbage collection algorithms can move objects, they may have some effects on the locality of objects. A well-designed garbage collection should have less running and pause time, more efficiency and bring better locality of objects.

This paper systematically and deeply investigates the garbage collection in Java virtual machine. The contributions of this work are as fowllows:

Firstly, a memory management framework named RSMTk which supports flexible runtime switch of GC algorithms and avoids the recompilation incurred by inlining of allocation function is designed and implemented. The impacts of different GC algorithms and object copying orders on the locality of objects are compared and evaluated. We also compare the effects of object copying orders of inter-generation and intra-generation. The results show that object copying order of intra-generation is the main factor for affecting the locality of objects. We also construct a framework combining techniques of prefetching and optimizing on locality and find that the prefetching based on the regular array structures is easier to obtain better performance. Besides, we investigate the improvement of memory system performance brought by on-chip local memory. The results demonstrate that the memory system using cache and on-chip local memory can outperform the standard cache-only system if both systems have equivalent on-chip storage capacity.

Secondly, we analyse the lifetime of objects in Java server applications and find that there exist lots of long lived objects. We design a long-lived-object-based hybrid (LLH) garbage

collection algorithm which combines the features of compaction, copying and generation and takes advantage of the dynamic switching between them to largely decrease the replicated operations on long-lived objects. SPECjbb2005 is used to evaluate LLH algorithm and the results show that compared with generational garbage collection, LLH can improve the Throughput of the application by 3%~25%.

Finally, in our work, we present task-pushing, a novel scalable parallel marking technique for garbage collection that does not employ any synchronization operation. Enlightened by the idea of communicating sequential processes (CSP), task-pushing arranges the computation of marking process into a process network to obtain good load balance. In order to entirely avoid synchronization, a high performance queue structure which uitilizes the inherent atomicity property of word-aligned memory access enabled by most modern processors is designed for communication between GC threads. We evaluate task-pushing on the real hardware platform and compare it with the existent work-stealing technique. The results show that task-pushing has better scalability than work-stealing. For pseudojbb and GCOld which are two Java server-kind benchmarks, compared with work-stealing, task-pushing can improve the speedups by 17.4% and 13.5% respectively when the processor number is 16.

Keywords: Java language, Java virtual machine, automatic memory management, garbage collection, object locality, object lifetime, parallel garbage collection, communicating sequential processes

中国科学院计算技术研究所 鄢銮 2025-03-14 14:35:00

目 录

摘　要 I

目 录 V

图目录 IX

表目录 XIII

第一章 绪论 1

1.1 内存管理：静态，栈和堆 1
1.2 显式内存管理 vs. 自动内存管理 2
1.3 内存图：存活性和指针可达性 3
1.4 基本的垃圾收集技术 4
1.4.1 引用计数（Reference Counting）算法 5
1.4.2 标记清扫（Mark-Sweep）和标记缩并（Mark-Compaction）算法 6
1.4.3 节点复制（Copying）算法 7
1.4.4 基本垃圾收集算法的比较 8
1.5 分代垃圾收集 9
1.6 渐进和并发垃圾收集 10
1.7 并行垃圾收集 10
1.8 Java 虚拟机中的垃圾收集 11
1.8.1 Java 语言及 Java 虚拟机简介 11
1.8.2 虚拟机与垃圾收集之间的交互 12
1.5 本文的主要贡献 14
1.6 论文的组织 15

第二章 垃圾收集相关工作的综述 17

2.1 垃圾收集对运行时系统（runtime system）的影响 17
2.2 垃圾收集与对象局部性 18
2.3 垃圾收集的动态机制 25
2.3.1 基于参数的动态机制 25
2.3.2 基于算法的动态机制 28
2.3.3 基于年龄的动态机制 31

2.4 并行垃圾收集技术 33
第三章 实验平台介绍 39
3.1 实验平台简介 39
3.2 一个灵活的算法可切换的内存管理研究平台 41
3.2.1 堆的组织形式 42
3.2.2 基于 Allocation Area 的设计 43
3.3.3 垃圾收集算法的切换 44
3.3 本章小结 44
第四章 内存管理对应用局部性的影响 45
4.1 引言 45
4.2 垃圾收集系统性能及对程序局部性影响的比较和评估 46
4.3 预取与局部性优化结合的软件框架 52
4.3.1 整体框架设计 52
4.3.2 预取-局部性优化 framework 的工作流程 54
4.3.3 预取方法及其效果 54
4.4.4 局部性优化及其效果 55
4.4 Local memory 对内存系统性能的影响 56
4.5 本章小结 59
第五章 基于长生命周期对象的混合垃圾收集 61
5.1 引言 61
5.2 基于对象年龄特性的相关工作 63
5.2.1 控制提升的阈值 63
5.2.2 基于年龄的（age-based）垃圾收集 65
5.2.3 对象的预提升技术（pretenuring） 66
5.3 研究的目标——服务器端的应用 67
5.3.1 经典垃圾收集算法在服务器类应用中存在的问题 68
5.4 垃圾收集系统性能的评价标准 69
5.5 服务器类型基准测试程序——SPECjbb2005 的行为特点分析 70
5.6 基于长生命周期对象的混合垃圾收集算法 73
5.6.1 算法概述 73
5.6.2 LLOS 中短生命周期对象的清除及对长生命周期对象的预测 74
5.6.3 实验结果分析 76
5.7 LLH 算法的改进 78
5.8 本章小结 80

第六章 基于 Task-pushing 的并行 GC 标记算法 83
6.1 引言 83
6.2 并行标记算法的相关工作 85
6.3 CSP 和高性能的通信队列 88
6.3.1 高性能的通信队列 88
6.4 Task-pushing 并行 GC 标记算法 90
6.4.1 初始的划分区域的设计 90
6.4.2 Task-pushing 算法概述 92
6.4.3 标记过程的正确终止 94
6.5 Task-pushing 算法的负载平衡 95
6.5.1 任务的分配策略 95
6.5.2 队列长度的选择 96
6.6 实验评估 96
6.6.1 基准测试程序和实验平台 96
6.6.2 SRSW 队列长度的选择 97
6.6.3 任务分配策略 98
6.6.4 原子 CAS 指令的影响 99
6.6.5 Task-pushing vs. Work-stealing 99
6.7 本章小结 101
第七章 总结及未来工作 103
参考文献 105
致 谢 115
作者简历 117

中国科学院计算技术研究所 鄢玺 2025-03-14 14:35:00

中国科学院计算技术研究所 鄢玺 2025-03-14 14:35:00

图目录

图 1.1 内存泄漏和悬挂引用 2
图 1.2 内存堆中对象的连通结构 4
图 1.3 引用计数示意图 5
图 1.4 基于半空间的节点复制收集 8
图 1.5 并行垃圾收集与串行垃圾收集的示意图 11
图 2.1 Cheney 的拷贝 GC 算法 19
图 2.2 Moon 的近似深度优先拷贝算法 20
图 2.3 Wilson 的层次分解拷贝算法 21
图 2.4 基于阈值方式来控制垃圾收集的示意图 26
图 2.5 使用和不使用 GCH 的垃圾收集点的比较 27
图 2.6 混合的标记清扫和节点复制算法 29
图 2.7 Halstead 的并行拷贝 34
图 2.8 Flood 的并行缩并 35
图 3.1 DRLVM 的主要组件 41
图 3.2 基于 Allocation Area 的 RSMTk 的结构 43
图 4.1 垃圾收集性能比较 _202_jess 48
图 4.2 垃圾收集性能比较 _209_db 49
图 4.3 垃圾收集性能比较 _213_javac 49
图 4.4 垃圾收集性能比较 _228_jack 50
图 4.5 垃圾收集性能比较 pseudojbb 50
图 4.6 垃圾收集性能比较 antlr 51
图 4.7 垃圾收集性能比较 jython 51
图 4.8 预取-局部性优化整体框架图 52

图 4.9 Object Access Buffer 的结构 53
图 4.10 收集对象访问信息的代码 53
图 4.11 插入预取前和插入预取后的代码 54
图 4.12 预取带来的性能提升 55
图 4.13 使用 JDK1.5 运行 SPECjvm98 时堆中不同区域的 cache 失效次数的比例 57
图 4.14 Local memory 对内存系统性能的影响 57
图 5.1 在 nursery 中使用分离的创建空间 63
图 5.2 Shaw 的分“桶”方案 64
图 5.3 按照对象年龄排序的堆的视图 65
图 5.4 Deferred older-first 收集方案 66
图 5.5 SPECjbb2005 中垃圾收集时存活对象的数量 71
图 5.6 SPECjbb2005 中垃圾收集时存活对象在堆中的分布 72
图 5.7 基于长生命周期的混合垃圾收集器 73
图 5.8 SPECjbb2005 的性能比较 76
图 5.9 SPECjbb2005 中垃圾收集的时间开销 77
图 5.10 SPECjbb2005 Nursery 中对象存活率的比较 77
图 5.11 基于 Appel-like 改进的混合垃圾收集 78
图 5.12 SPECjbb2005 性能比较（with LLH-Appel-like） 80
图 5.13 SPECjbb2005 中垃圾收集的时间开销（with LLH-Appel-like） 80
图 6.1 简单的算法所导致的负载不平衡 85
图 6.2 使用 Stealable-mark-queue 实现的动态负载平衡机制 86
图 6.3 Arora 的 work-stealing-queue 87
图 6.4 SRSW 队列的实现 89
图 6.5 使用 cache 优化后的 SRSW 队列数据结构 89
图 6.6 划分为 4 个区域的堆结构 90

图 6.7 具有 CSP-style 的划分区域的并行标记算法 91

图 6.8 对存活对象的遍历依赖于其它线程 91

图 6.9 标记过程的处理网络 93

图 6.10 task-pushing 的伪代码 93

图 6.11 并行标记过程的终止检测机制 94

图 6.12 不同队列长度对 task-pushing 可缩放性的影响（任务分配策略为 hybrid task sharing） 97

图 6.13 不同的任务分配策略对 task-pushing 的可缩放性的影响（队列长度为 1） 98

图 6.14 同步原语对可缩放性的影响（任务分配策略为 hybrid task sharing，队列长度为 2） 99

图 6.15 Task-pushing 与 work-stealing 的性能和可缩放性的比较 100

中国科学院计算技术研究所 鄢玺 2025-03-14 14:35:00

中国科学院计算技术研究所 鄢玺 2025-03-14 14:35:00

表目录

表 1.1 基本垃圾收集算法的特点比较 8

表 4.1 基准测试程序简介 46

中国科学院计算技术研究所 鄢銮 2025-03-14 14:35:00

中国科学院计算技术研究所 鄢玺 2025-03-14 14:35:00

第一章 绪论

1.1 内存管理：静态，栈和堆

内存是一种有限但却很有价值的资源。尽管处理器能够通过 cache 和寄存器来缓存一定量的数据信息，但却是非常有限的。内存作为处理器和磁盘之间的临时数据存储单元，其存在具有相当的必要性。随着现代微电子技术的发展，计算机可以使用的内存的容量也在飞快地增长，但是内存仍然不是取之不尽，用之不竭的。因此，如何正确而高效地管理这种珍贵的资源就是内存管理技术的一个重要目标。

内存的管理包括内存的分配和回收。从程序员的角度来说，内存的回收是为了更好的进行内存的分配，而且，内存的分配技术和回收技术并不是完全独立的，它们之间有着一定的依赖关系。内存的分配从大体上可以分为 3 类：静态分配，栈分配和堆分配。静态分配是最基本最简单的一种内存管理策略。它最早被实现在 Fortran 编程语言中[Jones96]，现代大部分的高级编程语言中的全局数据结构，一般也采用这种管理策略。在这种管理策略中，程序员在编译时确定程序中所使用的各种数据结构的大小，并决定为它们分配多大的内存空间，同时，编译器还将程序中所有的符号变量绑定在某个固定的存储位置上。这样，当程序刚开始运行时，所有的内存就都已经被分配好了，直到程序退出，内存才一起被释放。

静态分配的局限性在于，每个数据结构的大小必须在编译时知道，因此无法动态地创建数据结构。另外过程也无法递归，因为对于过程的每个活动（即过程体的一次执行），相同名字的局部变量被绑定在内存中相同的位置。因此，静态分配策略不足以满足过程式语言的要求，因为过程式语言允许局部变量和递归函数的存在。然而，后进先出（LIFO）的过程调用模式使得栈分配成为可能。在运行之前，系统为程序保留一块足够大的内存空间作为程序的栈。每次过程调用的时候，一个保存着过程局部变量信息的活动纪录（activation record）被压入栈中，并在过程返回的时候被弹出。

栈分配支持了活动记录级的局部变量，从而使得递归调用成为可能，增强了程序语言的表达能力。但是由于栈的后进先出的分配模式，被调用者的活动记录不可能比它的调用者的活动记录寿命更长。这意味着栈分配对内存分配和回收的顺序有一个限制，即后分配的对象会先被释放。现代大部分的应用，特别是有关符号处理和抽象建模的应用，其数据使用模式并不是静态的，也不能够限制分配和释放的时间顺序。最常见的例子是链表和树类的应用。在这类应用中，数据结构的大小不再是固定的，而是可以动态变化的，并且由这些数据结构所创建的对象的生命周期也是不可预测的。为了支持这种类型的应用，堆分配机制被提出和使用。一个堆就是一块内存区域，其中的对象能够以任意的次序被分配和回收，因此对象的生命周期可能比创建它们的过程具有更长的寿命。毫

无疑问，堆分配使得程序员能够控制对象的生命周期，从而获得了更大的灵活性。但是，如何正确地释放对象的存储空间就不再是一个非常简单的问题了。

1.2 显式内存管理 vs. 自动内存管理

许多语言，例如 C、C++、Pascal、Fortran90 等，都要求程序员通过使用 free()或者类似的函数接口来显式地释放不再被使用的对象所占用的内存空间。这给程序员的工作带来了极大的负担。程序员必须小心谨慎地调用 free()接口来释放内存空间，从而避免有可能导致严重后果的内存错误的发生。

如果程序员不能够及时地释放不再被使用的内存空间，将导致内存泄漏。对于长时间运行的服务器类型的应用来说，即使是速度非常缓慢的内存泄漏也是不能容忍的，因为内存泄漏最终会导致内存资源被耗尽而使程序崩溃。造成内存泄漏的原因主要有两种：一个是由于程序员没有及时地调用 free()函数接口，另一种情况是程序员在调用 free()之前错误地销毁了指向对象的最后一个引用，从而使对象不再会被使用但仍未被释放，这种对象被称作垃圾（garbage）。另外，如果程序员在指向对象的所有引用还没有被完全销毁之前就错误地调用了 free()函数接口，就会引起悬挂引用。程序有可能在之后的某个时刻会去访问这个引用，从而造成错误的执行或者引起内存访问异常。图 1.1 说明了这两种情况。这两种错误的情况是非常危险的，它们会使调试变得非常困难。如果某种分配函数碰巧没有重用被错误释放的对象，则悬挂引用可能不会引起错误。也就是说悬挂引用所引起的错误是存在一定的随机性的。而内存泄漏则很可能会在程序正常运行了很长的时间以后才表现出来。

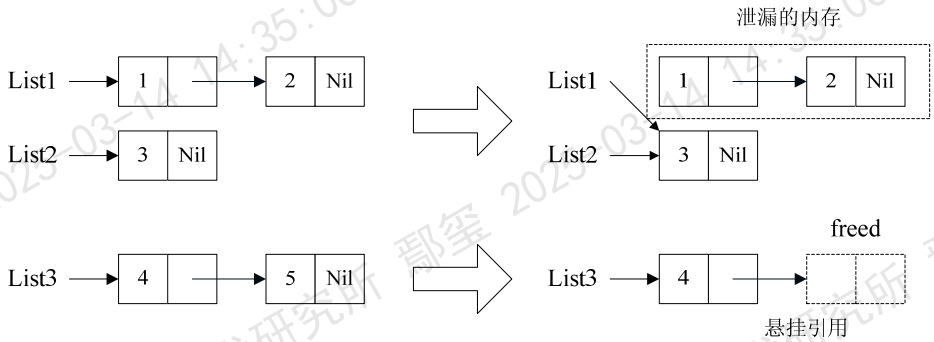

图 1.1 内存泄漏和悬挂引用

另外一种情况是当存在数据共享的时候。假设两个链表或树结构共享一部分的数据对象。当某个过程要将其中一个链表销毁的时候，那部分共享的数据对象应该保留下来，因为还有别的过程可能会访问到这些对象。这意味着一个过程对某个数据结构的操作依赖于另外一个可能操作相同数据结构的过程，也就是说，一个过程必须能够知道当前所要释放的对象的存活信息，即是否还存在着其他的对这个对象的引用[Boehm92]。这种对象的存活信息是一个全局的属性，因此这会导致必须引入一种全局的规范或簿记机制[Wilson94]。

为了解决以上的问题，现代大部分的函数式（functional）和面向对象（object-oriented）的编程语言（例如：Lisp, ML, Haskell, Java, C#等）都使用了自动内存管理的技术，也即垃圾收集。在这样的系统中，程序员无需显式地通过调用 free()函数接口来回收内存，而是由运行时系统自动地回收不再被使用的内存空间。

Jones 给出了使用垃圾收集技术的几个原因[Jones96]， 其中最主要的两个原因来自于应用问题的需求和软件工程的需求。垃圾收集能够使程序员摆脱显式回收内存的负担，从而在很大程度上避免了潜在的内存错误的发生。同时，垃圾收集避免了程序模块之间的不必要的依赖（引用信息的依赖），一个程序过程不再需要考虑对象的存活信息。垃圾收集的这种特点给软件工程带来了巨大的好处。程序员可以把时间和精力更多地花在高层的设计细节和所面对的程序设计的实现上。一个好的软件设计依赖于适当的抽象和模块化，而这种针对程序复杂性的方法的成功又依赖于模块之间接口的最小化[Meyer88]。尽管对于显式内存管理和自动内存管理的优劣没有一个全面而又明确的定论，但是从软件工程的角度来看，垃圾收集具有明显的优势。显式内存管理破坏了复杂软件系统的抽象和模块化，成为许多潜在错误的来源。检查内存错误的工具（最著名的例子是 Purify [Purify92]）及在不友好的环境中的保守垃圾收集[Boehm88]的存在，正说明了内存管理问题的严重性和其发生的频繁程度。

虽然垃圾收集在软件工程上具有明显的好处，但是它在时间上的性能开销却常常成为广大研究者攻击的目标。人们普遍的相信垃圾收集相对于显式堆管理是昂贵的。过去，对于某些应用而言，情况确实很严重。例如，20 世纪 70 年代到 80 年代早期的研究发现，大型 Lisp 程序在垃圾收集上花费的时间通常占程序执行全部时间的 40%[Steele75, Foderaro81, Gabriel85]。不过，也有一些研究表明垃圾收集在某些情况下与显式回收相比在性能上也是具有一定的竞争力的[Appel87, Zorn93]。事实上，由于垃圾收集的开销与应用、语言和内存资源密切相关的，因此很难对此给出一个简单的定量估计。Wilson 指出，对于一个实现良好的垃圾收集系统来说，10%的开销应该是比较合理的[Wilson94]。许多程序员认为垃圾收集的开销是不可接受的，但是还有些人认为与软件开发的便利性，软件开发周期的减少，以及软件可靠性的增加所带来的好处相比，性能上的小的代价是值得的。不论如何，垃圾收集在软件工程上的优势使得大量的研究者们不断的寻求提高垃圾收集系统性能的方法。目前许多被广泛使用的面向对象的编程语言系统都集成了垃圾收集，特别是 Java 语言这样的类型安全语言。

1.3 内存图：存活性和指针可达性

从垃圾收集的角度来看，用户程序的操作包括读写内存以及使用寄存器里的值来进行计算。程序在开始的时候可以直接操纵寄存器，程序堆栈和全局变量中的值。这些值中有一部分是指向堆中数据对象的引用，它们构成了根集（root set）。程序从根集出发通过不断地遍历堆中对象的相互指针来访问堆中的数据。在程序执行的任何一点，它所能

够遍历到的对象的集合称为可达（reachable）对象集合。然而，程序可能只会访问可达对象集合的一个子集，称为存活（live）对象集合。要精确地确定程序中的存活对象集合并不容易，它可能需要编译器的支持。在一个优化的编译器中，通过复杂的控制流和数据流分析，编译器可能能够确定一个对象不会再被程序所使用，即使它仍然是可达的，即仍然存在从根集遍历到该对象的引用路径。但是在通常情况下，垃圾收集并不能总是假设这种复杂的编译技术是存在的。因此，所有的垃圾收集系统，甚至几乎所有的显式内存管理系统都对存活对象集合做了一个保守的近似，即用可达对象集合来定义存活对象集合。不可达的对象可以被认为是已经消亡的，而它所占用的内存空间是可以安全的被回收的。

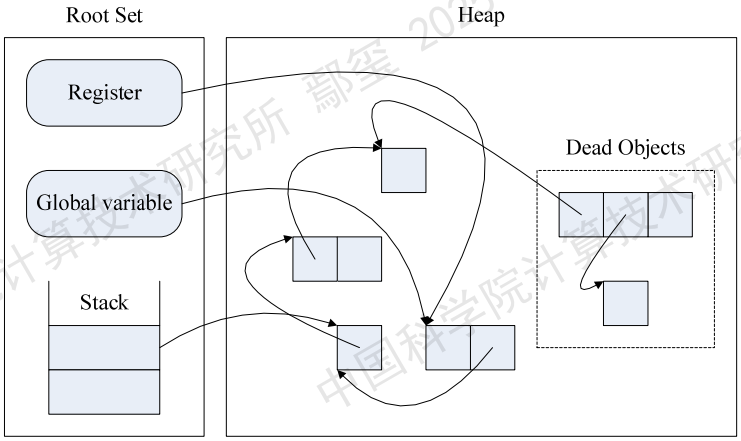

图 1.2 内存堆中对象的连通结构

存活对象集合的正式定义如下：

定义“→”为“指向”关系，M 是堆中任意的对象或根，N 是堆中任意的对象，Objects 是堆中所有对象的集合。当且仅当 M 拥有一个指向 N 的引用时，M→N 成立。堆中存活对象集合是根集合在这个关系下的传递引用闭包，也就是最小集合 *Live*：

$$Live = \{N \in Objects \mid (\exists r \in Roots.r \rightarrow N) \vee (\exists M \in Live.M \rightarrow N)\}$$

对于表示内存中的对象关系结构来说，有向图是一个非常有用的工具。许多垃圾收集系统的术语都来自于图论。在一个图的表示系统中，对象被表示成节点，指针被表示成边，root set 被表示成对象图的根。图 1.2 是堆中对象连通图的一个示意图。图中显示了 6 个对象，其中左边的 4 个对象是从根集可达的，因此是存活的对象。而虚线框中的两个对象不是根集可达的，是死的对象，它们所占用的内存空间应该被回收重用。当用户程序执行的时候，对象的分配操作会在对象图中生成新的节点，而对对象引用域的赋值操作会改变对象图中的边，从而影响对象图中节点的可达性。由于从垃圾收集器的角度来看，用户程序会持续不断地改变对象图的状态，因此用户程序也被称作“mutator”。

1.4 基本的垃圾收集技术

垃圾收集能够自动地回收内存堆中不再被使用的对象所占用的内存空间，它的实现

原理依赖于上一节所介绍的对象的存活性理论。抽象地来说，垃圾收集的基本功能的实现包括两个阶段：1. 区别存活对象和垃圾对象，即垃圾对象的发现；2. 回收垃圾对象所占用的存储空间，以便让程序能够重用这些空间。事实上，这两个阶段在功能上和时间上是可以有一定的重合的，存储空间的回收技术可能是依赖于垃圾对象的发现技术的。通常有两种技术可以用来区别存活对象和垃圾对象：引用计数（reference counting）和追踪（tracing）。“追踪”这个术语最早来源于[Lang87]，它主要是用来描述依靠遍历堆中对象连通图的方法来确定存活对象的技术，其中包括了标记（marking）和复制（copying）技术。

1.4.1 引用计数（Reference Counting）算法

引用计数算法是最早的通用内存管理算法之一[Collins60]。在引用计数算法中，每一个对象都需要使用一个附加的域来存放引用计数，即指向该对象的引用的个数。当程序正在执行的时候，对象的引用计数被不断地更新以便及时地反映出当前对象连通图的状态改变。每次有一个新的引用指向某个对象的时候，该对象的引用计数值就会被增加 1。当现有的指向某个对象的引用被删除的时候，该对象的引用计数值将被减 1。如果对象的引用计数值被降为 0，即没有任何引用指向该对象，则该对象变为不可达的，其所占用的内存空间就可以被系统回收了。图 1.3 显示了一个引用计数的例子。

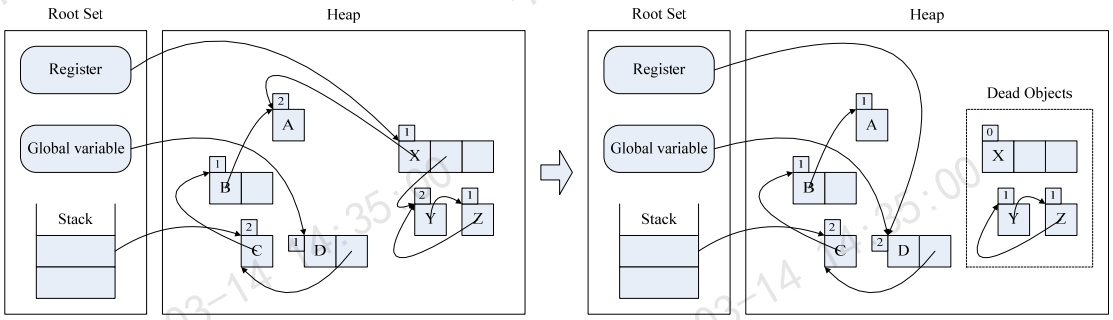

图 1.3 引用计数示意图

起初，如左图所示，寄存器中有一个引用指向对象 X，X 的引用计数为 1。接下来，程序修改了这个引用，使其指向了对象 D，此时对象 D 的引用计数变为 2，对象 X 的引用计数变为 0，于是 X 成为一个不可达的对象，其所占的内存空间可以被安全地回收。这种技术属于自动内存管理，因为对引用计数的维护可以由编译器产生的代码自动完成。当一个对象被回收的时候，引用计数算法会检查该对象的所有引用域，并且将该对象的引用域所指向的对象的引用计数值减 1，因为一个垃圾对象的引用不应该被用来判断其指向的对象的存活性。如图 1.3 的右图所示，对象 A 的引用计数值被减为 1，对象 Y 的引用计数值也被减为 1。这意味着，回收一个对象可能会导致需要传递地减少垃圾对象所指向的对象的引用计数值，同时回收许多其他的垃圾对象。

不幸的是，简单的引用计数算法无法回收环形的数据结构。图 1.3 的右图说明了这种现象。对象 Y 和 Z 形成了环状的结构，它们的引用计数值都是 1。实际上，它们已经

是不可达的对象了，但是它们的引用计数值将永远不会被减成 0。有一些特殊的引用计数技术能够处理环状的数据结构，但是非常复杂而且具有指数量级的时间复杂度[Salkild87, Pepels88, Bacon00]。引用计数的另一个严重的缺点就是为了维护对象的引用计数值，这种实现不得不付出昂贵的处理开销。对于每次引用的修改，它原来指向的对象和将要指向的对象的引用计数值都必须进行相应的调整。这种内存管理机制总是与用户程序或编译器紧密地藕合在一起，这使得引用计数系统更加难以维护。

引用计数算法从本质上具有较好的渐进特性，它不会造成用户程序的长时间的暂停。它在空间上的局部性可能和用户程序一样好，因为它不用访问堆中其他的内存单元就能够回收当前的垃圾对象。另外，在引用计数系统中，对象一旦成为垃圾就可以立刻被回收。然而，引用计数算法的这些优点几乎都被维护计数的高额开销及处理环状结构的困难性所抵消了。

1.4.2 标记清扫（Mark-Sweep）和标记缩并（Mark-Compaction）算法

标记清扫算法是一种基于追踪的垃圾收集算法[McCarthy60]。在这个算法中，程序在正常的执行过程中不需要执行额外的操作，垃圾对象也并不是在产生的时候就立刻被回收。垃圾回收的过程只有当内存资源已经耗尽的时候才会被启动。当垃圾收集被发起的时候，系统会将所有的用户程序暂时地挂起（这个过程被称为 stop-the-world），然后收集算法从根集出发遍历堆中所有的可达对象。在遍历到一个对象的时候，就对该对象进行标记，表明该对象是一个存活的对象。这样，当遍历过程结束以后，所有没有被标记的对象就是已经死亡的对象，它们所占用的内存空间会在清扫的过程中被回收。那些被标记过的存活对象的标记位也要在下一次垃圾收集调用之前被清除。与引用计数算法不同，环状数据结构的问题可以很自然地解决。然而，标记清扫的垃圾收集算法并不具有渐进的特性，它必须将用户程序的运行暂停下来，这种暂停所带来的开销可能是巨大的[Foderaro81]。在标记阶段，由于收集器只会访问到存活的对象，因此标记过程的时间复杂度是和存活对象的数量成比例的。然而，在清扫阶段，收集器必须扫描整个堆中所有的对象，因此，清扫过程的时间复杂度是和整个堆的大小成比例的。总的来说，标记清扫收集算法的时间复杂度和堆的大小成比例。

标记清扫的收集算法并不会改变对象在内存堆中的位置。然而，这种固定位置的内存管理方法可能会产生更多的内存碎片（fragmentation）。内存碎片可能带来的坏处有以下几种：

1. 碎片的产生会降低 cache 的性能；
2. 碎片的产生可能会降低页（page）级的局部性，从而导致更多的 TLB 失效和页面的换入换出；
3. 碎片的产生会降低内存分配的效率，因为分配函数必须在整个堆中寻找合适的内存空隙来容纳新分配的对象。甚至当堆中总的自由空间足够的时候，分配函数也可能会由于无法找到大小足够的内存空隙而导致垃圾收集的频繁发生。

标记缩并算法在一定程度上解决了标记清扫算法中难以避免的碎片问题以及由该问

题所引起的分配效率的问题。标记缩并算法的标记阶段与标记清扫算法是相同的。与之不同的是，当存活对象已经被标记出来之后，这些对象被移动到堆的一端，并且被连续地存放着，而堆的另一端则形成了一段可供分配的连续的自由空间[Haddon67, Saunders74, Jonkers79, Cohen83]。收集器通常会使用一种滑动的方法来缩并存活的对象，即线性地扫描整个内存堆，找到存活的对象，然后将其向目标方向滑动到和前一次找到的存活对象相邻的位置。最终，所有的存活对象就被一个接一个地相邻地放在了堆的某一端。

这种滑动的缩并有以下几个优点：

1. 消除了内存碎片。对象的连续排放及连续的自由空间使得分配可以高效地实现。对象的分配可以通过将指向自由空间的指针简单地向后移动所要分配的对象的大小即可。这种分配过程也被称作“bump pointer”分配
2. 由于滑动并不改变对象在堆中的相对顺序，因此缩并后的对象仍然保持着对象分配时的顺序。这会带来更好的对象局部性，因为对象的分配序通常会和后来对象的访问顺序保持一定的相似性[Clark77, Clark79]。

然而，在标记阶段结束以后，标记缩并收集算法通常需要再遍历存活对象 2~3 遍[Cohen83]。一遍用来计算存活对象将要滑动到的目标地址（这一遍的时间复杂度和堆的大小成比例），一遍用来更新存活对象中的引用域，然后再真正地去移动存活的对象。因此，一次标记缩并的时间开销可能比一次标记清扫更大。

1.4.3 节点复制（Copying）算法

同标记清扫和标记缩并算法一样，节点复制算法[Fenichel69, Cheney70]也是一种基于追踪技术的算法，并且同样不需要用户程序附加额外的操作。节点复制算法会将当前所收集的区域中的存活对象拷贝到其他的区域中去，而把垃圾对象都留在当前的区域中。当所有的存活对象都被迁移走了以后，当前的区域就不再有有用的数据了，从而可以作为下次收集时的目标区域。一个形象的术语“scavenger”常常被用来描述节点复制算法，因为它的行为就是从垃圾中挑出有用的东西，然后将其带走。

一个简单的节点复制算法是使用两个相同大小的连续内存空间，其中每一个空间被称为“半空间”（semispace）。这两个半空间所扮演的角色并不相同，其中一个是源空间（Fromspace），一个是目标空间（Tospace）。初始的时候，这两个半空间都为空，对象只能够在 Fromspace 中进行分配。当 Fromspace 中的自由空间被耗尽的时候，垃圾收集就被唤起。同前面提到的基于追踪技术的收集算法一样，节点复制算法也要将所有正在运行的用户程序挂起，然后从根集出发遍历 Fromspace 中的存活对象。标记缩并算法将标记和移动对象的过程分在不同的阶段来完成，而节点复制算法却将这两个阶段融合在了一起，即在遍历到一个存活对象的同时，就将该对象复制到目标空间中去。在复制一个对象的时候，收集器为该对象保留一个迁移地址（forwarding address），它实际上就是该对象在 Tospace 中的复本的地址，即对象的新地址。具体的实现可以将迁移地址保存在 Fromspace 中的对象头的元数据结构中（meta-data）。每当 Fromspace 中的某个对象被

遍历到的时候，收集器会检查该对象是否已经被复制过了，如果没有被复制过，则将该对象复制到 Tospace 中去，然后用该对象的迁移地址来更新指向该对象的引用域，否则就直接返回该对象的迁移地址并更新指向该对象的引用域即可。当所有的存活对象都被复制到 Tospace 中以后，Fromspace 和 Tospace 要互换一下角色，即原来的 Tospace 成为当前的 Fromspace，原来的 Fromspace 成为现在的 Tospace。

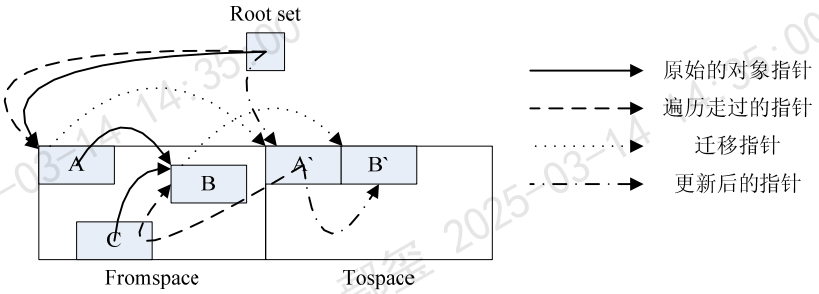

图 1.4 基于半空间的节点复制收集

然后收集过程结束，用户程序被重新唤醒，对象的分配将在当前的 Fromspace 中继续进行，直到 Fromspace 中的自由空间再次被耗尽。图 1.4 示意了这种基于半空间的节点复制算法。节点复制收集算法有个很自然的优点就是在拷贝对象的过程中，存活的对象被连续地存放在 Tospace 中，这种缩并的特性使得它不会面临内存碎片的问题。由于每次收集之后所释放的可以用来重新分配的自由空间都是连续的，因此该算法的分配函数也可以使用非常高效的 bump-pointer 分配法。另外，由于节点复制算法无需关心垃圾对象，因此它的时间复杂度是和存活对象的数量成比例的。但是，这种基于半空间的节点复制算法必须使用两个半区，其中一个半区作为预留的空间，不能进行对象的分配。因此，与标记清扫和标记缩并算法相比，如果具有相同大小的堆，节点复制算法中可以用来分配的内存资源更加有限，因此会使垃圾收集的发生更加频繁。

1.4.4 基本垃圾收集算法的比较

表 1.1 总结了前文描述的几种基本垃圾收集算法的特点：

表 1.1 基本垃圾收集算法的特点比较

	引用计数	标记清扫	标记缩并	节点复制
是否有渐进性	是	否	否	否
是否立即回收	是	否	否	否
能否解决环状结构	不能	能	能	能
能否解决碎片问题	不能	不能	能	能
时间复杂度	O(指针赋值次数)	O(堆的大小)	O(堆的大小)	O(活对象数量)
空间需求	工作集大小[1]	工作集大小	工作集大小	2*工作集大小

[1] 工作集大小是指任意时刻，堆中存活对象的大小总和的最大值。

1.5 分代垃圾收集

简单的基于追踪技术的垃圾收集都需要把正在运行的用户程序挂起，然后才能进行垃圾的回收工作。因为可能有大量的对象需要被处理，垃圾收集的过程可能会给用户程序带来很大的延迟。不论是标记清扫，标记缩并还是节点复制，它们都会反复地追踪并标记或复制堆中所有的存活对象。这种垃圾收集并没有考虑到对象的生命周期。事实上，一个好的垃圾收集器应该尽可能地避免重复地处理长生命周期的对象。另一方面，许多对象的生命周期是很短暂的。已经有大量的数据表明大多数的对象会在分配后很快地消亡，这被称为弱分代假设（weak generational hypothesis）[Deutsch76, LH83, Ungar84, Shaw88, Zorn90, DeT90, Hayes91]。

基于弱分代假设，分代垃圾收集（generational garbage collection）被提出。这种垃圾收集通过把对象按照年龄划分在不同的区域来尽量地避免重复地处理长生命周期的对象。这些区域被称为分代。不同的分代的收集频率不同，年轻的分代会比年老的分代具有更高的收集频率。这使得分代垃圾收集可以把工作集中在回收年轻的对象（即最有可能成为垃圾的对象）上，从而提高垃圾收集的效率。

在分代垃圾收集中，最年轻的分代通常被称为“nursery”，对象的分配也都是在 nursery 中完成的。当 nursery 中的自由空间被耗尽的时候，一次次级收集（minor collection）被发起。在次级收集过程中，nursery 中的活对象被拷贝到年老的分代中。当年老代的自由空间也被用尽的时候，对多个分代的主收集（major collection）就会被发起。尽管分代垃圾收集也需要把用户程序挂起，但是其中次级收集的频率要比主收集的频率高的多。由于次级收集所处理的年轻代通常比较小，并且年轻代中大部分都是死去的对象，因此，一次次级收集可以非常迅速地完成，从而只会给用户程序带来非常短暂的暂停时间。然而，主收集仍然可能会给用户程序带来非常严重的延迟，特别是当年老代中的存活对象很多，内存资源比较紧张的时候，主收集也可能会被频繁地发起。

另外，分代垃圾收集中次级收集的根集的确定也并不是十分的容易。除了标准的根集（寄存器，程序栈和全局静态存储区中的引用）以外，次级收集的根集还必须包括从年老代指向年轻代的代间指针。这种代间指针的创建主要有两个途径：指针的赋值或是一个包含指向年轻代指针的对象被拷贝到年老代中。垃圾收集可以很容易地识别由于对象的拷贝而创建的代间指针，而对于那些通过赋值创建的代间指针，则必需通过写拦截（write-barrier）在这些指针被写入的时候捕获和记录它们。如果代间指针被记录以后，处于年老代中的那个对象不再存活，则处于年轻代中的那个对象即使实际上已经消亡，也仍然不能够被识别出来。因此该对象在下次的次级收集中还是会被当作存活的对象而被复制到年老代中。这种在年老代中的垃圾通常被称作占位垃圾（tenured garbage），它在一定程度上影响了分代垃圾收集的效率。

1.6 渐进和并发垃圾收集

尽管分代垃圾收集通过频繁的次级收集能够在一定程度上减少平均的暂停时间，但是终究会发生的主收集仍然有可能会让许多应用产生令人无法接受的延迟。为了减少可能发生的暂停时间，有两种垃圾收集技术被提出：渐进垃圾收集和并发垃圾收集[Weizenbaum62, DeTreville90, Steele75, Dijkstra76, Kung77, Yuasa90, Baker78, Doligez93, Doligez94, Click05]。

渐进垃圾收集仍然需要挂起正在运行的用户线程，但它并不是收集整个堆中的对象，而是将堆按照某种方式划分成若干个区域，每次只回收部分区域中的垃圾对象，从而减少暂停时间。这种技术不会等到空闲的内存资源耗尽的时候才被发起收集，而是将收集的过程和用户程序的活动交织在一起，其做法可以是在每次分配的时候完成一小部分的标记或复制工作。

与渐进垃圾收集不同，并发垃圾收集是使用独立的收集线程来和用户线程并发地执行。这种垃圾收集通常被使用在拥有多处理器的系统中。通过并发的执行，垃圾收集系统将不会中断用户程序的执行，垃圾收集和用户程序的交错执行完全通过操作系统的进程调度机制来完成。

在渐进和并发垃圾收集中，由于用户程序和垃圾收集交错地运行，不可避免地需要引入一些同步的机制。堆中的对象对于垃圾收集和用户程序来说是一种共享的资源。垃圾收集的正确性依赖于对堆中对象连通图的准确的把握，而用户程序却会不断地改变这个连通图的状态（这也是用户程序被称为 mutator 的原因）。而且，对于会改变对象位置的垃圾收集器来说，垃圾收集对对象位置的改变，也要及时地让用户程序感知。

由于渐进和并发垃圾收集的主要目标是减少垃圾收集给用户程序所带来的暂停时间，这种技术通常被使用在对响应时间有要求的应用中（如实时系统）。对渐进和并发的垃圾收集技术的深入研究并不在本文的工作范围之内，若要了解更多更详尽的信息，请参阅[Jones96]。

1.7 并行垃圾收集

过去大量的有关垃圾收集的研究都把注意力集中在了单处理器上。然而，随着现代微处理器技术的发展，多处理器的计算机得到了越来越广泛地应用。多核的桌面计算机在市场上已经相当普遍，大规模的多处理器机也在服务器端被大量地部署，而且可以预见，这个趋势会持续发展下去。这使得不少的研究人员已经把对垃圾收集研究的注意力转移到多处理器的平台上来。

与并发的垃圾收集不同，并行的垃圾收集不是指垃圾收集与用户程序之间的并行执行，而是指多个垃圾收集线程之间的并行。在 stop-the-world 的垃圾收集中，人们很自然地会想到，在多处理器平台上，如果只运行一个垃圾收集线程，则其他的处理器资源将

会被浪费。虽然并发的垃圾收集能够缓解这一问题，因为从理论上说，在并发的垃圾收集中，可能没有一个全局的暂停周期，但是，标准的并发垃圾收集只有一个垃圾收集线程，而在多处理器的机器上，仅有一个专门的垃圾收集线程往往不能够满足多个用户线程的分配请求。因此，并行的垃圾收集仍然是非常有必要的，其追求的目标是可缩放性（scalability），即垃圾收集过程的加速比（speedup）应该和可用处理器的个数成比例。图 1.5 显示了并行垃圾收集与普通的串行的 stop-the-world 的垃圾收集的区别。

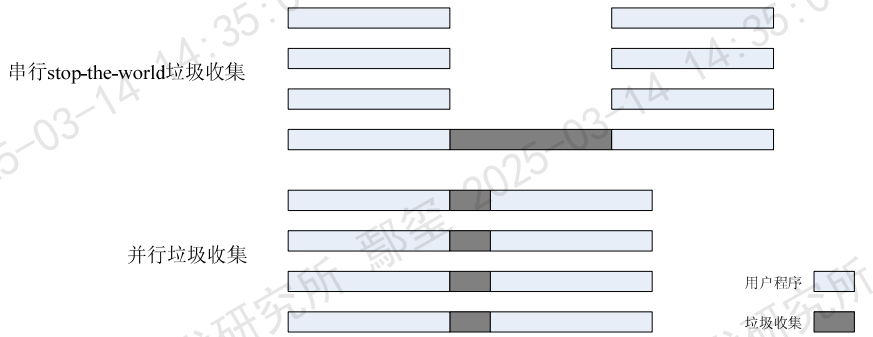

图 1.5 并行垃圾收集与串行垃圾收集的示意图

1.8 Java 虚拟机中的垃圾收集

1.8.1 Java 语言及 Java 虚拟机简介

Java 语言是一种面向对象的类型安全的语言[LY96]。由于它具有平台无关性，安全性和网络移动性等特点，使得它在软件工程上具有极大的优势。Java 语言的编译器首先将 Java 高级语言编译成一个通用的中间语言，即字节码（bytecode）。由该中间语言所形成的文件被称为类文件（class file）。类文件只能够在 Java 虚拟机上执行。这意味着，从理论上说不论底层的硬件平台是基于何种体系结构的，只要在该平台上部署了 Java 虚拟机，则任何 Java 程序都能够在该平台上顺利地运行，因此，Java 程序具有非常好的可移植性。尽管从实际上来看，Java 的平台无关性仍然依赖于特定的 Java 虚拟机，但是，SUN 公司制定并推出了一套标准的 Java 虚拟机规范。这套规范将 Java 虚拟机的实现和 Java 语言分隔开来，也就是说，只要 Java 虚拟机的实现，Java 程序所使用的语言特性和编译 Java 程序所使用的 Java 编译器都符合通用的虚拟机规范，则该 Java 虚拟机就能够很好地支持该 Java 程序。

Java 虚拟机就象是一台抽象的堆栈式计算机，它的主要任务是装载类文件并执行其中的字节码，字节码的操作数都来自于堆栈。由于目前大部分正在被使用的处理器都采用的是寄存器结构，因此，虚拟机需要把这些对堆栈的操作解释或翻译成寄存器机上的指令，这主要通过解释器或即时编译器（just-in-time compiler）来完成。解释器的执行方式是一次解释执行一条字节码，对解释后的机器指令并不作保存。如果该字节码被执行多次，则需要反复地解释执行。而即时编译器的执行方式则与之不同。在这种执行模式

下，第一次被执行的方法[1]会被编译成本地代码，并且该本地代码会被缓存在内存中，当该方法再次被调用的时候可以重用。这在一定程度上加速了 Java 程序的执行。

然而，由于即时编译的开销是算在 Java 程序运行的总时间里的，因此，即时编译的运行速度也影响着 Java 程序的性能。通常来说，复杂的编译优化能够使编译出来的代码性能提高，但是编译自身的开销却会增加，选择何种编译技术需要仔细地权衡。目前大部分高性能的 Java 虚拟机都采用一种自适应的优化技术。在这种方法里，初始的 Java 方法采用解释的方式或是运行速度很快，优化级别很低的编译方式来执行。在运行的过程中，虚拟机监视程序的活动情况，并记录下使用非常频繁的代码段，然后在适当的时机，对这些热点的代码重新进行更复杂更高级的编译。

Java 虚拟机支持动态类装载。这种支持动态装载的类装载器结构使得 Java 具有更灵活的网络特性。Java 虚拟机不仅能够从本地机器上装载类文件，也能够下载并装载网络上的类文件。然而这种灵活的网络特性也给 Java 虚拟机的安全性提出了更高的要求，因为网络上存在许多恶意的代码，Java 虚拟机必须能够防止这些恶意代码对本地系统造成影响。实际上，Java 虚拟机通过将类装载器作为名字空间的一部分来实现这种安全的管理。不同的类装载器装载的类属于不同的名字空间，这样，虚拟机可以按照名字空间将代码归入不同的类型（称为保护域），并确定某类型的代码可以进行的操作。

作为一种类型安全的语言，Java 语言还具有许多内置的安全特性：

- 类型安全的引用转换；
- 结构化的内存访问（不能使用指针来访问任意的内存地址）；
- 自动内存管理（垃圾收集）；
- 数组边界检查；
- 空引用检查。

这些安全的特性使得程序员更容易避免编程时可能引入的错误，从而使得用 Java 语言进行软件开发变得非常方便和高效，特别是对于大规模的服务器端的应用来说更是如此。

1.8.2 虚拟机与垃圾收集之间的交互

现代大部分的 Java 虚拟机都会使用 stop-the-world 的垃圾收集。尽管一些高级的产品级虚拟机中也会有对并发垃圾收集的支持，但是一般情况下，stop-the-world 的垃圾收集都会被作为缺省的配置。由于并发的垃圾收集技术超出了本文讨论的范围，本节将主要描述基于 stop-the-world 的垃圾收集与虚拟机之间的交互。

由前面的讨论，垃圾收集的正确性依赖于对象的存活性理论，即对象的可达性。当垃圾收集发生的时候，系统挂起正在运行的用户程序，然后从根集出发，顺着指向对象的引用遍历堆中的对象。然而，垃圾收集是如何知道哪些位置存放的是引用的呢？对于 C 语言的运行时环境来说，由于常规的编译器都没有对垃圾收集有任何的支持，因此，

[1]在 Java 语言的术语系统中，函数被称为方法。

要在这样的系统中使用垃圾收集就必须使用一种叫做保守式的垃圾收集技术[Boehm88, Boehm93, Appel89a, Goldberg91, Edelson90]。这种技术需要对一个内存字做出一定的假设，假设该字是引用或者不是引用，这种估计通常会由于要保证正确性的需要而变得非常保守。

幸运的是，Java 语言作为一种强类型的语言，在即时编译器的支持下，垃圾收集可以精确地找到所有引用所在的内存位置[Agesen97, Agesen98a]。有两种基本的方法用来支持精确的垃圾收集：标记（tagging）和引用图（reference map）。标记的方法是在每一个字宽的数据项中保留一位或多位来标识该字是否是一个引用。这种方法并不适用在 Java 编程语言中，因为 Java 语言要求有 32 位和 64 位的整数和浮点数。

对于引用图的方法来说，有两种形式的引用图：对象引用图和栈帧引用图。对象引用图表明了某个类的对象的哪些域是引用域，这个信息被保存在该对象的类结构中。栈帧引用图则标明了用户程序的栈帧和当前寄存器中的哪些项是引用。与对象不同，栈帧中的引用的布局会随着程序的运行而改变。这意味着程序运行的每一点可能都需要为之保存相应的栈帧引用图。为了减少存储空间上的开销，栈帧引用图通常只在某些固定的程序运行点才会被保存，这些点被称为垃圾收集安全点（GC safe point or GC point）。这意味着垃圾收集只有在用户程序运行到 GC 安全点的时候才能够被发起。

对于单线程的环境来说，GC 安全点的机制工作良好。有两个运行点需要被设为 GC 安全点：对象分配点和方法调用点。对象分配点可能会引发垃圾收集，在此设置 GC 安全点是为了保证垃圾收集发生的时候当前栈帧的引用图是可用的。而在方法调用点设置 GC 安全点是为了得到更深层的栈帧的引用图。当线程在分配对象时由于内存资源不足而引发垃圾收集的时候，该线程就自动地处在了 GC 安全点了。

然而，在多线程的环境里，情况就没有这么简单了。如果一个线程在分配对象的时候需要引发垃圾收集，则该线程正好处在 GC 安全点，但是，其他的线程是否恰好到达了 GC 安全点却是未知的，因此必须有一种同步机制使得所有的线程都能在 GC 安全点被挂起。有两种方法可以用来解决这个问题：轮流检测（polling）和代码修补（code patching）。

轮流检测是在即时编译器产生代码的时候，在 GC 安全点插入一小段代码来检测垃圾收集是否发生。判断垃圾收集是否发生的条件可以用全局变量来控制。例如，当一个线程引发垃圾收集的时候，该线程将一个布尔型的全局变量 gc_stop_threads 设置为真，其他的用户线程在每次到达 GC 安全点的时候都要去检测这个全局变量的值。如果该值为真，则该用户线程就会将自己挂起并通知引发垃圾收集的线程。这样，即使垃圾收集没有发生，每个线程在运行到 GC 安全点的时候都需要增加一次访存和一次比较。为了减少比较分支带来的代价，产品级的 Java 虚拟机通常使用例外的机制来代替比较[1]。例外的机制依赖于使用一个被保护的页。在没有发生垃圾收集的时候，该页的访问权限被

[1] 基于例外机制的轮流检测技术已经被应用在 SUN 公司开发的 JDK1.5 中。

设为可读，每个线程在到达 GC 安全点的时候只需要对该页内的某个地址做一次没有实际效果的访存（在 MIPS 的体系结构中，可以使用“lw r0, 0(rx)”指令来实现，其中 rx 表示装有保护页内某地址的寄存器）。当某个线程引发垃圾收集的时候，该线程将保护页的访问权限设为不可读，则其他的线程到达 GC 安全点的时候就会发生例外，然后，该线程在例外处理程序里将自己挂起。

代码修补的方法能够完全避免对用户程序正常执行路径的干扰。即时编译器产生用户线程代码的时候无需插入额外的检测代码。当用户线程引发垃圾收集的时候，该线程使用一个分支指令来替换所有方法中的 GC 安全点的指令。该分支指令一般是跳转到一段“蹦床”（trampoline）代码中，然后在这段“蹦床”代码中完成相应的挂起操作。在垃圾收集完成之后，收集线程再将所有方法中的 GC 安全点的指令恢复成原来的状态。与轮流检测相比代码修补的方法可以提高程序的运行速度，但是增加了代码在空间上的开销，而且实现起来比较麻烦，容易产生错误[Agesen98b]。

在多线程的环境中，另外一点需要注意的就是，仅仅用对象分配点和方法调用点来做 GC 安全点可能是不够的，因为当垃圾收集发生的时候，某个线程可能正处于一个需要运行很长时间的循环中，这样就可能会造成该线程迟迟不能被挂起而导致垃圾收集一直不能开始工作。因此，在一个循环中如果没有对象分配和函数调用的话，就需要把该循环的回边[1]（back edge）所在的位置设置成为 GC 安全点。

1.5 本文的主要贡献

本文系统深入地研究了 Java 虚拟机中的内存管理机制，其主要的创新和贡献如下：

第一，设计并实现了一个支持灵活的运行时切换垃圾收集算法的内存管理软件框架——RSMTk。该框架利用 Allocation Area 的概念来使不同的收集算法共用相同的对象分配函数，从而避免了在收集算法切换过程中由于分配函数的内联而引起的方法重编译。比较了不同的垃圾收集算法对应用的对象局部性的影响。比较了深度优先拷贝序，广度优先拷贝序和保持对象分配序的收集算法对对象局部性的影响。并且比较分析了在拷贝垃圾收集中，分代内的不同分区间的拷贝序和不同分代间的拷贝序对对象局部性的影响。结果表明，分代内的不同分区间的对象拷贝序是影响对象局部性的主要因素。构建了一个结合预取和局部性优化技术的软件框架，结果发现基于规则的数组结构的预取更容易取得性能的改进。调查了 on-chip 的 local memory 在 Java 应用中对内存系统性能的影响。结果表明，on-chip local memory 和 cache 相结合的系统能够比 cache-only 的系统具有更优的性能。

第二，分析了服务器端 Java 应用中的对象生命周期行为，发现其中存在着大量长生命周期的对象。本文的工作设计并实现了一个混合垃圾收集，利用几种经典收集算法的运行时动态切换及其相互的协作来减少垃圾收集对长生命周期对象的重复操作。我们使

[1] 回边是指一条向前跳转的分支指令，在编译生成的机器代码中，这种回边指令通常意味着一个循环的出现。

用 SPECjbb2005 对该算法进行了评估。实验结果表明，与经典的分代垃圾收集相比，我们的混合收集算法能够使 SPECjbb2005 的 Throughput 分值提高 3%~25%。

第三，设计并实现了一种新的并行垃圾收集标记算法——task-pushing。该算法基于 CSP 的风格，通过将标记过程构造成一个处理网络来避免对同步原语的依赖同时获得良好的动态负载平衡。为了支持这种无需同步操作的并行标记算法，我们设计了一种高性能的通信队列，该队列支持单个读者和单个写者，利用现有硬件所支持的字对齐访问的原子性来避免对同步原语的使用。我们在 Harmony[1]的平台上实现了 task-pushing 的算法，并在一个 16-way Intel Xeon multiprocessor 硬件平台上对 task-pushing 算法进行了评估。为了和现有的算法进行比较，我们也实现了 work-stealing 的算法，该算法是我们所知的引入同步原语最少、动态负载平衡最灵活的算法。实验结果表明，与 work-stealing 技术相比，task-pushing 所带来的性能好处会随着处理器个数的增加而变得越来越大。对于 pseudojbb 和 GCOld 这两个服务器端的基准测试程序来说，当处理器的个数为 16 时，task-pushing 的加速比（speedup）比 work-stealing 分别提高了 17.4%和 13.5%。

1.6 论文的组织

本文的组织如下：

第一章，简要地介绍了内存管理的历史，讨论了自动内存管理所具有的优势。描述了垃圾收集的可达性原理，回顾了几种经典的垃圾收集算法，并简要地讨论了 Java 虚拟机与垃圾收集之间的交互。

第二章，从垃圾收集对对象局部性的影响，垃圾收集的动态机制和并行垃圾收集技术这三个方面综述了垃圾收集的相关工作。

第三章，介绍了本文工作中所搭建和使用的实验平台。描述了一个灵活的算法可切换的内存管理软件框架。

第四章，详细地评估了垃圾收集对对象局部性的影响。对不同的垃圾收集算法和不同的对象拷贝序进行了比较，描述了一个预取和局部性优化结合的软件框架，调查了 on-chip 的 local memory 对内存系统性能的影响。

第五章，分析了服务器端 Java 应用的对象生命周期行为，描述了一种针对对象生命周期行为特点的允许算法动态切换的混合垃圾收集算法，并对该算法进行了性能的评估。

第六章，描述了一种新的可缩放的并行垃圾收集标记算法。该算法通过使用一种无需同步支持的高性能的通信队列来进行动态的负载平衡，从而在并行标记过程中避免了对同步原语的依赖。

第七章，对当前的工作进行了总结，并展望了将来可能的研究工作。

[1] Harmony 平台是 Apache 软件基金会的一个开源 Java 虚拟机项目。

中国科学院计算技术研究所 鄢玺 2025-03-14 14:35:00

第二章 垃圾收集相关工作的综述

程序设计语言的发展史可以看作是其为抽象提供不断强化的支持以及将原本需要手工或显式完成的工作自动化的过程[Jones96]。在冯·诺依曼的体系结构中，程序设计语言和内存管理是密不可分的。同程序设计语言一样，内存管理也经历着一个从显式到自动的发展过程。

在使用机器语言和汇编语言的年代中，程序员必须对代码和数据的内存布局有一个清晰的把握，程序员必须密切关心代码和数据所存放的内存位置。高级语言的出现在一定程度上减轻了程序员的负担。对静态数据和栈数据的分配是在编译器的帮助下完成的，而对堆数据的分配则由程序员通过显式地调用库函数来实现。

然而，这种由程序员显式地分配和释放堆内存的做法仍然比较容易引入内存错误。垃圾收集技术（即自动内存管理）的出现进一步缓解了这一问题。在使用垃圾收集的系统中，程序员不再需要关心何时应该释放不用的内存，从而不会因为错误地释放了某个仍在被使用的数据而带来某种灾难性的后果。

垃圾收集技术最早被使用在 Lisp 编程语言中[McCarthy60]，之后又有一些语言开始使用这项技术。如今，面向对象的语言由于它的抽象性，模块化及代码重用等优点而变得越来越流行。大部分的面向对象的语言都使用了垃圾收集，即使是像 C++这样起初并不支持垃圾收集的高级语言，现在也引入了保守的垃圾收集技术。尽管作为一项很古老的技术，垃圾收集曾经被大量的研究者们广泛而深入地调查和研究过[Appel89b, Deutsch76, Dijkstra76, Jones96, LH83, Ungar84]，但是，内存管理和编程语言的这种发展趋势，使得垃圾收集技术成为在工业界和学术界的研究者们持续争论的焦点[Blackburn02, Cheng01, SM03, Stefanovic99, Blackburn04a]。特别是 Java 语言的出现及其日益增加的受欢迎的程度，进一步推进了这个趋势。这不仅仅是因为 Java 语言使用了垃圾收集，更重要的是 Java 语言还把对垃圾收集的使用作为其语言规范的一部分[Gosling96]。

2.1 垃圾收集对运行时系统（runtime system）的影响

垃圾收集对运行时系统的影响主要包括以下几个方面。首先，垃圾收集本身的运行开销是算在应用程序的整个运行时间之内的。即：

$$Runtime_{App} = Runtime_{Mutator} + Runtime_{GC}$$

因此，一个高效的垃圾收集系统应该尽可能地减少垃圾收集自身的运行时间，即提高垃圾收集的效率。

第二，由于应用程序会改变内存中对象的连通状态，而垃圾收集的正确性又依赖于

对象的连通状态，因此，大多数的垃圾收集算法都需要先将应用程序暂停下来（stop-the-world），然后才能进行垃圾内存的回收。这意味着垃圾收集会使应用程序产生停顿（pause）。对于某些实时或交互式系统来说，这种停顿可能是不能够忍受的。因此，好的垃圾收集应该尽量减少暂停时间。

分代，渐进和并发的垃圾收集算法能够在一定程度上减少暂停时间，但是，减少暂停时间也要付出一定的代价。分代垃圾收集通过频繁的次级收集来减小每次垃圾收集的时间开销，然而对用户程序的频繁的挂起和恢复本身就会带来额外的开销，特别是当用户线程的数量远大于可用处理器的数量的时候，情况可能会变得比较严重。

并发的垃圾收集为了保证算法的正确性，通常会引入大量的同步，在多处理器上，这些同步可能会对用户程序的性能造成影响。而在单处理器上，并发的垃圾收集还会增加线程切换的开销。因此，减少暂停时间和提高整个应用的性能是需要权衡的两个目标。

第三，垃圾收集对用户程序（mutator）的性能会产生一定的影响。有的垃圾收集需要和用户程序进行配合，例如，分代垃圾收集需要用户程序提供写拦截来记录代间指针。因此，高效地实现写拦截是垃圾收集需要考虑的一个主要方面。

内存分配的效率也在很大的程度上影响着用户程序的性能，特别是对于面向对象的程序来说，对象的分配是一个性能关键的操作。而垃圾收集的算法决定了分配函数的实现，因此垃圾收集算法应该尽可能地提供更高效的内存分配机制。

另外，不论是对象的分配还是回收，这两个垃圾收集的主要功能都决定着对象在堆中的布局，从而影响着对象的局部性。对于不会移动对象的垃圾收集来说，能够影响对象局部性的唯一操作就是对象的分配。在编译器或 profile 的支持下，分配函数可能能够识别哪些对象更应该被分配在相邻的区域里，从而改善对象的局部性。而可以移动对象的垃圾收集则拥有更多的机会来改善局部性，因此，通常来说，可以改变对象布局的垃圾收集算法比不能移动对象的垃圾收集具有更多的优势。

2.2 垃圾收集与对象局部性

随着处理器的运算速度和访存速度的差距日益增加，数据的局部性正在变得越来越重要。当前的体系结构利用 cache 来缓解内存和处理器之间的性能差距。Cache 主要通过两个因素来达到这个目的：时间局部性和空间局部性。时间局部性意味着最近被访问的数据很快又将被访问；空间局部性则意味着空间上存储在一起的数据容易一起被访问[HP02]。然而，面向对象的程序很可能并不具备良好的空间局部性，这主要是由面向对象的程序特点来决定的。

从实现的角度来看，一个对象就是包含着对象信息的一块存储空间。程序从一个对象访问到另一个对象是通过对象之间的指针来完成的。换句话说，对象之间的组织形式更象是链表，树或者是图的数据结构。在这种组织形式下，一个对象不能保证和它的后继对象放在相邻的位置，从而导致了较差的空间局部性。这种特点也并不是面向对象的

程序所独有的，任何采用动态堆内存分配并使用指针来访问数据项的语言都面临着类似的问题。而像 Fortran 这样的主要使用数组结构的程序语言来说，当程序具有规则步长的访问模式的时候，通常会表现出良好的空间局部性。

许多研究者已经对垃圾收集对对象局部性的影响进行了大量深入的研究。一些研究者认为，可以通过在垃圾收集的时候对对象进行群聚（clustering）来改善对象的局部性[Clark77, Clark79, White80, Stamos82, Blau83, Stamos84, Andre86, Hayes91]。群聚的策略分为静态和动态两类。在静态的群聚策略中，垃圾收集基于对象的连通图，选择一种静态的拷贝序，而动态的群聚策略则是基于对象的动态访问模式来选择对象的拷贝序。这些早期的研究主要关注于垃圾收集系统中页级的行为，因为当时的物理内存还是非常稀缺的资源，页故障带来的延迟常常成为影响程序性能的一个主要因素。

Cheney[Cheney70]的拷贝算法是一个经典的节点复制垃圾收集算法，该算法采用了宽度优先的遍历序来拷贝对象。为了更好的理解该算法，我们先来介绍一种抽象方式：三色抽象。

黑色：意味着该对象和它所有的直接后继都已经被访问过了：垃圾收集已经完成了与它相关的工作，不再需要访问它了。

灰色：这些对象已经被访问过，但是还有没有被访问过的直接后继。这些对象还需要再次被扫描以访问它们的后继对象。

白色：尚未被访问过的对象。

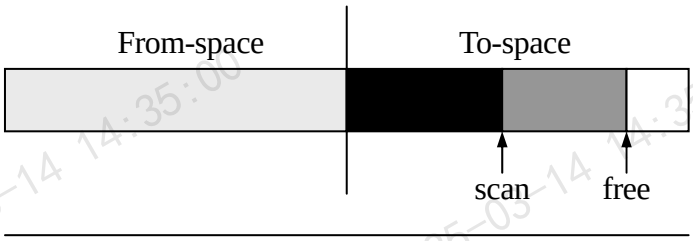

图 2.1 Cheney 的拷贝 GC 算法

如图 2.1 所示，在 Cheney 的拷贝垃圾收集算法中，内存被分为两个半区：from-space 和 to-space。当垃圾收集发起的时候，堆中所有的对象都在 from-space 中，而整个 to-space 都是空的，scan 指针和 free 指针都指向 to-space 的起始位置。垃圾收集首先扫描根集中的引用，将它们所指向的对象从 from-space 拷贝到 to-space 中。被拷贝的对象在 to-space 中被标记为灰色，并且 free 指针始终指向灰色对象区和自由空间的边界（在图 2.1 中，自由空间为白色）。接下来，收集器扫描已经被拷贝的对象，寻找这些对象中指向 from-space 中的对象的引用，并且把这些引用所指向的对象也从 from-space 拷贝到 to-space 中来。如果 to-space 中的某个对象的所有引用域都被扫描过了，则该对象被标记成黑色，并更新 scan 指针，继续扫描下一个灰色的节点。Scan 指针总是指向黑色对象区和灰色对象区的边界。当 scan 指针赶上 free 指针的时候，收集器已经复制完所有堆中的

对象，两个半区互换角色之后，垃圾收集算法结束。

Cheney 的拷贝算法曾经被广泛地使用，因为它不需要额外的栈或队列结构来实现宽度优先的遍历。[Moon84]认为在具有大的虚存空间的系统中，数据的页级的局部性非常重要，而垃圾收集应该能够用来改善数据的局部性。他指出，一个近似深度优先的拷贝序能够改善数据的局部性，因为深度优先的拷贝序倾向于把具有父子关系的数据对象放在同一个页面中。当数据对象的组织结构类似于深度很浅的树结构的时候，深度优先的拷贝序确实可以带来不少的好处。

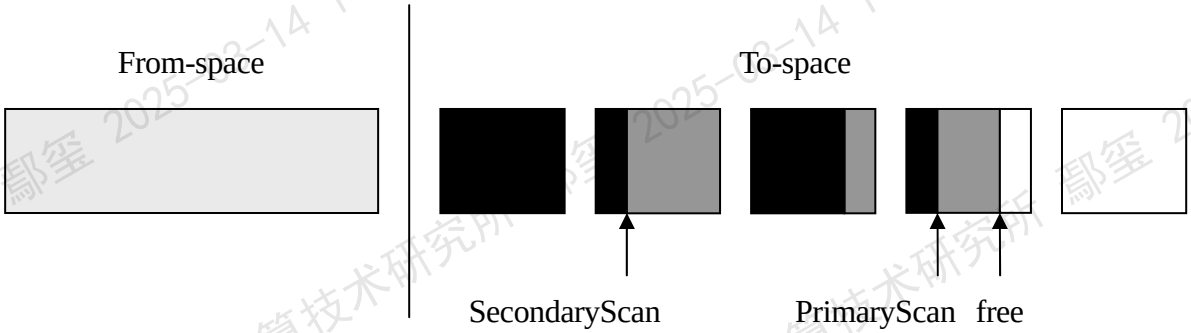

图 2.2 Moon 的近似深度优先拷贝算法

Moon 改进了 Cheney 的算法，图 2.2 描述了该算法的过程。To-space 被划分为若干个块，一般来说块的大小等于系统中页面的大小。对象仍然是被拷贝到 free 指针所指向的位置，free 指针始终是灰色对象区和自由空间的边界。与 Cheney 算法不同的是，Moon 的算法使用两个 scan 指针。Primary scan 指针总是和 free 指针在相同的块中，并指向该块中黑色对象区和灰色对象区的边界，Secondary scan 指针记录着最早的灰色对象区和前面的黑色对象区的边界。Secondary scan 指针之前的对象都已经被标记为黑色。扫描算法总是优先扫描 primary scan 指针所指向的灰色对象。当 free 指针越过一个块的边界到达下一个块的时候，primary scan 指针立刻指向下一个块的起始位置，而不管上一个块中是否还有灰色的对象，接着，扫描过程又从下一个块开始。当 primary 指针赶上 free 指针的时候，Moon 的算法又会去扫描 secondary scan 指针所指向的灰色对象，并将这些灰色对象所指向的在 from-space 中的对象拷贝到 free 指针所指向的位置。当 free 指针又前进，primary scan 指针指向的位置又有灰色对象的时候，扫描算法又会开始扫描 primary scan 指针所指向的对象。当 secondary scan 指针越过当前的块，指向下一个块的起始位置的时候，它首先要扫描该块中黑色的对象并忽略它们，直到遇到灰色的对象。当 secondary scan 指针赶上 free 指针的时候，对象扫描完毕，垃圾收集算法结束。

Moon 的算法实际上存在两个层次上的遍历序。在块的内部是基于 Cheney 的宽度优先遍历，而在块间则是遵循着深度优先的遍历序。

[Stamos84]描述了通过对对象进行静态群组（grouping）来减少缺页故障的方法。他发现不论是深度优先遍历序还是广度优先遍历序都比随机的遍历序具有更好的局部性，

因而会产生更少的缺页故障。Andre 在他的工作中发现对于 Lisp 系统来说，保持对象初始的创建顺序比他们所尝试过的任何遍历序都会带来更好的局部性[Andre86]。Courts 在一个 Lisp 系统中实现了一个改进局部性的垃圾收集器[Courts88]。他指出，在垃圾收集系统中，糟糕的引用局部性会导致产生大量的页故障。他们的垃圾收集器采用了一种自适应的动态重组的方法，利用特殊的硬件来支持渐进的垃圾收集。他们的收集器经常会按照程序访问对象的真实顺序来拷贝对象，因此会带来更好的局部性。他们的算法能够将页故障的等待时间降低为原来的 1/4，或者在不影响运行速度的情况下，可以允许减少一半的内存需求。然而，他们所使用的动态重组技术在普通的硬件上仍然有着相当昂贵的代价。[Baker91]描述了拷贝垃圾收集中影响 cache 性能的若干因素，并建议通过在垃圾收集的时候使用 cache 旁路或使用独立的处理器的方法来保持用户程序的局部性。

Wilson et al. 提出了一种被称为层次分解（hierarchical decomposition）的静态对象重组的方法来改进内存页级的局部性[Wilson91]。他们发现使用一个两级的队列来改进 Cheney 的扫描算法能够更有效地组织对象。图 2.3 示意了 Wilson 的层次分解算法。

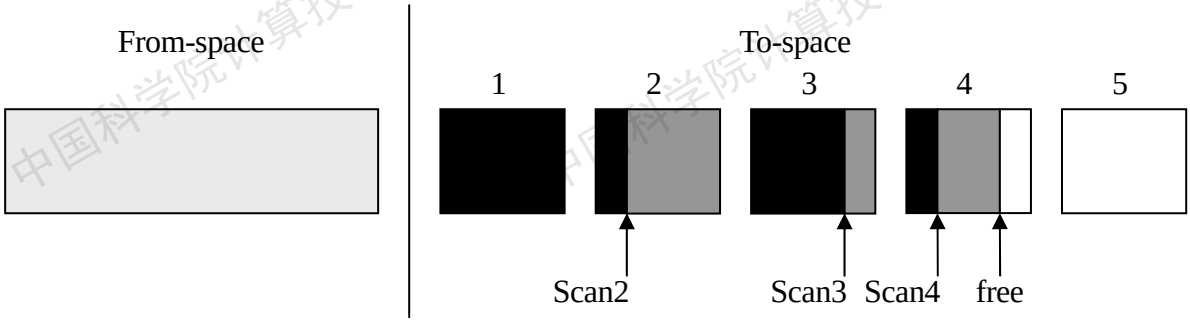

图 2.3 Wilson 的层次分解拷贝算法

Wilson 的算法与 Moon 的近似深度优先的算法有一些相似之处。与之不同的是，它记录了所有被部分扫描过的内存块的 scan 指针（如图 2.3 中的 scan2, scan3, scan4）。该算法也是优先扫描 free 指针所在的内存块。当 free 指针指向当前块的末端的时候，算法会去扫描最靠前的 scan 指针，并将相应的 from-space 中的对象拷贝到下一个块中，接着将 free 指针指向该块的自由空间与灰色对象区的边界，然后继续从该块开始扫描。

与 Moon 的算法相比，Wilson 的算法有一个优点，就是不用重复地扫描被部分扫描过的块中的黑色对象。该算法也存在两个层次上的遍历序，在块的内部是基于 Cheney 算法的宽度优先遍历，而在块间则是另一个层次上的宽度优先遍历序。Wilson 的算法按照层次的结构来安排对象的布局，他们的实验说明了这种布局更能够把相关的对象放在相邻的区域，从而带来更好的局部性。另外，Wilson 还将哈希表（hash table）进行了特殊的处理。因为在层次分解的算法中，哈希表通常会导致把不相关的对象群聚在一起而带来比较糟糕的局部性。

在后来的研究中，Lam et al.发现层次分解也并不总是最优的方法[Lam92]。层次分解

仅仅对于那些看起来像平衡二叉树的数据结构并且行为类似于树的遍历的程序才会比较有效。他建议对具有不同遍历模式的程序应该使用不同的群组策略，并且特别支持那些利用对象的类型信息来确定适合的群组模式的方法。

在最近的研究中，Siegwart 和 Hirzel 结合了层次分解算法和并行的垃圾收集技术，提出了一种新的并行层次拷贝的垃圾收集算法（parallel hierarchical copying GC）[Siegwart06]。该算法的主要动机是在追求并行垃圾收集的高可缩放性的同时，获得更好的 mutator 的数据局部性。这种算法的主要思想是将 to-space 中的内存块既用来作为并行处理的工作单元，同时也用来作为层次拷贝的分解单元。他们的算法的成功得益于每个内存块都有各自独立的 scan 指针。

他们的工作的另一个贡献是对层次分解的拷贝算法进行了详细的调查。他们的结果说明层次分解的拷贝算法不仅能够在页级，同时也能在 cache 级获得良好的局部性。

Calder et al. 将一种为指令 cache 而开发的布局技术应用到对数据对象的布局上来[Calder98]。他们使用一种编译器制导（compiler-directed）的方法为局部变量，全局变量，堆对象和常数创建地址布局，以此来减少数据 cache 的失效次数。他们方法需要一次特殊的训练遍（training pass）来收集 profile 信息，并利用收集到的信息来指导对数据的布局。尽管这种布局的过程是全自动的，但是可能会用很长的时间来分析和计算。他们的方法对堆中对象的局部性没有太大的改进，但是对局部变量和全局变量的局部性却有较为明显的改善。

Chilimbi 和 Larus[CL98]描述了如何利用分代拷贝（Generational-copy）的垃圾收集来改变对象的布局，从而改善对象的局部性。他们使用了一种运行时的 profile 机制来获得对象的访问模式。编译器在每个装载对象基地址的访存指令的后面插入一小段代码。这段代码会把当前被装载的对象基地址存入一个对象访问缓冲区（object access buffer）中。这个缓冲区是一个顺序的结构，在它的末端是一个访问权限被设置成了不可写的守护页。当对象访问缓冲区被填满的时候，后继的写入操作会触碰到守护页从而引起例外的发生，然后由例外处理程序来对缓冲区中的对象访问序列进行分析。

运行时的 profile 可以使程序员避免额外的 profile 执行周期，即无需特别的训练遍（training pass），同时，也省去了寻找有效的训练输入集的麻烦。然而，运行时的 profile 必须有足够小的运行时开销，否则，性能的改进就会被 profile 的开销所抵消。Chilimbi 的运行时 profile 具有较小的开销，这主要得益于面向对象程序的两个特点：

1. 大部分对象的大小都很小；
2. 大部分对象的访问都不是轻量级的；

如果对象的大小很小，则区别相同对象中的不同的域就没有太大的必要，因为一个 cache 行通常能够容纳好几个对象，因此，profile 能够实现在对象粒度而不是域的粒度上。另外，如果大部分的对象的访问不是轻量级的，也即对象的基地址被装载一次以后，该对象的多个域会一起被多次地访问。这意味着仅仅拦截对对象基地址的装载可能不会带来很大的开销。

当由于对象访问缓冲区被填满而引发例外的时候，例外处理程序分析缓冲区中的对象访问序列，计算对象之间在访问时序上的亲缘关系（即哪些对象会在时间上被连续地访问），并构造一个亲缘关系图以供垃圾收集使用。当拷贝垃圾收集发生的时候，Chilimbi 使用了一种贪心的算法，以亲缘关系为权重，深度优先地遍历对象亲缘关系图，并按照遍历的顺序将对象拷贝到目标空间中去。他们的实验使用一种叫做 Cecil [Chambers93] 的动态类型的，纯面向对象的语言环境。实验结果表明，他们的方法能够减少 21-42%的 cache 失效次数，对程序的总体性能有 14-37%的提高，并仅带来了 3-6%的 profile 的运行时开销。而且，他们还与 Wilson 的层次分解拷贝算法进行了比较，结果减少了 cache 失效次数 20-41%，提高了程序的性能 18-31%。

在后来的研究中，Chilimbi et al. 又把目光转向了对象内部结构的布局上[CDL99]。他们提出了两种通过改变对象内部结构来改善数据局部性的技术：结构分裂（structure splitting）和域重排（field reordering）。与依赖于对象亲缘关系的技术不同，这种技术主要针对于更大型的数据结构。对于与 cache 行大小相当的数据结构来说，通过把对象的域按照冷热程度[1]划分为两个部分，能够使一些数据重组的技术仅应用在热的域上，从而改善程序的 cache 行为，因为对象结构中热的部分与 cache 行相比还是很小的。

而对于比 cache 行大小大得多的数据结构来说，他们使用域重排来把那些时序上亲缘关系比较大的域放在相邻的位置上，从而增加 cache 行的利用率。然而他们的技术并不是完全自动化的，而是需要程序员的大量的手工干预。

Shuf et al. [Shuf02]提出了一个新的分配方案来改进对象的局部性。他们使用 prolific 类型的概念。Prolific 类型就是经常被实例化的类型。他们的分配方案尝试把 prolific 类型的对象放在一起。然而，他们的方法基于静态的，离线的（offline）profile 信息。虚拟机和运行时环境并不会通过主动地去观察数据的动态访问序列来决定应该如何调整对象在内存堆中的布局。这样，他们的方法所产生的对象布局就有可能不能反映真实的数据访问模式，而且，他们的方法也不可能检测到程序相（phase）的改变，并及时做出相应的调整。

Huang et al. [Huang04] 使用了一种被称为在线对象重排（online object reordering）的技术。该技术通过采样来识别热的方法、热方法中所使用的热的域以及这些域的类型。在垃圾收集的时候，收集器把这些热的域所引用的对象和他们的父对象拷贝到一起。由于他们所使用的虚拟机本身拥有内建的采样机制来识别热的方法，用以进行自适应的编译优化，因此他们利用了这种现有的机制来减小采样的开销。与 Chilimbi 的技术相比，在线对象重排使用的是基于类型的较粗粒度的 profile 信息，因此信息的准确度相应的会受到一些影响。

在最近的研究中，Chen et al. 指出对局部性的优化不应该只在垃圾收集的时候才被动地去做[Chen06]。他把局部性优化的启动条件同垃圾收集分离开，利用对象分配率，TLB

[1]冷热程度是指数据被访问的频繁程度。

和 cache 失效率等参数来决定何时发起局部性优化。另外，他还把对 cache 的局部性优化和对页面级的局部性优化进行了组合。他们的结果表明把垃圾收集看作是一种显式的局部性优化机制而不仅仅是为了满足内存的需求的做法是有好处的，而且组合页级和 cache 级的局部性优化也会给系统带来更大的好处。

一些研究者将垃圾收集和预取技术结合起来，使用垃圾收集来辅助预取的实施。从某种意义上来说，预取和局部性优化是两种正交的技术。预取解决的是访存瓶颈的表象问题，即容忍访存延迟；而局部性优化则主要是用来解决访存问题的根源，因为理想的最优的局部性应该能够最大限度的减少访存的通信量。然而，从另一方面讲，预取和局部性又不无关系，空间局部性本质上也具有预取的特点。因为当访问内存中一个 cache 行的某个字的时候，该 cache 行中的其他的字都会一并被装载到 cache 中去。因此，对空间局部性的优化，也可以被看作是充分地利用这种隐式的预取。

下面我们就再来回顾一些与软件预取相关的研究。

Luk 和 Mowry[LM96, LM99]研究了一些针对递归数据结构[1]（recursive data structures, RDSs）的软件预取技术。他们的技术包括贪心（greedy）预取，历史指针（history-pointer）预取和数据线性化（data-linearization）。他们使用了一种非常简单的别名分析和有限的局部性分析来决定在什么时机采用什么预取方法。

Roth 和 Sohi[RS99]讨论了 Jump-pointer 预取。他们使用该预取方法来容忍链式数据结构所带来的访存延迟。有三种 Jump-pointer 预取方案被提出：queue jumping, chain jumping, 和 root jumping。这些方案都可以用软件方法、硬件方法或软硬结合的方法来实现。

Badawy et al. [Badawy01]针对规则的、不规则的科学计算和指针追逐（pointer-chasing）类型的程序，评估了内存带宽对软件预取和局部性优化的有效性所造成的影响。他们发现，对于许多应用来说，当内存带宽足够的时候，软件预取优于局部性优化，而当内存带宽受限的时候，情况恰恰相反。

Chilimbi 和 Hirzel[CH02]开发了一个针对具有大量指针操作的通用程序的动态预取方案。他们使用一个有限状态机来预测热数据流，该有限状态机表示的是程序中经常执行的地址流。他们的系统会动态地在适当的程序点插入发现和预取这些热数据流的代码。

Cahoon 和 McKinley[CM01]提出了一个有效的数据流分析技术来识别 Java 程序中对递归数据结构的遍历。他们使用过程内和过程间的分析来发现循环和递归函数中反复重现的指针变量，并用这种分析来驱动贪心预取和历史指针预取。

Inagaki et al. 为动态编译器设计了一种步长（stride）预取算法[Inagaki03]。他们的算法尝试去发现循环遍内（intra-iteration）和循环遍间（inter-iteration）的步长模式。并利用这种模式来插入相应的预取指令。

Adl-Tabatabai et al. [AHSS04]在即时编译器生成的代码中插入预取指令，并利用垃圾收集对对象布局的能力来维护对象的距离信息，以使预取能够有效地进行。他们利用硬

[1] 链表，树和图都是递归数据结构。

件性能监视器（hardware performance minitor）来收集 cache 失效的信息并确定发生 cache 失效的指令的位置，从而得到适当的预取指令插入点。

2.3 垃圾收集的动态机制

迄今为止，垃圾收集已经经历了 40 多年的历史了。在过去的研究中，有大量的不同的垃圾收集算法被提出，它们都有着各自的优点和缺点。对于不同的应用和资源环境，这些垃圾收集算法对运行时系统都有着不同的影响。即使是同一种垃圾收集算法，也有着灵活可变的参数和配置，而这些可变的参数也同样会给垃圾收集系统带来不同的影响。

垃圾收集是一个非常广阔的研究领域。由于它的算法和形态非常的复杂多变，这使得它的动态机制也十分的灵活多样。因此，对垃圾收集的动态机制作一个详尽的分类并不是一件十分容易的事情。在这一节里，我们对垃圾收集的动态机制作一个简单而粗略的分类，我们将从以下 3 个方面来总结前人在垃圾收集的动态特性上所进行的研究工作:

1. 基于参数的动态机制;
2. 基于算法的动态机制;
3. 基于年龄的动态机制;

2.3.1 基于参数的动态机制

当一个垃圾收集算法已经确定的时候，该算法通常可以通过改变一些参数的配置来改变自身的形态。对于一个非分代的垃圾收集来说，堆的大小是一个经常被讨论的参数。这个参数之所以重要，是因为它通常决定了垃圾收集发起的时机。垃圾收集通常是在内存分配请求不能得到满足的时候才被发起，此时，堆中的自由空间已经完全被耗尽。然而，垃圾收集的发起时机也并不总是由堆的大小唯一确定的，仍然存在许多特殊的机制，使得垃圾收集在最适合的时候被发起。因此，从某种角度上来说，垃圾收集的发起时机又是一个与堆的大小分离的参数。

分代垃圾收集按照对象的年龄将整个堆划分为不同的区域，这种设计扩展了垃圾收集算法的可变参数。在分代垃圾收集里，除了整个堆的大小以外，人们可能还需要考虑不同分代的大小，不同分代的收集所发起的时机，以及分代的个数等等。

[Moon84] 指出，在早期的 Lisp 语言机器中，由于物理内存十分有限，当虚存空间将要用尽的时候，垃圾收集通常会引起大量的页面的换入换出。在这种情况下，垃圾收集会使交互式应用程序的响应时间变得非常差。因此，用户通常更倾向于将垃圾收集关闭或者将系统重新启动。Moon 还指出，有些应用程序在垃圾收集关闭的的时候会运行得更快。

为了拥有良好的性能，Appel 建议一个程序的堆空间的占用率（堆空间的占用率是指存活对象的大小与堆空间大小的比例）应该低于 1/3[Appel89b]。在这个水平上，ML 语言的垃圾收集开销是 11%；如果占用率降到 1/7，则垃圾收集的开销会降为 6%。因此，当一次垃圾收集之后所计算出来的堆空间的占用率高于 1/3 的话，就应该扩展堆的大小。

然而，遗憾的是，Appel 仅仅描述了这种改变堆大小的机制，却没有详细地调查这种机制对程序性能的影响。

Alonso 和 Appel 实现了一个“advice server”[AA90]。“advice server”是一个用户级的进程，它被用来决定如何使一个分代拷贝的垃圾收集器（generational copying garbage collector）能够最好地利用系统中可用的内存资源。在每一次垃圾收集结束之后，用户进程和“advice server”进行通信，而“advice server”通过使用 UNIX 系统环境中的 vmstat 程序的输出来监视当前系统中自由页面的数量和缺页故障发生的频率，并以此来对垃圾收集系统做出指导，以决定该如何调整堆的大小。他们的一个基本策略是当内存压力增加的时候通过减小堆的大小来降低页故障发生的频率。因此他们的垃圾收集系统在一个存在多个应用同时运行的环境中也能够得到有效的控制。

Kim 和 Hsu 分析了 SPECjvm98 中的一些 Java 程序的内存系统行为[KH00]。他们工作中的一个观察就是 IBM JDK 1.1.6 所使用的缺省的堆的配置会导致频繁的垃圾收集，从而引起应用程序的低效执行。尽管他们所报告的结果只是某个特定系统中的现象，但是仍然具有一定的普适性。他们的结果说明控制垃圾收集的发生和堆的扩展是一种十分必要的技术。

Brecht et al. 在 Boehm-Demers-Weiser (BDW) [Boehm88]的保守的标记清扫垃圾收集器上研究了如何根据系统中可用物理内存的多少来自适应地决定何时发起垃圾收集以及何时扩展堆的大小[Brecht01]。他们的调度策略遵循以下 3 个规则:

1. 如果可用的物理内存是非常充足的，则当堆中的自由空间被耗尽的时候，垃圾收集并不发生，而是激进地扩展堆的大小。
2. 随着堆大小的增加，可用的物理内存相对的越来越少，他们尝试保持一定量的可用物理内存以避免换页的发生。这主要通过更频繁地发生垃圾收集，加上少量地扩展堆的大小来实现。
3. 当可用的物理内存变得相对少的时候，为了避免垃圾收集过于频繁，他们记录每次垃圾收集后所回收的自由空间的大小。如果最近一次垃圾收集所回收的自由空间并不是十分充足的话，那么当下一次堆中自由空间耗尽的时候，垃圾收集将不再被启动，而是继续扩展堆的大小。

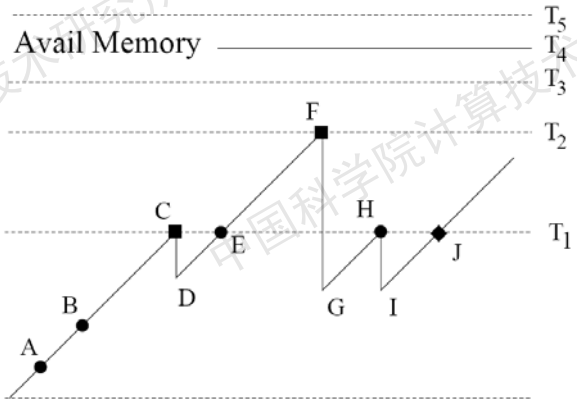

图 2.4 基于阈值方式来控制垃圾收集的示意图

他们采用基于阈值（threshold）方式来扩展堆的大小。图 2.4 示意了这种基于阈值的方法。图中黑色的点所表示的都是堆中自由空间被耗尽的时间点。当堆的大小没有超过过任何阈值的时候，堆总是被激进地扩展（A, B 点）。当堆的大小到达第一个阈值 T1 的时候（C 点），垃圾收集被启动，内存被回收到 D 点的位置。由于回收的自由空间并不充足，因此在 E 点的时候，他们的系统选择将堆的大小扩展到第二个阈值 T2。在 F 点，由于被使用的内存第一次到达了一个新的阈值，因此系统选择进行垃圾收集，内存被回收到 G 点的位置。由于这次回收的自由空间比较多，因此当被使用的内存又一次达到阈值 T1 的时候，系统依然选择发起垃圾收集。

他们的算法根据系统中的物理内存的数量来决定发起垃圾收集的时机，在一定程度上避免了页故障的发生，提高了应用程序的性能。然而，他们的垃圾收集系统并不和操作系统进行交互以获得可用物理内存的数量信息，而是针对固定的物理内存大小来制定特定的规则。

Andreasson et al. 使用机器学习的方法来决定垃圾收集发起的最佳时机[Andreasson02]。与前面介绍的研究不同，他们的研究是针对一个并发的垃圾收集器。另外，由于他们的方法需要学习和训练，这种策略通常要经过很长的时间才能够起到效果。

Yang et al. 通过修改操作系统并使用一个近似的重用距离（reuse distance）直方图来估计当前可用的物理内存的大小。他们为收集器开发了一种线性的模型来使 Java 虚拟机能够自动选择堆的大小以充分地利用物理内存[Yang04]。不象 Alonso 和 Appel 的工作，Yang et al. 的系统不仅能够在物理内存压力增大的时候减少堆的大小，也能够在物理内存压力减轻的时候重新扩展堆的大小，以降低垃圾收集的频率。另外，他们的系统不是针对静态已知的物理内存大小，而是通过操作系统来动态地收集物理内存的可用信息。由于他们所使用的算法比较复杂，而且需要修改操作系统，因此他们选择使用模拟的方法来进行研究。

Hertz et al. 开发了一个 paging-aware 的垃圾收集器[Hertz05]。通过修改操作系统来与垃圾收集器相配合，他们的垃圾收集器能够避免触碰已经被替换出内存的页面，从而减少了垃圾收集的时间。

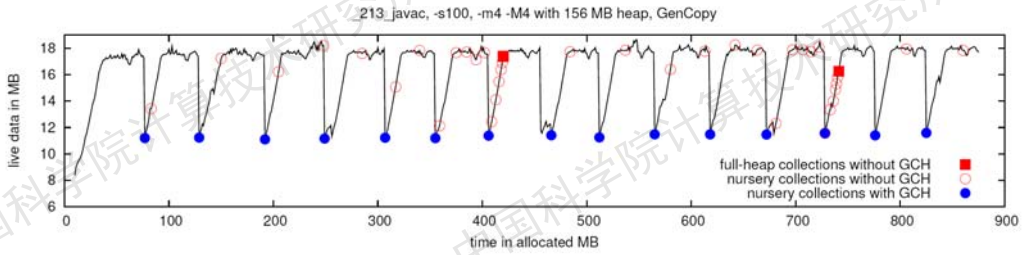

图 2.5 使用和不使用 GCH 的垃圾收集点的比较

Buytaert et al. 详细地分析了 Appel 风格的分代垃圾收集。他们发现标准的垃圾收集发起的条件并不能够使垃圾收集在最优点发起[Buytaert05]。他们认为垃圾收集应该在存活对象最少的时候发起，因为一个拷贝收集的时间开销是和存活对象的数量成比例的。

他们将这个时间点称为 Favorable Collection Point（FCP），进而开发了一个被称为 Garbage Collection Hints（GCH）的技术来指导垃圾收集在合适的时机被发起。他们的基本思想是使用离线的 profiling 来找到 FCP 所对应的 Java 程序的方法入口，然后在该程序点加入指示（hint），以使垃圾收集在该点被发起。图 2.5 说明了 GCH 的效果。

与此同时，Ding et al. 也进行了相似的研究[Ding05]。他们利用一个 Lisp 解释器来说明将垃圾收集的发生仅仅限制在程序相的边界（phase boundary），能够减少垃圾收集的开销并改善数据的局部性。

前面介绍的有关调整堆大小和确定收集发起时机的研究不是要对虚拟机进行修改，就是要改动操作系统。Zhang et al. 开发了一个程序级的自适应内存管理（Program-level Adaptive Memory Management, PAMM）技术[Zhang06]。他们的方法并不改变现有的虚拟机，也不会改变虚拟机所确定的堆的大小，这种大小被称为硬边界（hardbound）。而程序可以控制的堆大小被称为软边界（softbound）。硬边界可以不受物理内存的限制，而软边界主要是用来自适应地找到最优的堆大小。他们首先用 Java 程序实现一个 PAMM 控制器，然后在 Java 应用程序的相边界处插入一些代码来和 PAMM 控制器交互。PAMM 控制器能够从操作系统和垃圾收集器中得到页故障和堆中当前被占用的内存大小的信息。如果当前被占用的内存大小超过了软边界，则会发起垃圾收集，这种收集的发起被称为 reach-softbound invocation。如果从上一次收集到现在发生的页故障超过了一个限度，则也会引发垃圾收集，这种发起被称为 paging invocation。然后，PAMM 会自适应地调整软边界的位置。

Zhang 的方法把对内存管理的调度实现在了用户程序级，减轻了虚拟机开发人员的负担，而且这种技术将可以很方便地被移植到不同的虚拟机平台上。

2.3.2 基于算法的动态机制

在绪论中，我们已经详细地介绍了几种经典的垃圾收集算法，然而每一种经典的垃圾收集算法，都有各自的优点和缺点。通过对这些算法进行比较和分析，人们可能能够找到这样一些良机：能否将不同的算法进行组合，利用某种算法的优点来弥补和克服其他垃圾收集算法的缺点。事实上，有不少的研究人员，确实是这样做的。

Lang 和 Dupont 研究比较了标记清扫（mark-sweep）和节点复制（copy）的收集算法[Lang87]。节点复制算法的一个主要缺点是必须预留一半的堆空间作为收集时候的目标空间（to-space），而这一半的堆空间是不能够被用户程序所使用的。标记清扫收集算法无需预留空间，因为它不会移动对象。但是标记清扫收集算法的时间复杂度是和堆的大小成比例的，而不是和存活对象的数量成比例。因此当堆中存活对象的数量并不多的时候，标记清扫收集算法的效率就远不如节点复制算法了。另外，标记清扫收集算法无法控制内存碎片的产生，这不仅会使对象分配的效率降低，而且会使数据对象的局部性变得很糟糕。尽管缩并（compaction）能够使内存碎片的问题得到缓解，但是 Lang 认为缩并技术过于复杂而且开销太大，因此在他们的算法里，并没有使用缩并技术。

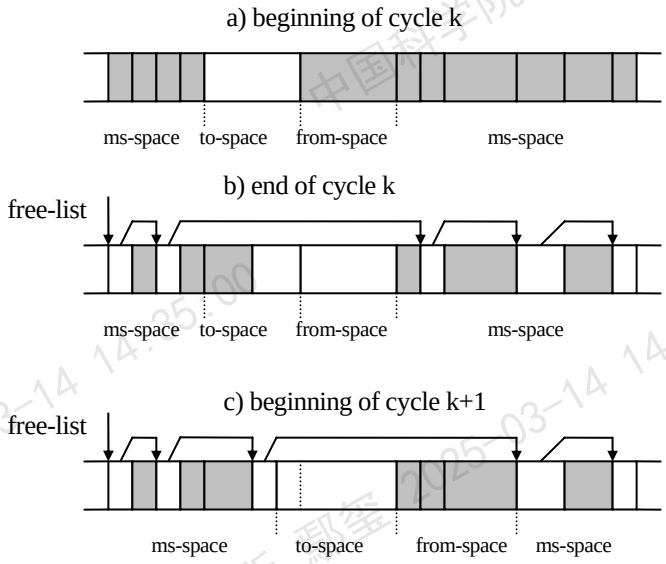

图 2.6 混合的标记清扫和节点复制算法

他们提出了一种混合（hybrid）的垃圾收集算法，该算法结合了标记清扫和节点复制，使得它们可以互相弥补对方的缺陷。图 2.6 描述了该算法的实现过程。他们把堆中的区域分成两个部分，对这两个部分分别使用不同的收集算法：节点复制和标记清扫。通过移动复制收集的区域，他们的算法能够缩并堆中不同的部分，从而缓解标记清扫算法中的碎片问题。从这个角度来说，他们的算法具有渐进的缩并的特点。由于他们把节点复制算法限制在一个小的区域里，因此只有这个小区域的一半的空间需要被预留，而不需要预留整个堆的一半空间，从而缓解了节点复制收集算法所带来的空间压力。

Zorn 利用一些中等规模的通用 Lisp 程序的踪迹（trace）比较评估了标记清扫和基于半空间（semi-space）的节点复制的收集算法[Zorn90]。在他的研究中，这两种算法都实现在分代的环境中。他的研究结果发现与节点复制算法相比，标记清扫收集算法会稍微慢一些，但是却大大降低了对内存空间的需求。

Sansom 描述了一个能够在节点复制算法和滑动缩并算法（sliding-compaction）之间切换的混合垃圾收集器[Sansom91]。他们的动机是希望让程序在存活对象的数量超过堆空间的一半的时候，仍然能够继续正常的运行。他们利用了一些合成的基准测试程序，并分析了这两种收集算法的性能平衡（trade-off）曲线，结果表明性能曲线的交叉点发生在堆的占用率（heap residency）为 25%~30%的时候。

Smith和Morrisett为一个ML语言的编译器实现了一个mostly-copying[1]的保守垃圾收集器[SM98]，并将它与Boehm-Demers-Weiser（BDW）的保守垃圾收集器[Boehm88]进行了比较。他们的主要目标并不是想通过详细的比较来评估这两种垃圾收集算法，而只是想说明

[1] 之所以被称为 mostly-copying，是因为它是一个保守收集器，因此只有那些所有指向自己的引用的位置都能够被确定的对象才能够被拷贝。在这样的算法中，to-space 和 from-space 不一定是由连续的内存空间组成，而是由一些内存块链接而成，因此是一个逻辑的概念。如果对一个对象来说，不能够确定所有指向它的引用的位置，则这个对象的地址是不能改变的，该对象所在的内存块会被直接从 from-space 的链表中删除，然后再链入 to-space 的链表中。

他们的这个新算法的效率优于BDW收集器。他们使用一些小规模的ML语言的基准测试程序进行测试，结果发现，与BDW收集算法相比，mostly-copying的收集算法能够带来至多20%的加速。

Printezis描述了一个可以动态切换的混合垃圾收集系统[Printezis01]。他们使用一个分代垃圾收集，其中年老代被组织成能够在标记缩并和标记清扫收集算法之间切换的结构，而且这种切换也无需重新改变堆的组织形式，而只需要支持自由链表结构的创建。在堆的大小被增长了之后，收集器切换到标记缩并的模式，因为这种模式可以利用新增加的自由空间来进行快速的bump-pointer分配。然后，他们的收集器又会切换回标记清扫算法，因为这种算法相对来说具有更快的收集速度。如果系统发现内存的碎片增多的时候，他们的收集器又会再次切换回标记缩并，以缓和碎片带来的问题。

Fitzgerald和Tarditi通过实验的方法证明了垃圾收集对运行时系统的影响是和应用相关的[FT00]。他们比较了一个使用不同写拦截技术的分代垃圾收集和一个非分代的基于半空间的节点复制垃圾收集，使用20个Java基准测试程序，并将基准测试程序和垃圾收集器两两搭配，然后对性能进行了比较。结果发现，每种垃圾收集技术都有可能在与某种应用搭配的时候取得最好的效果。在这种实验结果的基础上，他们主张利用基于profile的方法，来静态的选择应用程序和垃圾收集器的最优搭配。然而，他们的调查没有考虑到应用程序的不同输入集，而且仅仅使用了两种收集算法。

Attanasio et al. 比较了IBM的JikesRVM中的不同垃圾收集算法的性能[Attanasio01]。他们的研究结果同Fitzgerald和Tarditi的结论是相似的。另外，他们还发现了另一个影响垃圾收集系统性能的因素：内存堆的大小。这些研究都有力地说明了构建一个能够在不同的环境下动态切换到最适合的垃圾收集算法的系统是有一定必要的。

Soman et al.扩展了JikesRVM中的MMTk，在这个平台上设计并实现了一个动态切换垃圾收集算法的软件框架[Soman04]。他们通过静态的profile来选择应用、堆大小和垃圾收集器的最优搭配，并使用注解（annotation）的方法来调度垃圾收集器的切换。然而这种控制机制在程序运行的刚开始就已经被应用了，而且不能动态地改变。为了调查自适应的动态切换机制，他们实现了一个简单的策略来切换GMS（generational-mark-sweep）和SS（semi-space）收集器。他们的实验结果表明，基于注解的调度方法能够使平均的性能接近静态绑定的最好性能，而自适应的切换机制的平均性能虽然能够比静态绑定的最坏情况要好15%，但是与最好的情况仍然有15%的差距。

另外，他们指出，内存分配和写拦截这样的性能关键的函数在高性能的Java虚拟机中的实现都是需要被内联的，而不同的垃圾收集算法可能有着不同的分配函数和写拦截的实现，因此，在切换垃圾收集算法的时候，已经内联了这些函数的Java方法必需被重新编译，甚至需要On-Stack-Replacement（OSR）的支持[FQ03]。不过，他们的实验结果表明，这种重新编译及OSR的开销并不明显。

2.3.3 基于年龄的动态机制

简单的追踪式垃圾收集器有着许多的缺点，由于必须标记和复制所有的存活对象，因此垃圾收集可能造成很严重的延迟。然而，并不是堆中所有的存活对象都具有同样的特点，那么从直觉上来看，垃圾收集应该根据不同对象的特点而采取不同的策略。

年龄是对象的一个及其重要的特点。早在20世纪70年代，一些研究者们就曾经观察到有关对象年龄的一些规律。Deutsch曾经提到，统计数据显示，一个新分配的对象很可能不是“被钉住”，就是在相对较短的时间内被抛弃[Deutsch76]。Foderaro和Fateman发现，在一次垃圾收集中，超过98%的可回收内存是在上一次垃圾收集之后分配和丢弃的[FF81]。

在有关对象年龄的讨论中，弱分代假设被提出：大多数的对象在年轻的时候死亡。许多研究者已经收集了相当多的数据来支持这一假设[LH83, Ungar84, Shaw88, Zorn90, Hayes91]。然而，仍然会有一些对象，他们的生命周期会比较长。在简单的追踪式收集算法中，这些生命周期较长的对象不是被反复地标记和追踪，就是被反复地从一个半区复制到另一个半区。Lieberman和Hewitt最早将分代的思想应用在垃圾收集中[LH83]。分代的策略将堆按照年龄划分为几个区域，对象在最年轻的分代中被分配。年轻分代的对象经过若干次的收集之后如果仍然存活，将会被拷贝到年老的分代中，这个过程被称为提升（promote）。对年老代的收集频率要比年轻代低得多，这样就减少了反复地对寿命较长的对象进行操作的次数。如果弱分代假设成立，则年轻代中大部分的对象在收集的时候都已经死亡，因此，对年轻代的收集效率将会非常高，带来的暂停时间也会非常少。

分代的思想并没有严格的定义分代垃圾收集的结构，因此，分代的垃圾收集具有非常广阔的设计空间。研究者们通常会考虑的可变参数有分代的数目，分代的大小以及提升的阈值。

如果我们可以通过把堆分成 2 个分代来降低暂停时间并减少对较老的对象进行重复的操作，那么想要看看采用多个分代能否得到进一步的改进就是很合乎逻辑的想法。中间的分代用于过滤那些被过早地从最年轻的分代中提升的对象。我们一般期望中间分代中的对象的寿命是介于最年轻分代和最老分代的对象之间的。当收集中间分代的时候，中间分代中的大部分对象已经死亡，这样，收集中间分代的效率也能够很高。而且，这些中等寿命的对象不会因为被拷贝到最老的分代中，而使最老的分代过快地被填满，从而导致开销很大的全堆的收集。

然而，对于分代垃圾收集的测量结果并没有呈现出这个效果。相反的，在内存回收率这一方面，多分代的收集器表现出很大的下降。在非常年轻的对象和稍老的对象之间，回收率有很大区别，但此后的各个分代身上并未表现出这一点[Hayes91, DeT90, Shaw87]。Dieckmann 和 Holzle 对 SPECjvm98 中的基准测试程序的分配行为进行了详细全面的研究[DH98]。他们发现年老对象的年龄分布是和应用非常相关的。大部分的应用表明对象会在几乎相同的年龄成簇地死亡，这个发现和[Hayes91]的研究结果是一致的。这说明对于 Java 程序来说，多代的收集器可能也不是十分必要的。另外，更多的分代会

使得实现变得非常复杂。因为有多个区域会被局部的收集，因此会有更多的代间指针需要被记录，从而增加了写拦截的开销。基于这些原因，许多收集器只限于使用 2 到 3 个分代。

对于分代的大小，如果只考虑 2 个分代的情况，这个问题就变成了考虑最年轻分代的大小，即 nursery 的大小。研究者们对于这个可变的参数并没有太多的讨论。这可能是因为这个问题和应用及需求是十分相关的。如果 nursery 的大小很小，则每次次级收集的时间就会很短，从而会减小暂停时间。但是，这却会使垃圾收集变得频繁，增大 mutator 被挂起和恢复的时间开销。同时，频繁的次级收集会使对象过早地被提升，这会导致年老的分代过快地被填满从而引发对全堆的主收集。更糟的是，这些过早提升的对象虽然会在年老的分代中死亡，但是这种死亡却只有在主收集中才能被发现，这些在年老分代中死亡的对象被称为“占位垃圾（tenured garbage）”。如果主收集还没有发生，这些占位垃圾会使其在年轻代中的后代也存活下来，并且被提升，即使这些后代实际上已经死亡。

过早的提升还对用户程序的局部性有着负面的影响，这是因为程序中大多数访问的目标是较为年轻的对象，移动这些对象将会使程序的工作集分布得更加稀疏。然而，增大 nursery 的大小，可能会使每次次级收集的时间开销增加，从而增大了暂停时间。

一个经典的算法是 Appel 风格的分代垃圾收集算法[Appel89b]，它能够动态地改变 nursery 的大小。该算法将堆分为 2 个分代：nursery 和 mature。初始的时候，nursery 占据了整个堆空间。对象只能够在 nursery 中分配。当 nursery 被填满的时候，一个次级收集被发起，并且把存活的对象拷贝到 mature 中。Mature 的大小被扩展，同时 nursery 的大小被相应地减小。因为 nursery 的大小减小了，次级收集发生的频率就会增加，nursery 中的对象就只能有更少的时间去死亡。当 nursery 的大小降到一定的阈值，一个全堆的主收集被发起。主收集之后，所有的自由空间又被返还给 nursery。Blackburn et al. 比较了 Appel 风格和固定 nursery 大小的分代垃圾收集[Blackburn02]。其实验结果表明，Appel 风格的分代收集在大部分情况下优于固定 nursery 大小的分代收集，而不同 nursery 大小的分代收集之间的性能差别并不是十分明显。

提升的阈值决定了一个对象要经过多少次对年轻代的收集才能够被提升到年老的分代中。许多研究者用它来控制提升的速度。把这个阈值设为 1 将导致全体的提升：在每次收集之后，所有的对象都将被提升，即使某些被提升的对象可能实际上还很年轻。

Wilson 和 Moher 将 nursery 设计成两个半空间的组织形式，如果 nursery 中的对象没有被提升，则该对象将会在两个半区之间来回复制[WM89]。Wilson 发现，把提升前所需复制的次数设置为 2，效率很高。这一做法能够将幸存者减少到原来的 1/2，同时复制的开销只增加了不到一半。另一方面，继续增大提升的阈值很可能只能稍稍降低幸存者的数量[Ungar84, Shaw88, Ungar88]。Wilson 主张，一般必须把阈值增大 4 倍或更多才能消灭剩下的幸存者中的一半[WM89]。

Wilson 的方法需要记录对象的年龄，即对象经过的次级收集的次数。一般的做法是为对象增加一个年龄域。但是这样的做法往往会使对象的局部性变得更加糟糕，因为在

面向对象的程序中，对象的大小一般都比较小。Shaw 提出了一个方法来避免在每个对象内保存年龄。他把每个分代进一步细分成 2 块或更多块的空间，称之为“桶”（bucket）[Shaw88]。桶和分代之间有一些本质上的区别。尽管对象也是按照年龄被隔离到各个分代中去的，但是分代之间更多地是依靠收集的频率来划分。一个分代中不同的桶只是用来区别分代里不同年龄的对象，当一个分代被收集时，它所有的桶都会被扫描和复制。一个桶中的对象，在经过若干次收集之后，会被复制到代表着更高年龄的桶中。最高年龄的桶中的对象将在一定的收集次数之后被提升到下一个分代中。这样，桶的机制能够让被提升的对象具有更加精确的年龄。

以上这些方法都是静态地设置提升的阈值，然而，对象生命周期的统计分布并非总是静态的。Ungar 和 Jackson 认为固定年龄的提升策略太受限制[Ungar88, Ungar92]。他们采用一种被称为“基于统计特征的反馈调节提升”（demographic feedback-mediated tenuring）机制来解决这个问题。他们的基本策略是，如果一次次级收集中幸存下来的对象数量很少，则次级收集所带来的中断时间可能并不是很长，因此这些对象也许并不值得被提升。如果次级收集中存活的对象数量超过一定的阈值，这意味着次级收集的中断时间可能超过了可接受的最大限度，那么就在下一次次级收集中提升一部分年龄较大的对象。

2.4 并行垃圾收集技术

随着共享内存的多处理器（Shared memory multiprocessors）被广泛地部署，许多内存管理技术的研究者已经将目光转移到多处理器系统上来。大部分的 Java 虚拟机都会使用一种被称为“stop-the-world”的垃圾收集技术，该算法会先将所有的用户线程都暂停下来，然后才进行垃圾收集的工作。对于“stop-the-world”的垃圾收集系统来讲，在多处理器系统上，如果只有一个垃圾收集线程在处理内存回收的工作，则只有一个处理器将被充分利用，而其它的处理器资源将被浪费。在这种情况下，将垃圾收集线程并行化以充分地利用所有的处理器资源将是一个很显然且很有意义的想法。

为了避免给用户程序带来太大的延迟，一些研究者们设计出了并发的垃圾收集技术。该技术是让运行在某一个处理器上的垃圾收集线程和用户线程同时运行，从而减少垃圾收集的暂停时间。尽管这种技术也能够充分利用多个处理器资源，但是垃圾收集线程的并行化仍然是必需的。特别是对于一个多处理器系统来说，由于多个应用线程的内存分配的速度非常快，单独的一个垃圾收集线程的内存回收速度很可能不能够满足多线程的分配请求。因此，垃圾收集本身仍然需要被并行化，可缩放性（scalability）仍然是这种并行化技术的主要设计目标。

早期的并行垃圾收集技术的研究始于[Halstead85]。Halstead 在共享内存的多处理器系统中为 Multilisp[1]开发了一个并行的拷贝垃圾收集。他的算法是基于一个使用了

[1] Multilisp 是 Lisp 语言支持并行执行的一个扩展版本。

semi-space 的节点复制收集器。在他的算法中，每个处理器都拥有自己的局部的 from-space 和 to-space，这样每个处理器都可以在局部的空间中进行分配，而不存在处理器之间的竞争。

当收集发生的时候，每个处理器运行一个垃圾收集线程。每个收集线程都使用一个经典的节点复制算法去遍历所有的 from-space 中的对象（包括自己局部的和其它收集线程局部的 from-space），并将遍历到的对象拷贝到自己局部的 to-space 空间中。在遍历的过程中，几个垃圾收集线程可能会同时追踪到同一个对象，这时就会有竞争发生。正确的操作只允许一个线程能够成功地将该对象拷贝到自己局部的 to-space 中，因此必须使用原子的操作来完成迁移指针的安放。成功完成该原子操作的线程将把该对象拷贝到自己局部的 to-space 中，而失败的线程将使用被安放的迁移指针来更新相应的引用域。

比较高效的做法是使用体系结构提供的原子指令支持来实现这个原子操作，然而，Halstead 认为在 Multilisp 中，这种同步操作的竞争不会太频繁，因此他的实现中使用了锁来维持该操作的原子性。图 2.7 示意了这个过程。

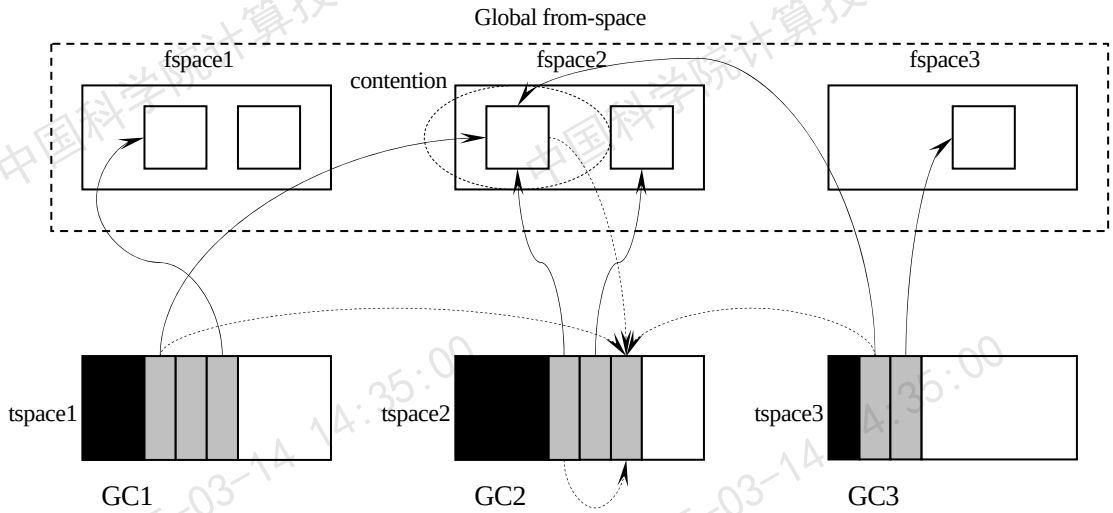

图 2.7 Halstead 的并行拷贝

在 Halstead 的并行算法中，from-space 相当于是全局的，每个收集线程都在一个全局的堆中遍历存活的对象。也就是说，该方法没有使用任何的负载平衡，因此，它并没有获得足够的可缩放性。

Imai 和 Tick 扩展了 Halstead 的并行拷贝收集算法，他们在 Halstead 的算法中增加了动态的负载平衡的机制[IT93]。他们的主要思想是在收集的过程中，将装有灰色对象的块放入一个全局共享的工作池（work pool）中，从而动态地分配遍历任务。收集开始的时候，垃圾收集线程尽量公平地划分根集合，所有的线程都顺着根集去遍历 from-space 中的存活对象，然后将新产生的灰对象块放入全局共享的工作池中。当某一个线程扫描完根集合或一个灰对象块后，就从共享工作池中取出一块新的灰对象块，然后继续扫描。由于 from-space 和工作池都是全局共享的数据结构，因此在他们的算法中，遍历

from-space 中的对象和对工作池的访问操作都需要同步的支持。

Endo et al. 设计了一个并行的“stop-the-world”标记清扫收集算法。他们使用了一种被称为 work-stealing 的技术来实现负载的平衡[Endo97]。每个垃圾收集线程都有一个被称为 stealable-mark-queue 的辅助队列。垃圾收集线程周期地检查自己的 stealable-mark-queue，如果该队列为空，就把一些未扫描的对象指针放入该队列中。其它饥饿的线程可以从这些队列中“偷取”一些对象指针以继续进行扫描和遍历。对 stealable-mark-queue 的访问必需使用同步操作。并且，由于在他们的算法中标记位是用位图的方式实现的，因此，对标记位的操作也必需同步。他们初始的实现是使用锁来保证对标记位操作的原子性，但是他们发现有大量的时钟周期被消耗在这些对锁的操作上，因此，他们在后来的实现中使用了原子的 CAS（Compare-And-Swap）指令来替换锁，从而改进了算法的性能。

Flood et al. 在一个由 SUN 微系统实验室的 Java 技术研究组开发的研究型 Java 虚拟机的平台[WG98]上，实现了一个并行的节点复制算法和一个并行的标记缩并算法[Flood01]。在负载平衡的实现上，他们扩展了 Endo et al.的工作，利用 Arora et al.提出的一个非阻塞的双端队列[Arora98]来实现更高效的 work-stealing 机制。

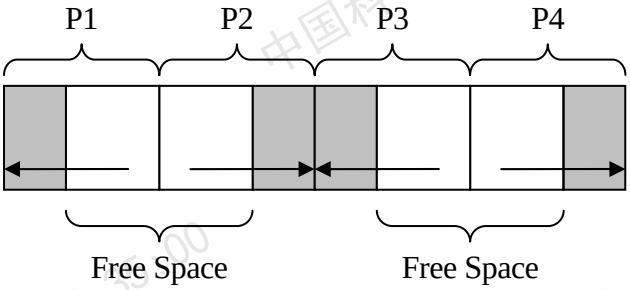

图 2.8 Flood 的并行缩并

在他们所使用的虚拟机平台中，初始的缩并的串行版本是将所有存活的对象都缩并到堆的底部。但是在并行的缩并算法中，要想保证一个线程在移动对象的时候不会覆盖掉另一个线程还未移动的对象是比较困难的。Flood 提出的解决方案是将堆分为 n 个区，n 就是所使用的垃圾收集线程的个数。每个线程都占有一个区，然后仅仅缩并自己所占用的区域内的存活对象。为了减小内存的碎片，增大可以用来分配的连续自由空间的大小，垃圾收集线程进行缩并的方向是奇偶交替的。图 2.8 显示了并行缩并的一个例子。图中堆被分为 4 个区域，缩并完成之后，存在两个连续的自由区域。

Cheng 和 Blelloch 设计了一个支持并行的实时的垃圾收集器[Cheng01]。他们使用一个全局共享的栈来实现任务的动态负载平衡。由于栈结构是共享的，因此出栈入栈的操作需要同步。简单的同步机制是将出栈和入栈的操作作为临界区，然后使用锁的机制来保证该临界区操作的原子性。然而，这种简单的同步机制会严重地影响并行算法的可缩放性，特别是对于并行的垃圾收集来说，这种共享的栈是进行任务分配的主要数据结构，

因此多个线程对共享栈的访问会非常频繁。Cheng 希望对共享栈的访问操作能够不需要锁，而仅仅使用原子加（atomic fetch&add）和原子 CAS（atomic compare&swap）指令就可以实现。他们使用了一种被称为“Room Synchronization”[Blelloch01]的同步机制来达到这种目的。

不管是入栈还是出栈都有两个基本的操作：改变栈指针和访问栈中元素。简单的同步机制需要使用锁的机制将这两个基本操作原子化。如果仅仅使用原子加操作来改变栈指针而不使用锁，则当入栈和出栈的操作重叠的时候，就会发生数据竞争（data race），这主要是因为出栈入栈所改变的栈指针的方向不同。然而，如果仅仅是某两个线程的出栈操作重叠，或者入栈操作重叠，则使用原子加来改变栈指针的做法就不会导致错误的发生。基于这个事实，room synchronization 的主要思想是将这个共享栈划分为 push-room 和 pop-room 这两个抽象的逻辑概念。当共享栈处于 push-room 的状态时，只有入栈的操作能够执行。当一批进入 push-room 的入栈操作执行完后，共享栈又被切换到 pop-room 的状态，此时，正在等待的出栈操作将进入 pop-room，然后，这些出栈操作才能够执行。这样，共享栈在 push-room 和 pop-room 之间来回切换，从而保证了对共享栈操作的正确性。尽管整个执行的过程中无需锁的操作，但是该算法仍然需要原子指令的支持。

Attanasio et al. 为几种经典的垃圾收集算法开发了相应的并行版本[Attanasio01]，包括节点复制、标记缩并和分代垃圾收集，并对这些并行收集算法进行了比较。在实现中，他们使用一个共享的工作缓冲区链表来进行动态的负载平衡。每个垃圾收集线程都不停地从该共享链表中获取一个工作缓冲区来进行对象的遍历和扫描过程。新遍历到，但还没有被扫描的对象指针被放入局部的工作缓冲区中，直到局部缓冲区满了，便将该局部缓冲区放入共享的缓冲区链表中。通过将标记位实现在对象头的元数据中，他们的设计不需要在标记操作中使用同步。然而，在访问共享的缓冲区链表的时候，仍然不能避免使用原子指令。

Ossia et al. 设计了一个面向服务器应用的并行、渐进和并发的垃圾收集[Ossia02]。他们使用了一种被称为“work packet mangement”的机制来实现负载平衡。这种机制和 Imai 的共享工作池机制相似，但是 Ossia 的收集器把全局的工作池划分为一些更小的子池以减少原子操作的开销。

Abuaiadh et al. 提出了一种高效的并行缩并算法[Abuaiadh04]，该算法比 Flood 提出的并行缩并算法[Flood01]具有更少的时间开销并且具有更好的缩并质量[1]。Flood 所使用的普通的双指针缩并算法需要遍历堆 3 次，而 Abuaiadh 的缩并算法只需要遍历堆 2 次。不象 Flood 的算法，Abuaiadh 的算法并不会将堆中的存活对象缩并成一堆一堆的，而是将所有的存活对象都缩并到堆的底部。

Abuaiadh 的缩并算法包括两个阶段：移动阶段和指针修复阶段。移动阶段仍然是使用双指针的方法，一个 from 指针指向当前扫描到的存活对象，一个 to 指针指向该活对

[1] 更高的缩并质量意味着能够获得更大的连续的自由空间。

象将要移动到的目标地址。在该阶段，所有的存活对象都被移动到目标地址。为了能够成功地修复指向对象的引用，他们的算法将堆中的内存分块，并且使用额外的数据结构来记录每个块中第一个存活对象所移动到的目标地址。由于对每个块只需要记录一个对象的相应信息，因此这个额外数据结构将不会占用很多内存资源。在修复对象引用域的阶段中，根据某个引用域所指向的对象的老的地址可以得到该对象在被移动前所在的块，他们利用 mark-bit table 来确定该对象在该块中是第几个存活的对象，然后，根据该块中第一个对象的目标地址便可以得到该对象的目标地址了。他们的算法减少了需要记录的迁移地址，并且通过一种巧妙的修复指针的机制使得迁移地址的计算和对象的移动能够在扫描一遍堆的过程中完成。

为了能够以很高的质量来并行地实现上述的缩并过程，Abuaiadh 的并行缩并算法将堆划分为若干区域。与 Flood 的算法不同，这种区域并不是与收集线程的个数相对应，而是以更小的粒度来划分。不过，区域仍然比块要大。初始的时候，每个收集线程获取一个区域，并且在区域内部进行缩并，这些被缩并的区域就有了一定的连续自由空间。然后，收集线程获取新的需要缩并的区域，并将存活的对象移动到一些已经被缩并过的区域的自由空间中。这样，Abuaiadh 算法能够并行地将所有的存活对象缩并到堆的底部，从而得到很好的缩并质量。

并行的垃圾收集技术能够充分地利用可用的处理器资源，从而加快垃圾收集的速度，减小垃圾收集所带来的延迟。同所有的并行程序一样，并行垃圾收集所追求的主要目标仍然是高的可缩放性。为了达到这个目标，研究者的努力主要集中在两个方面的工作上：减小同步的开销和获得更好的负载平衡。从某种意义上来说，这两方面的工作是有所矛盾的。从任务划分的角度上来说，任务划分的粒度越细，则更有利于得到更好的负载平衡，然而却可能招致更多的同步开销。因此，在并行垃圾收集算法的设计上，我们应该尽可能地在同步开销和负载平衡之间作出更好的权衡。

中国科学院计算技术研究所 鄢玺 2025-03-14 14:35:00

第三章 实验平台介绍

3.1 实验平台简介

在本文的研究工作中，我们主要使用的软件平台包括 SUN 公司的 JDK，SimOS 全系统模拟器以及 Apache Harmony 的 Java 虚拟机平台。本节将对这几个软件平台进行简要的介绍。

SUN JDK:

SUN 公司所开发的 JDK 是一个 Java 语言开发平台，它由一个 Java 虚拟机和一系列的标准类库组成。它所支持的体系结构包括 SPARC，X86 和 AMD64。从 JDK1.5 的版本开始，SUN 公司开放了他们的 JDK 源代码，这使得我们有机会将该版本的 JDK 移植到 MIPS 平台上来，于是可以被龙芯系列处理器平台所使用。然而，SUN 所使用的开源软件许可证并不是 GNU，并且对软件的自由开发有所限制。

在 JDK1.5 中，缺省的垃圾收集配置是一个串行的 stop-the-world 的分代垃圾收集器。堆被分为 2 个分代，4 个区域：

Eden：属于年轻的分代，新的对象在该区域中被分配；

From-To：属于年轻的分代，是该分代中的老化区域。经过一次次级收集后，Eden 中一部分存活的对象被拷贝到该区域中；

Tenured：属于年老的分代，经过一次次级收集后，年轻代中一部分存活的对象被提升到该分代中；

Perm：不属于任何分代。该区域主要用来存放一些在程序运行过程中永远不会消亡的对象，例如与 Class 相对应的对象。这些对象将直接在该区域中被分配；

由于 JDK 是一个复杂的产品级 Java 虚拟机，其中使用了非常多的性能优化的先进技术，虚拟机中各个模块之间的耦合度也非常紧密，它们之间的接口繁多且复杂。因此，JDK 不太适合于用来进行垃圾收集算法的研究。但是，由于我们已经将它移植到 MIPS 的平台上，这意味着它可以运行在一些支持 MIPS 体系结构的全系统的模拟器平台上。这使得它更适合于被用来做一些与体系结构支持相关的研究工作。

SimOS:

SimOS 是由 Stanford 大学研究开发的一个全系统模拟器。SimOS 的初始版本实现了一个完全的单处理器和多处理器的计算机系统模型。在这个系统模型上可以启动并运行一个 IRIX（SGI 公司的一个 Unix SVR4 的实现）操作系统。经过我们的一些移植和修改工作，目前我们所使用的 SimOS 平台支持 MIPS 指令集，并且能够运行 Linux 操作系统。

由于 Java 虚拟机需要用到大量操作系统相关的功能和调用，并且 Java 虚拟机本身的庞大规模及其优良的动态特性使得将它编译为一个静态链接的版本并不容易。因此，很难让 Java 虚拟机运行在一个用户级的模拟器平台上。而 SimOS 这样的全系统的模拟器则大大降低了运行 Java 虚拟机平台的难度，并且能够获得更加真实的运行时环境。

Apache Harmony:

Apache Harmony 的提案在 2005 年 5 月被 Apache 软件基金会（ASF）接受，并且按照 ASF 惯例成为一个孵化器（incubator）项目。

Harmony 制定了两个目标，首先是开发出一个独立并且与现有 JDK 兼容的 JavaSE5 的实现，并且以 Apache 软件许可证 2.0 版发行；其次建立一个开放的模块化运行时架构，包括虚拟机和类库之间及其内部的模块化，并通过这个平台，允许社区在此基础上自由定制自己的 Java 实现，或者对某个模块单独进行创新。

Apache Harmony 项目的成立以及它的这两个目标具有很大的现实意义。首先，由于商业 JDK 的流行性，它们几乎成为标准，所以 Harmony 必须与它们保持高度兼容，才能够使应用程序的迁移成本最低，也相对容易被用户所接受；其次，Harmony 存在的一个重要意义在于它是一个属于开源社区的 Java 平台，在这个平台上，社区可以自由地移植和创新，而一个开放的模块化的架构，将为移植和创新带来最大的便利；最后，Apache 软件许可证是一个对商业公司和开源社区都比较友好的开源许可证，因此 Harmony 可以给大范围的开发人员和用户带来便利。

基于以上原因，Harmony 项目成立一年时间，得到广泛的支持和关注，并且得以飞速发展。目前，Apache Harmony 已经拥有一个活跃的开发社区，并且接受来自公司，学校和个人的多次捐赠。IBM 在 developerWorks[1]上提供了一个 Java 虚拟机环境的免费下载，以支持 Harmony 项目的开发，还多次捐赠了核心类库的实现和测试代码，在 IBM 英国 Hursley 软件开发实验室和中国软件开发中心各有一个开发团队积极参与 Harmony 项目。此外 Harmony 项目还接受了三个 Java 虚拟机实现的捐赠，另外还有一个开源 Java 虚拟机 SableVM 正在积极地与 Harmony 社区合作以实现与 Harmony 类库的集成。JavaSE 实现难度最大的是其规模庞大的类库，而最近的类库 API 覆盖率统计表明，Harmony 的 J2SE1.4.2（JavaSE4）类库 API 的覆盖率已经超过 80%，JavaSE5 的覆盖率已经达到 79%以上。目前经不完全测试，在 Harmony 上已经可以良好运行 Eclipse、JEdit、Ant 等流行的 Java 工具，并且可以部分运行 Tomcat 和 Geronimo 等企业应用。

DRLVM 是 Intel 公司开发的一个支持 JavaSE5 的虚拟机实现，并且已经被 Harmony 项目所接受成为其 JavaSE5 的核心虚拟机实现。DRLVM 是 Dynamic Runtime Layer Virtual Machine 的缩写，它主要由 C++和部分汇编语言编写而成。

[1] http://www-128.ibm.com/developerworks/

DRLVM 的主要特点包括以下四点：

- 模块化：根据功能的不同划分成不同的模块，并且各个模块都有各自的良好的接口定义；
- 可插入性：模块的实现可以在编译时或者运行时被替换，一个模块可以有多种不同的实现；
- 一致性：在不同的平台上接口是相同的；
- 高性能的：良好定义的接口使得开发者可以开发出针对特定的平台进行特别优化的模块实现；

由于 DRLVM 的设计具有非常好的模块化风格，并且，模块之间的接口定义十分清晰而且简洁，因此，可以很方便地对其中的某个模块进行创新的开发和研究。所以，我们选择了这个平台作为我们进行垃圾收集算法研究的主要平台。在该平台上进行垃圾收集的开发，可以使程序员和研究人员把主要的精力放在收集模块本身的问题上，而不用过多地去考虑接口之间的依赖问题。图 3.1 显示了 DRLVM 的模块化结构。

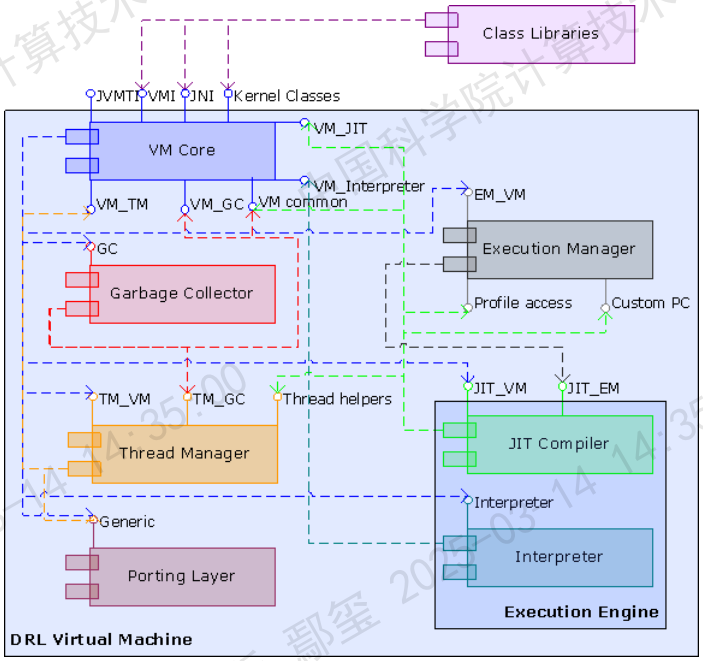

图 3.1 DRLVM 的主要组件

3.2 一个灵活的算法可切换的内存管理研究平台

为了对不同的垃圾收集算法进行研究，我们在 Harmony 的平台上设计并实现了一个内存管理研究平台。由于研究垃圾收集算法的自适应的运行时切换技术是本文工作的一个重要内容，因此，我们的内存管理平台建设主要考虑的一个方面就是构造灵活可切换的软件框架。为了后面的描述方便，我们所搭建的内存管理研究平台被称为 RSMTk（Runtime Switchable Memory management Toolkit）。

3.2.1 堆的组织形式

在 RSMTk 中，堆被分为 3 个区域：NOS（Nursery Object Space）、MOS（Mature Object Space）和 LOS（Large Object Space）。同时，堆又被分为 2 个分代：年轻的分代和年老的分代。NOS 就是年轻的分代而 MOS 和 LOS 则构成了年老的分代。

NOS 和 MOS 都是以块（block）为单位来组织的，块的大小可以配置，缺省的配置为 64KB。LOS 则组织成自由链表的形式，采用标记清扫的方式来进行收集，而且大型的对象将会直接被分配在 LOS 中，这主要是因为拷贝大型对象的开销很大，因此，一般的垃圾收集设计考虑都是将大型的对象放在一个特殊的区域里，并且尽量避免对大型对象的拷贝。到底多大的对象才算是大型的对象？关于此问题并没有定论，我们的设计中允许对大型对象的最小大小进行配置。

对于分代垃圾收集来说，它将在 NOS 中分配新的对象。当 NOS 的空间被新分配的对象耗尽时，次级收集被发起，分代垃圾收集器将 NOS 中的存活对象提升到 MOS 中去。写拦截被建立在年老的分代和年轻的分代之间。对于非分代的垃圾收集来说，收集器将仅使用 MOS 和 LOS。我们目前所实现的对 MOS 的收集有标记缩并和节点复制算法。

由于基于自由链表的分配可能招致内存碎片，从而会导致较差的局部性并会降低分配的速度，因此我们的设计将尽量抛弃这种分配算法。无论是在 NOS 中分配还是在 MOS 中分配，我们都使用 bump-pointer 的分配算法。这种分配算法有两个好处，一是分配算法非常简单，仅需要将 free 指针向后移动新分配对象的大小；二是对象将按照分配的顺序被连续的排放——许多研究者认为，保持对象的分配序能够获得非常好的局部性。而对大型对象的分配将不得不使用自由链表的分配算法。不过由于面向对象程序的一个特点就是大部分的对象都比较小，因此，通常来说，大型对象出现的频率都相对较小（除了一些特定的应用以外）。

对于 bump-pointer 的分配来讲，堆空间又存在着另一个粒度上的划分：线程局部的分配单元。这是因为 Java 运行时环境是一个支持多线程的环境，因此分配函数必需支持多线程的并发操作。如果不提供线程局部分配的支持，则每次分配一个对象，都必须使用同步的操作，这在多线程、多处理器的环境中是非常低效的。因此，通常的实现是线程首先在共享的堆空间中分配一个线程局部分配单元，以后就可以在该局部分配单元中无需同步的自由分配，直到该局部分配单元被分配满了，再重新从共享堆空间中分配一个新的线程局部分配单元。

对线程局部分配单元大小的确定是一个需要权衡的问题。如果太小则会引入过多的同步操作，如果太大则会造成分配的不均匀，并可能导致一些空间的浪费。我们的设计中使用 Thread_local_allocation_unit 数据结构来管理线程局部分配单元，并允许配置该单元的大小。缺省的配置是使用块的大小作为线程局部分配单元的大小，即 64KB。

3.2.2 基于 Allocation Area 的设计

Soman et al.通过扩展 MMTk[Blackburn04b]实现了一个能够在运行时切换算法的垃圾收集器[Soman04]，但是由于 MMTk 的初始的设计目的并不是为了满足动态切换的需要，因此，他们在实现过程中也遇到了一些问题。每种收集器都使用自己的堆空间，因而在一些算法的切换中，需要将对象从一个收集器的空间复制到另一个收集器所管理的空间中去，并且造成了空间的浪费。

Soman 和 Fitzgerald 都曾经指出在运行时动态地切换垃圾收集算法可能会遇到一个问题：由于不同的收集算法可能使用了不同的分配函数，而分配函数又是一种性能关键的函数，在大多数的高性能的 Java 虚拟机实现中，这种性能关键的函数都需要被内联，因此，当使用了不同分配函数的收集算法互相切换的时候，一些已经经过内联优化的 Java 方法就需要被无效掉，并且需要被重新编译[Soman04, FT00]。

为了避免上述的问题，RSMTk 的设计基于一种理念：堆空间只是存放对象的容器，而垃圾收集算法则是管理这个容器的方法。因此，只要堆的组织形式足够的灵活，不同的收集算法可以共同管理同一个堆空间。另外，不同的收集算法可以尽量使用相同的分配函数，RSMTk 通过基于 Allocation Area 的设计来达到这个目的。

RSMTk 对于除大型对象之外的对象均采用 bump-pointer 的算法来进行分配。分配函数的实现仅仅依赖于 Allocation Area，其实现是通过 Allocation_area 数据结构来完成。也就是说，分配函数仅会在 Allocation_area 数据结构所管理的内存空间中进行分配。Allocation Area 在这里实际上是担当着内存分配和内存收集之间的接口的角色。分配函数无需关心收集算法是如何进行的，只要收集算法在结束后能够正确地设置 Allocation Area，则分配操作就可以无需改变地进行下去。基于 Allocation Area 的 RSMTk 的结构如图 3.2 所示。

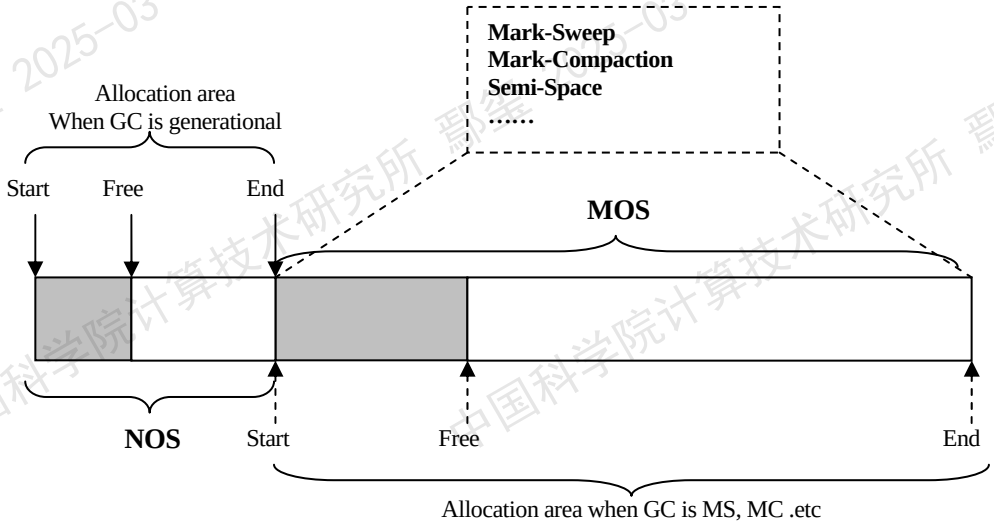

图 3.2 基于 Allocation Area 的 RSMTk 的结构

采用这样的设计可以获得两个好处：

1. 由于对不同的收集算法，分配函数都采用的是 bump-pointer 算法，并且仅仅依赖于 Allocation Area，因此，可以对不同的收集算法采用相同的分配函数的实现，从而避免了在切换垃圾收集算法后因为内联了分配函数而导致的方法重编译；
2. 使用了基于 Allocation Area 的设计后，收集算法的实现也会更加灵活，特别是对于一些基于区域的收集算法来说，收集算法可以将 Allocation Area 指定到堆中任意的一个连续区域。

3.3.3 垃圾收集算法的切换

本节简要地描述了 RSMTk 中的几个不同垃圾收集算法之间的切换过程。由于 RSMTk 中的分代垃圾收集的年老分代的管理算法为标记缩并，为了描述方便，分代垃圾收集被简称为 GenMC，标记缩并算法为 MC，基于半区的节点复制算法为 SS。

GenMC→MC：RSMTk 需要进行一次次级收集，将 NOS 中的存活对象移动到 MOS 中，并将 MOS 中剩余的自由空间作为该次收集结束后的 Allocation Area。之后，NOS 将不再被使用，直到再次切换回分代的收集；

MC→GenMC：RSMTk 无需发起任何额外的收集过程，只需要将 Allocation Area 重新绑定到 NOS 中即可；

MC→SS：RSMTk 在进行一次标记缩并之后，如果存活对象的大小小于堆空间大小的一半，则此次切换能够成功进行，否则此次切换过程失败。如果切换过程能够成功，RSMTk 将堆空间平分为两个半区，其中，低端的半空间为下次收集的 From-Space，高端的半空间为下次收集的 To-Space，并将低端的半空间作为新的 Allocation Area；

SS→MC：如果当前的 To-Space 为高端的半空间，则无需发起任何的收集过程，只需要将当前的 To-Space 作为新的 Allocation Area 即可；否则，RSMTk 需要进行一次节点复制，将 From-Space 中的存活对象拷贝到 To-Space 中去，然后将存活对象之外的所有剩余空间作为新的 Allocation Area；

GenMC→SS：该过程即为 GenMC→MC→SS；

SS→GenMC：该过程即为 SS→MC→GenMC；

3.3 本章小结

本章描述了我们所使用的实验平台及其搭建。本文中大部分的工作都是在 RSMTk 的平台上实现的。与 MMTk[Blackburn04b]等其它的内存管理工具集相比，RSMTk 具有更加灵活的软件框架。在设计的初始阶段，我们就对不同收集算法在运行时的灵活切换进行了细致的考虑。在 RSMTk 中，不同的收集算法管理着相同的内存堆空间，并且运用了基于 Allocation Area 的设计理念，从而节省了空间资源，同时避免了收集算法切换时由于不同分配函数的内联而引起的方法重编译。

第四章 内存管理对应用局部性的影响

4.1 引言

当今计算机硬件发展的一个趋势就是处理器性能和内存系统性能之间的差距正在变得越来越大。从内存中读取一个数据项所需要的时钟周期数可能是处理器操作该数据项所需时钟周期数的几十倍甚至上百倍。数据 cache 作为缓和处理器和内存系统之间性能差距的桥梁，通过将最经常和最可能被访问的数据项进行缓存来充分地利用数据访问的局部性。因此，获得好的数据局部性以减少 cache 失效的次数对于追求更优的性能来说是非常关键的。

对于规则的科学计算的程序来说，存在着许多静态的和离线 profile 的软件技术来改进数据的局部性。但是，对于具有良好动态特性的面向对象的运行时环境来说，静态和离线的方法往往由于不能够获得准确的运行时信息而导致无法得到显著的效果。而采用垃圾收集的运行时系统将使在运行时改变并优化数据的局部性成为可能。不像 C 语言这样的采用显式内存管理的运行时系统，垃圾收集系统可以通过移动存活对象的位置来改变对象的布局，从而获得改善对象局部性的良机。

软件预取是另外一种用来改善内存系统性能的软件技术。与改进数据局部性的技术不同的是，预取的主要目标是尽可能地容忍 cache 失效所带来的延迟。对于一些数组的操作来说，预取技术通常能够取得较好的效果，因为程序对数组的访问操作往往能够呈现出较为明显的有规则的模式。

本章的研究在 RSMTk 的平台上对分代、标记缩并和基于半空间的节点复制这三种经典的垃圾收集算法的性能及其对 Java 应用程序的数据局部性的影响进行了比较和评估。并且尝试构建一个软件框架，该软件框架结合了预取和局部性优化技术，在即时编译器的支持下，通过插入一些 profile 的代码来记录数据（包括对象和数组）的访问序，并对该访问序进行分析以决定采用预取或局部性的不同优化策略。

本章的另一部分工作研究了利用 on-chip 的 local memory 来减少访存开销的技术。实验结果表明，对于 Java 应用来说，只要能够将 local memory[1]映射到正确的地址空间，充分利用对象的局部性，local memory 将会带来较大的性能好处，而且 local memory 与 cache 结合的系统将会比 cache-only 的系统具有更优的性能。

本章的组织如下：4.2 比较和评估了不同的收集算法的性能及对局部性的影响；4.3 描述了预取与局部性优化结合的软件框架；4.4 讨论了 on-chip 的 local memory 对内存系统的性能优化；4.5 对本章进行了总结。

[1]为了描述的简化，本章中将用 local memory 来指代 on-chip 的 local memory。

4.2 垃圾收集系统性能及对程序局部性影响的比较和评估

本节在 RSMTk 的平台上比较了分代，标记缩并和基于半区的节点复制收集算法。所使用的基准测试程序包括 SPECjvm98[1]和 DaCapo[2]中的一些程序以及 pseudojbb。表 4.1 给出了这些基准测试程序的简要描述。所使用的硬件平台是 Intel Pentium4 2.6GHz 单核处理器，系统平台是 Fedora Core4 release，操作系统内核为 Linux-2.6.11。

表 4.1 基准测试程序简介

Name	Suite	Description
jess	SPECjvm98	Java Expert Shell System based on NASA's CLIPS expert shell system
db	SPECjvm98	In-memory database
javac	SPECjvm98	Java language compiler from the JDK 1.0.2
jack	SPECjvm98	Java parser generator that based on the Purdue Compiler Construction Tool Set
jython	DaCapo	Python interpreter
antlr	DaCapo	Parses one or more grammar files and generates a parser and lexical analyzer for each
pseudojbb	----------	A variant of SPECjbb2000, Java server benchmark

Chilimbi 和 Larus 提出了一种利用分代拷贝垃圾收集来改进数据局部性的软件技术[CL98]。他们的工作在 Cecil[Chambers93]的平台上实现，并且取得了不错的结果。Huang et al.设计了一种被称为 Online Object Reordering（OOR）的技术来改进对象的局部性[Huang04]。与 Chilimbi 的工作不同，他们没有使用基于对象实例的运行时 profiling，而是采用了基于 Class 的 online profiling。这样做的好处是能够降低运行时 profile 的开销。该工作是在 JikesRVM 的 Java 虚拟机平台上实现的，然而，他们所报告的数据结果并不是十分理想。Huang 所使用的内存管理也是一个分代拷贝垃圾收集器。

分代拷贝垃圾收集是分代垃圾收集的一种，其年老的分代使用的是基于半区的节点复制收集算法，或者是基于多个分区的拷贝收集。在分代拷贝垃圾收集中，存在着两种拷贝序：1)分代间的拷贝序；2)分代内的不同分区间的拷贝序。[Huang04]中的另外一个工作是比较评估了分代拷贝收集中不同的静态拷贝序给对象局部性所带来的影响，并且发现，深度优先的遍历序能够带来较好的对象局部性。然而，同 Chilimbi 的工作一样，他们并没有进一步去评估这种对对象局部性的影响到底是由上述两种拷贝序中的哪种产生的，或者到底哪种拷贝序是对象局部性改变的主要贡献者。

在本章的工作中，我们分别评估了分代间拷贝序和半区间拷贝序对程序中对象局部性的影响。结果表明，半区间拷贝序所带来的影响要比分代间拷贝序大得多。一个直观的解释是，由于弱分代假设，在分代间拷贝中存活下来的对象相对较少，因此，对这些对象布局的改变，并不会对年老代中对象的整体布局带来太大的影响。

[1] http://www.spec.org/osg/jvm98/
[2] http://www.dacapobench.org/

图 4.1~图 4.7 对 3 种经典的垃圾收集算法的性能进行了比较。它们包括分代垃圾收集（Gen）、标记缩并（MarkCompact）和半区节点复制（SemiSpace）。其中，Gen-df 表示次级收集中将年轻分代中的存活对象拷贝到年老的分代中的时候是按照深度优先（depth-first）的遍历序去遍历存活对象的，而 Gen-bf 则是按照广度优先（breadth-first）的遍历序进行遍历。同样，SemiSpace-df 是以深度优先的遍历序将存活对象在两个半区之间进行拷贝，而 SemiSpace-bf 则是以广度优先序进行拷贝。图中横坐标是不同的堆大小，纵坐标是运行时间。分代垃圾收集的 NOS 区采用固定的大小，其大小为 4MB。

图中比较了程序运行的总时间（Total Time）、垃圾收集所消耗的时间（GC Time）以及用户程序所消耗的时间（Mutator Time）。程序运行的总时间即为垃圾收集时间和用户程序时间之和。垃圾收集的时间开销反映了垃圾收集算法自身的效率，而对于仅有垃圾收集算法不同的运行时系统来说决定用户程序时间的主要因素包括分配函数和写拦截函数的实现以及垃圾收集算法对用户程序的对象局部性的影响。由于在 RSMTk 中，不同的垃圾收集算法都采用相同的分配函数实现，而经过我们的实验测量，分代垃圾收集中所使用的写拦截函数的运行时开销最大不超过 3%，因此，影响 Mutator Time 的主要因素应该是垃圾收集算法对对象局部性的影响。

从所比较的实验数据中，我们可以看出，对于大部分的应用来说，分代垃圾收集具有更高的收集效率，即在大部分情况下，分代垃圾收集的收集时间开销较小。特别是当堆较小，内存资源比较紧张的情况下，分代垃圾收集具有更高的相对效率。这主要是因为，MarkCompact 和 SemiSpace 在堆较小的情况下，每次收集所回收的自由空间变小了，然而，每次收集的时间开销却没有成比例的减少，收集的效率便会因此而下降。而分代垃圾收集却具有相对稳定的效率，因为固定的 NOS 大小，使得它每次次级收集所回收的自由空间大小和收集的时间开销都基本上保持在一个稳定的状态。除非当堆特别小，MOS 非常小，几乎每次次级收集都会引发主收集的时候，分代垃圾收集的时间开销才会相对较大。

在大部分的情况下，MarkCompact 收集算法能够带来相对较低的 Mutator Time，这个结果证实了一个公认的猜想——保持对象的分配序能够带来更优的数据局部性。_228_jack 是一个特例，我们认为该程序中一些长生命周期的对象对局部性有着非常重要的影响，垃圾收集对这些对象的任何操作，都会引起 Mutator Time 的大量增加。而在堆空间较大时，分代垃圾收集由于能够极大地减少对 MOS 的收集而减少了对这些对象的操作，因此获得了更好的 Mutator 的性能。

SemiSpace 在堆较大时，用户程序的对象局部性也趋于好转，在不少的情况下，能够和 MarkCompact 相当，甚至在少数情况下能够超过 MarkCompact。我们发现，对于 SemiSpace 的收集算法来说，对象的遍历序对对象局部性的影响并没有想象中那么大。对于大部分的应用，不同的遍历序对 Mutator Time 几乎没有影响，只有_209_db 会有较为明显的影响，而对于 antlr，则是在某些堆大小的情况下会有较大的影响。另外，对于分代垃圾收集来说，遍历序对用户程序局部性的影响明显较小。这说明，对于分代拷贝

垃圾收集来说，对象局部性改变的主要贡献者应该是分代内不同分区间的拷贝序。这也是我们下一节的局部性优化工作没有取得很好效果的一个主要原因。

对于程序运行的总时间（Total Time）来说，当堆很小的时候，该时间主要由垃圾收集所消耗的时间决定，而当堆较大时，则主要由 Mutator Time 来决定。这说明在一个允许动态切换垃圾收集算法的系统中，当垃圾收集的运行时间占程序总运行时间的比例较大的时候，应该根据垃圾收集的效率来进行收集算法的选择；而当堆较大，内存资源充足，垃圾收集的时间开销不大的时候，则应该主要根据收集算法对对象局部性的影响来进行收集算法的选择。在这种情况下，一个简单的选择是 MarkCompact 算法，或者是某种保持对象分配序的收集算法。

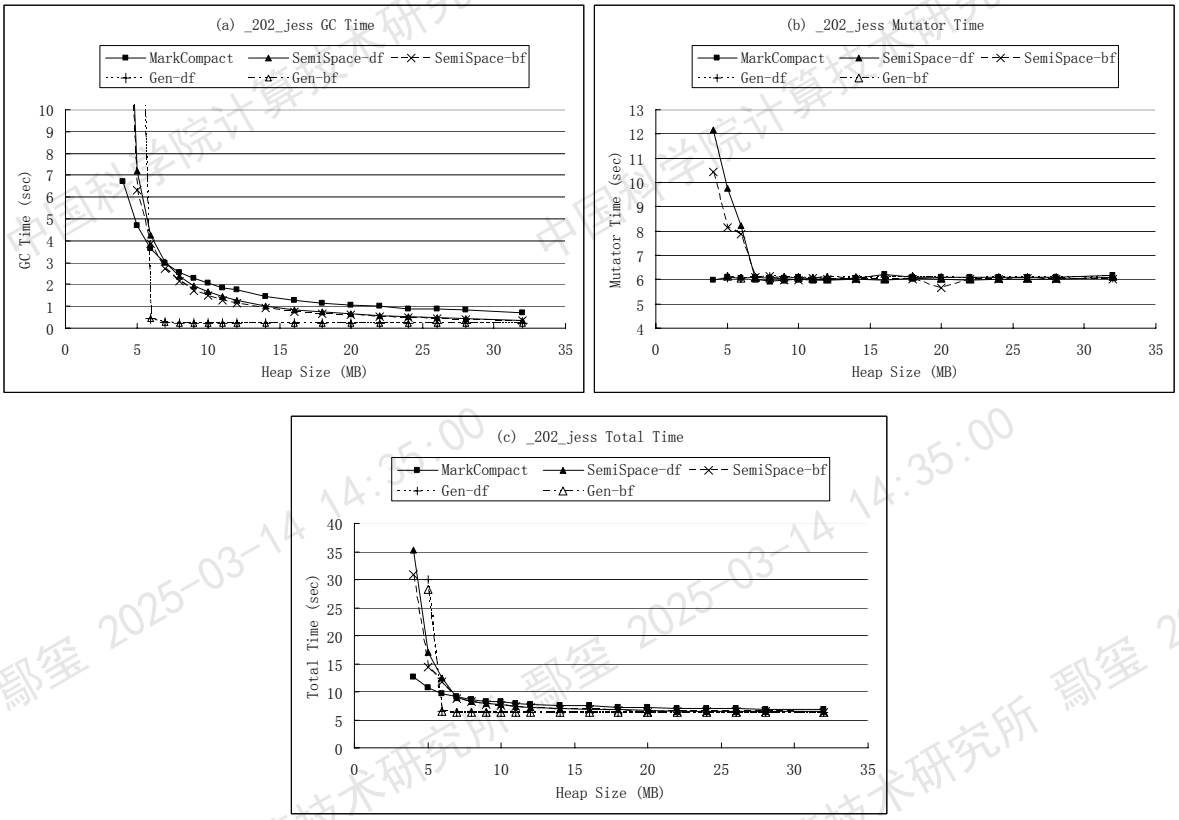

图 4.1 垃圾收集性能比较 _202_jess

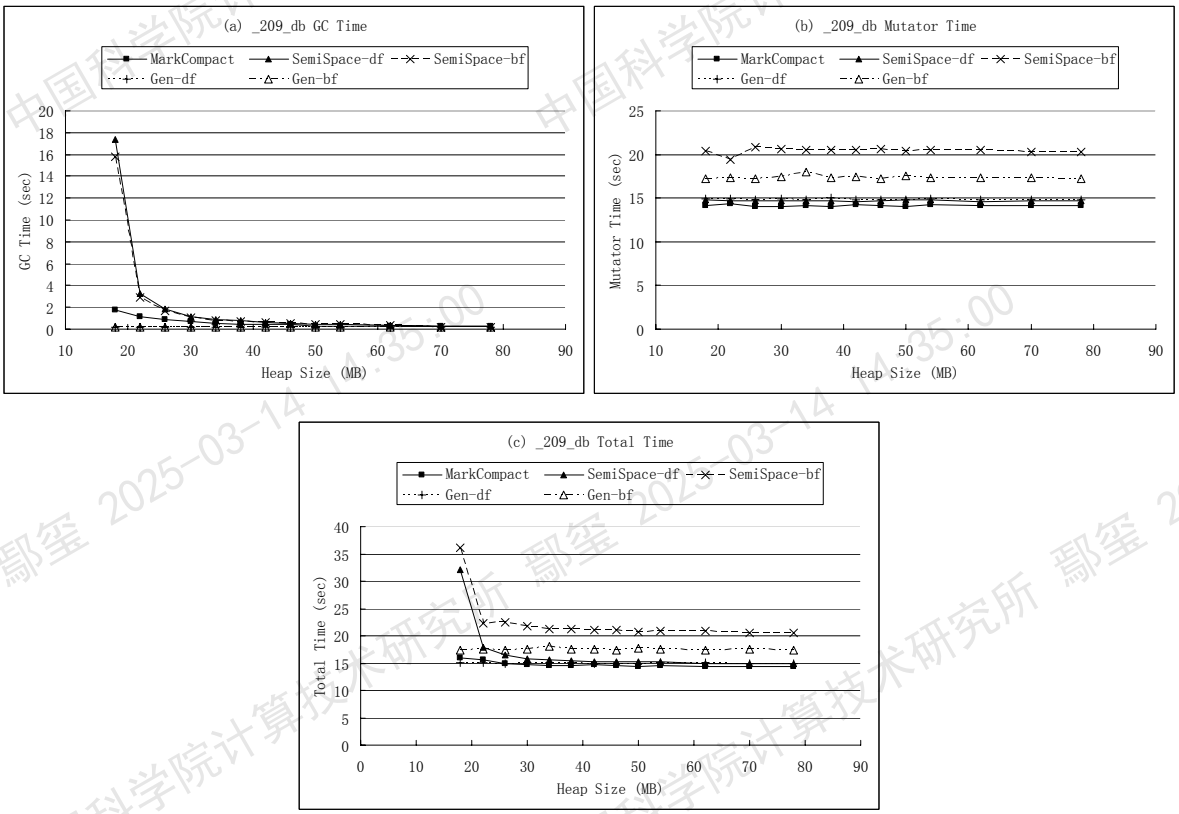

图 4.2 垃圾收集性能比较 _209_db

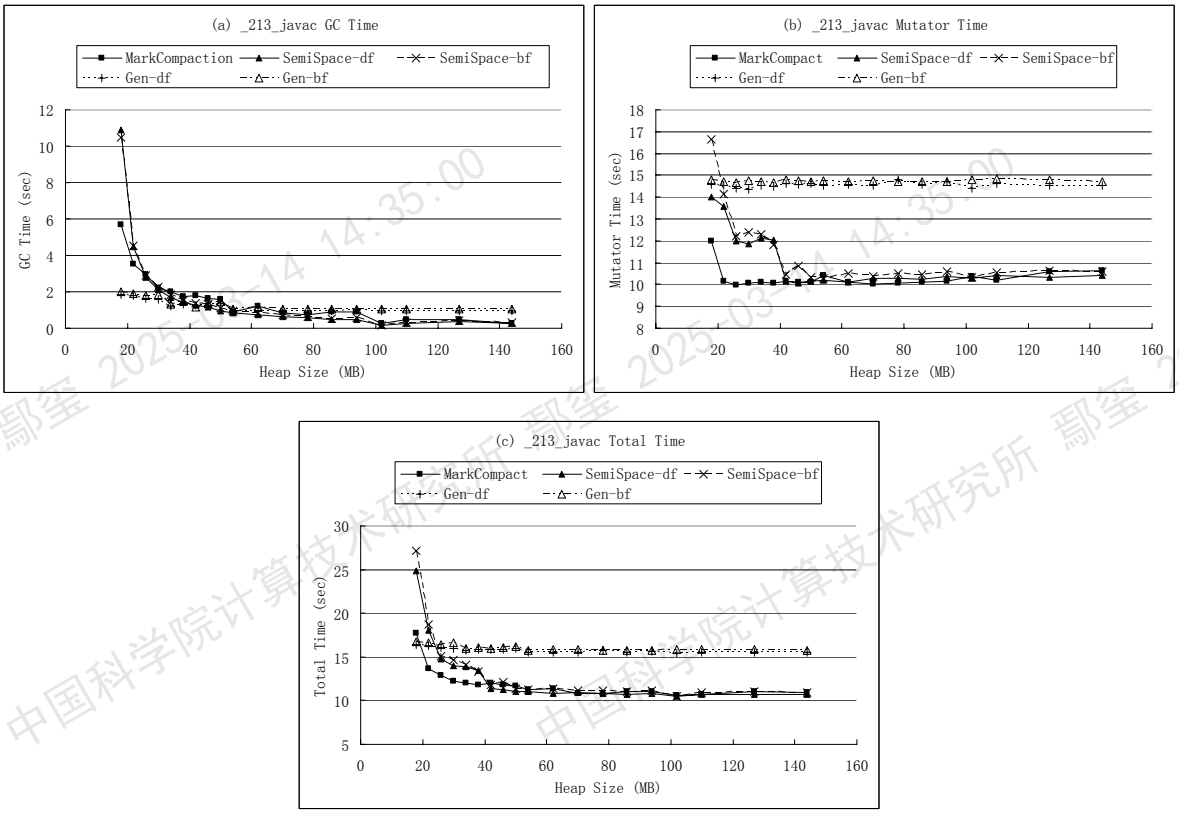

图 4.3 垃圾收集性能比较 _213_javac

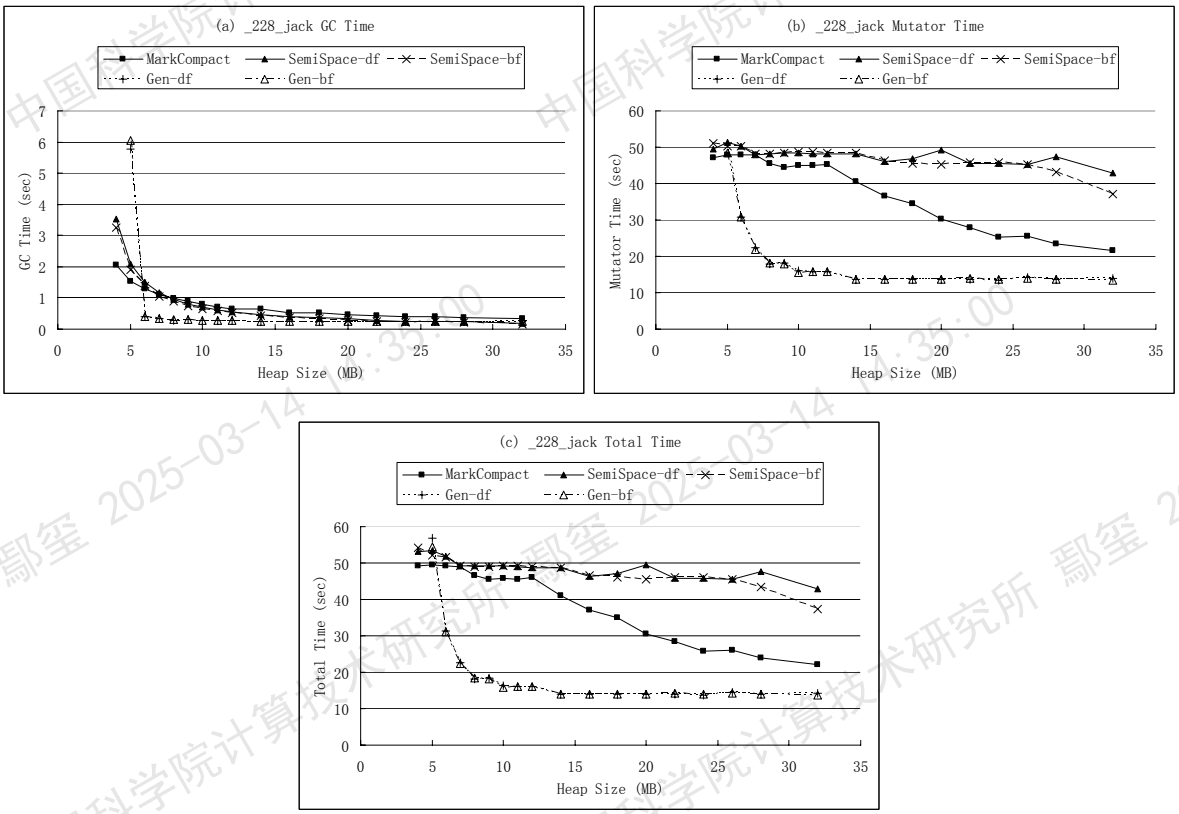

图 4.4 垃圾收集性能比较 _228_jack

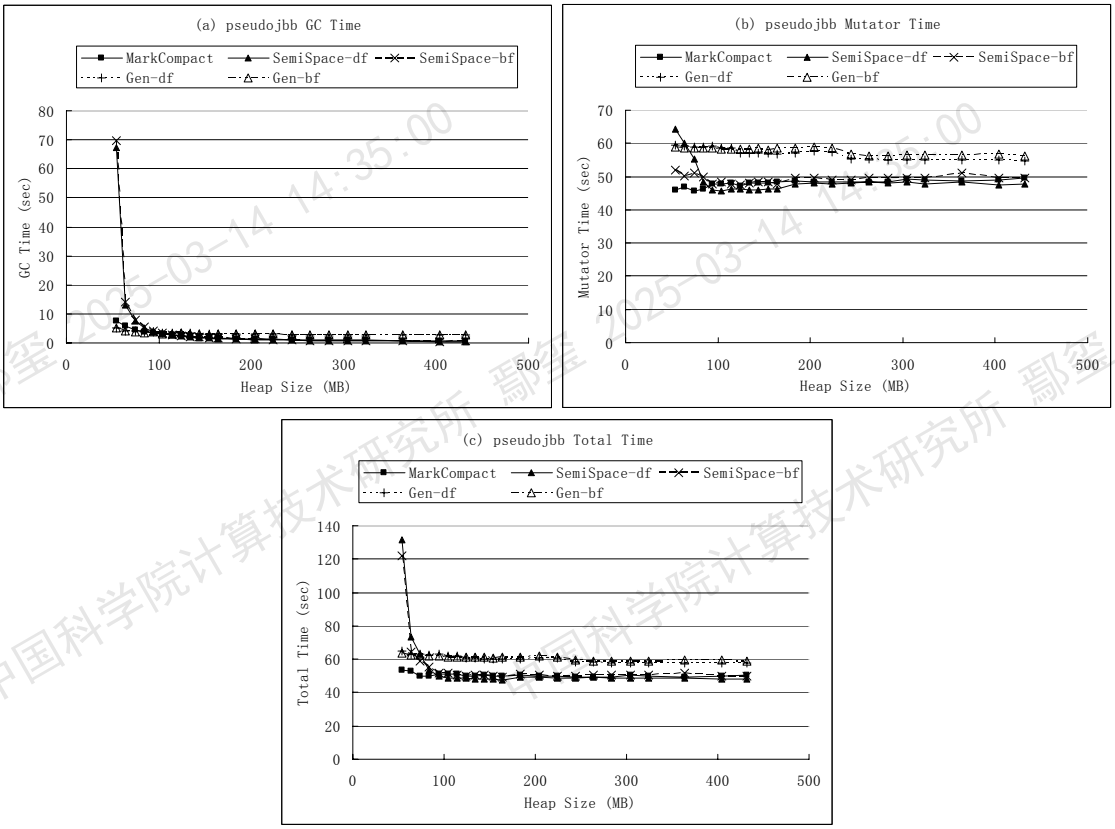

图 4.5 垃圾收集性能比较 pseudojbb

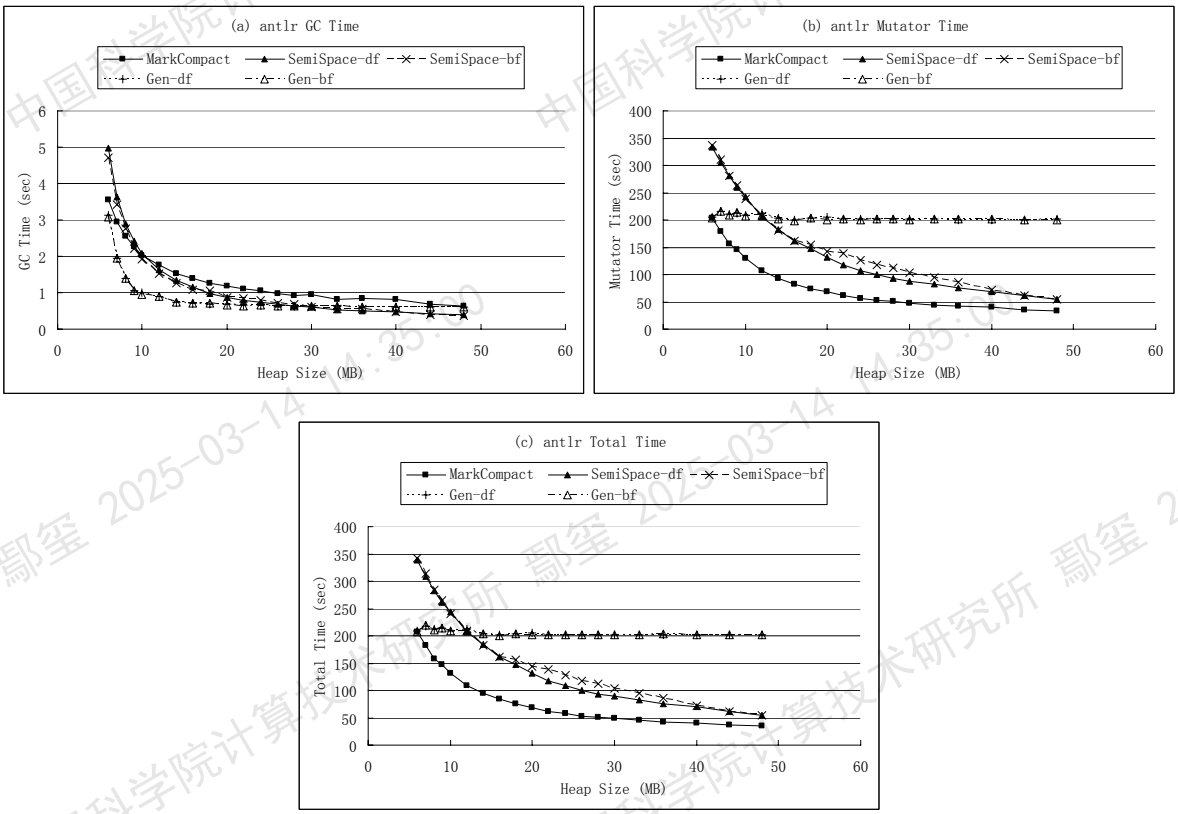

图 4.6 垃圾收集性能比较 antlr

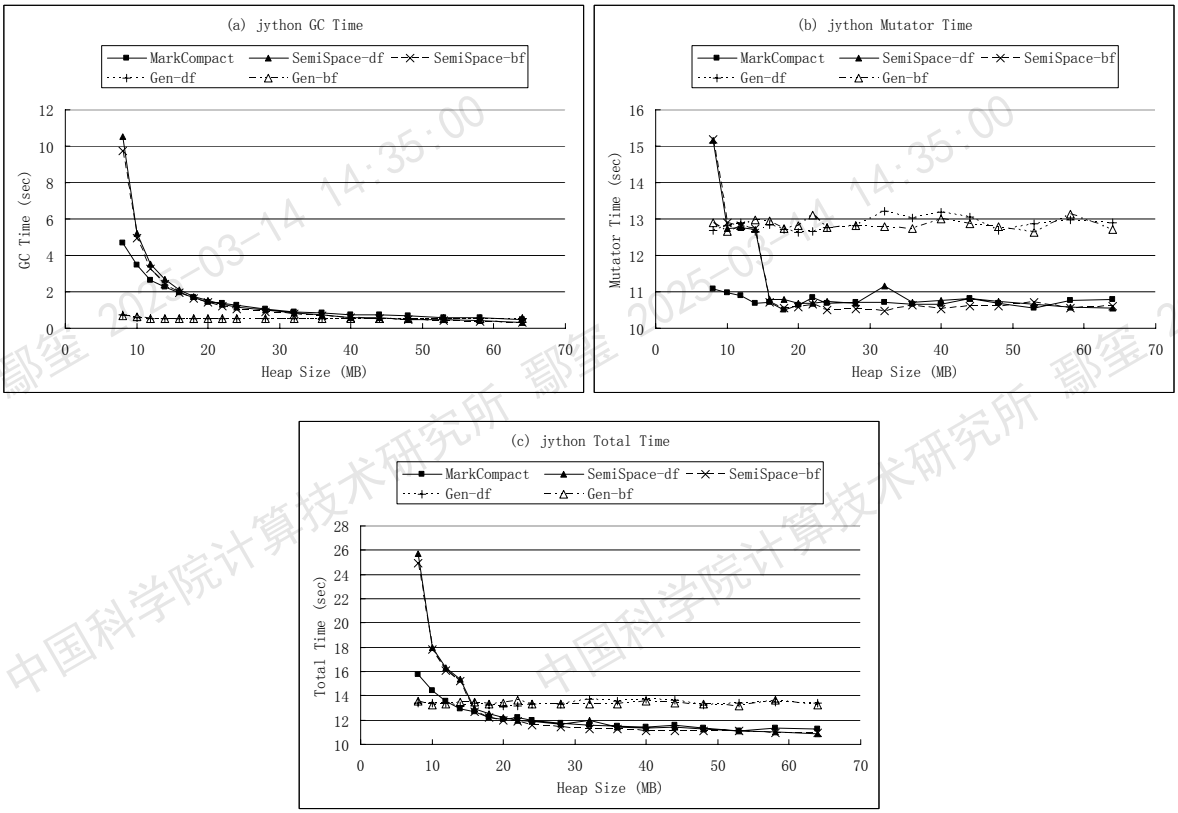

图 4.7 垃圾收集性能比较 jython

4.3 预取与局部性优化结合的软件框架

软件预取和数据局部性优化是用来改善内存系统性能的两种软件技术。预取的主要目标是容忍 cache 失效所带来的访存延迟，而数据局部性优化则主要是通过对数据的重新布局来减少 cache 失效发生的次数。本节的工作想要在面向对象的运行时系统中实现一个结合这两种软件技术的 framework，而这主要出于以下几个动机：

1. 面相对象的运行时系统具有较强的动态特性，程序的运行时行为经常会动态的改变。因此，不论是预取还是数据局部性优化都需要一种动态的机制来获取程序的运行时行为。在运行时系统中实现一个获取程序运行时行为的机制，能够被这两种软件技术所共用；
2. 预取技术和局部性优化技术相辅相成，它们之间能够互相补充。预取技术的有效性依赖于准确的预测将来需要被访问的数据项。应用程序中对数组类型数据的访问模式通常具有更强的规则性，而指针类型的数据结构的规则性则不那么强。因此，使用预取技术来处理不规则的指针运算会比较困难。我们于是希望能够通过局部性优化来缓解对指针类型数据的访问所带来的 cache 失效问题。预取技术的有效性还依赖于充足的访存带宽，而局部性的优化技术则与访存带宽没有直接的关系。另外，改变对象的布局只对数据的空间局部性有影响，这种局部性优化能够有效的一个前提是对象的大小小于 cache 行的大小。尽管在面向对象的程序中，普通对象的大小大都比较小，但是数组对象的大小却相对较大，一般都比 cache 行的大小大得多，因此，基于改变对象布局的局部性优化技术对于数组对象来说并不能起到实际的效果，而这些数组对象却更适合于使用预取技术来处理；

4.3.1 整体框架设计

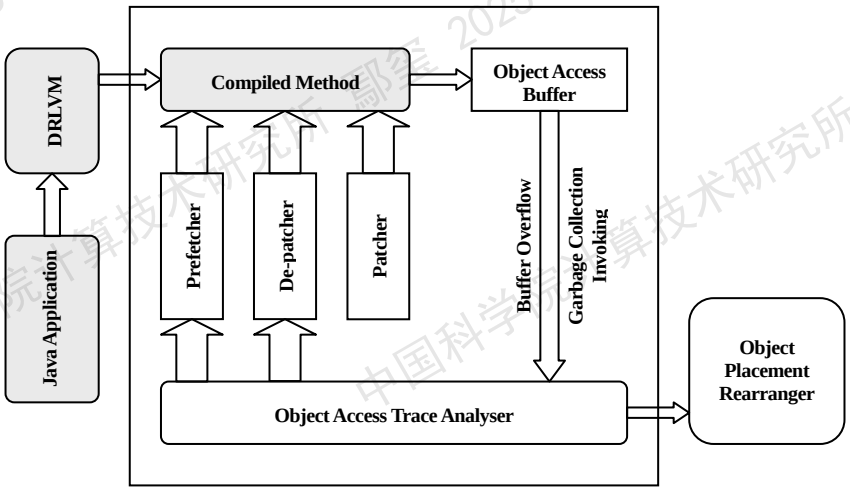

图 4.8 预取-局部性优化整体框架图

图 4.8 描述了预取与局部性优化结合的软件框架图，其中白色的方框为该 framework 中的各个部件。

Patcher: 用来给应用程序进行插桩的工具，由虚拟机调用。

Object Access Buffer: 一个 64K 字节的缓冲区，其中每一项大小为 12 字节。缓冲区自顶向下被顺序写入，如图 4.9 所示，定义缓冲区的顶部为 trace_buff_top, 底部为 trace_buff_bottom，其以下的页面属性被设置为只读。当缓冲区被填满而发生溢出时，便会产生段错误信号。

Object Access Trace Analyser: 该 Trace 分析器对对象访问缓冲区中的数据进行分析。如果对该分析器的调用来自缓冲区的溢出，则分析器产生对预取的指导——若分析的结果表明当前应用程序对数组对象的访问存在一定的规律性时，它将唤醒 De-patcher 和 Prefetcher。如果对该分析器的调用来自垃圾收集的发起，则它将通过构造对象亲缘关系图（Object Affinity Graph）来指导垃圾收集对对象布局的改变。

De-patcher: 用来对应用程序解除插桩的工具，由 Trace 分析器根据程序运行的实际情况自适应地调用。

Prefetcher: 用来对应用程序进行预取优化的工具，由 Trace 分析器根据程序运行的实际情况自适应地调用。

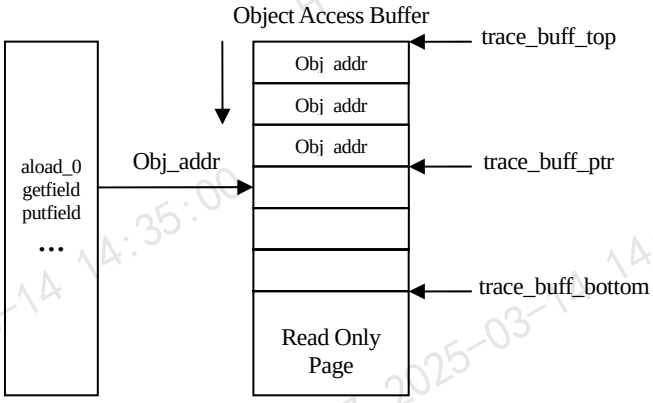

图 4.9 Object Access Buffer 的结构

```
profiled code for objects:
    push edx
    mov %edx,   trace_buff_ptr
    mov (%edx), objptr_reg
    mov 4(%edx), offset_reg
    mov 8(%edx), ip
    add %edx,   12
    mov trace_buff_ptr, edx
    pop edx
```

图 4.10 收集对象访问信息的代码

4.3.2 预取-局部性优化 framework 的工作流程

对堆中数据的访问可以分为三类：虚方法表、对象的域和数组元素，其中对象的域和数组元素的访问超过 90%。因此，我们的工作仅关注对对象域和数组元素的访问。Java 程序中使用 Java 字节码作为中间表示，其中可以用来表示对象和数组访问的指令有：getfield、putfield、aaload、aastore、baload、bastore 等。我们称这些指令为对象相关的指令。当一个基本块中包含对象相关的指令时，那么该基本块为对象相关的基本块。

预取-局部性优化 framework 的工作流程如下：

1) 在编译过程中，虚拟机调用 Patcher 对对象相关的指令进行插桩，插桩的代码如图 4.10 所示；
2) 应用程序在运行的过程中，将收集到的对象访问信息写入到 Object Access Buffer 中。该缓冲区记录了被访问对象的首地址，被访问的域在对象中的偏移（如果该对象是数组，则该项记录的是被访问的数组元素的 index 值）以及该对象访问指令的 pc 值；
3) 当 Object Access Buffer 溢出时，产生用户段错误例外，虚拟机接收到信号，唤醒 Trace 分析器；另外，当堆中空闲内存被耗尽而引发垃圾收集的时候，Trace 分析器也会被调用；
4) Trace 分析器判断被调用的原因，如果是被缓冲区溢出的信号处理程序所调用，则进行预取的优化，并删除插桩的代码；否则，调用来自垃圾收集，于是 Trace 分析器构造对象亲缘关系图来指导垃圾收集进行对象布局的优化；
5) 回到步骤 2 继续执行。

4.3.3 预取方法及其效果

我们的实现中所采用的预取方法主要是针对对数组元素访问的步长来进行的。在程序中规律的数组对象访问主要来自于三个方面：一是循环内不同的指令的访问步长；二是循环间相同指令的访问步长；三是频繁执行的小方法中相同指令的访问步长。我们对一些基准测试程序中收集到的数组对象访问信息进行分析，发现这些访问主要是来自后面两个方面。于是，我们主要实现了识别同一条指令所访问的数组元素之间的步长规律的方法。

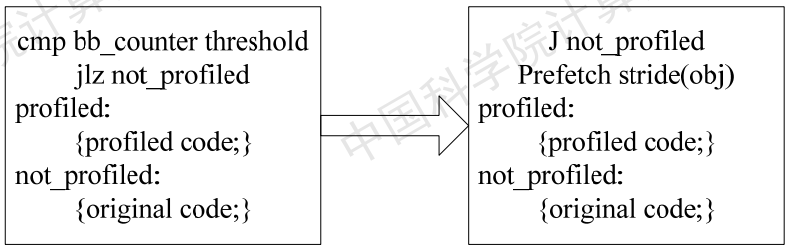

图 4.11 插入预取前和插入预取后的代码

我们的算法识别频繁发生的步长及该步长所对应的对象访问指令。假设当前指令所访问的数组元素的地址为 addr，热点步长为 stride，则算法在该指令处插入的预取指令为：prefetch (addr + d*stride)，其中 d 为预取距离。在插入预取指令后，算法通过 De-patcher 使 Java 方法无需再向 Object Access Buffer 中记录被访问的对象信息。插入预取后的代码如图 4.11 所示。插入预取指令前，我们只对热的基本块进行 profile。算法使用一个运行时基本块计数器 bb_counter 来计算基本块的热度。该计数器在基本块的入口处被增加 1，在碰到对象相关指令时与一个阈值进行比较。如果大于阈值，则 profiled code 将会被执行。

图 4.12 显示了我们的预取机制所带来的性能提高。我们使用了 SPECjvm98 和 DaCapo 中的一些基准测试程序。从图中可以看出采用了预取优化的程序性能提升了 1.5%到 18.1%。提升最多的程序是 ps，达到了 18.1%，其次 jess 和 fop 也有 13.8%和 8.3%的提高。由于该预取算法只是针对同一条指令的访问步长进行预取，因此，如果程序中规律的数组对象访问来自于循环内不同指令间的访问步长，那么该算法则不会有显著的效果，db 就是这样一个程序，它有 85%的时间运行在一个循环中，并且该循环针对较大的记录结构进行排序，因此在不同的指令之间存在规律的对象访问，而相同指令的规律访问则相对较少。所以该算法对 db 只有 1.5%的提高。

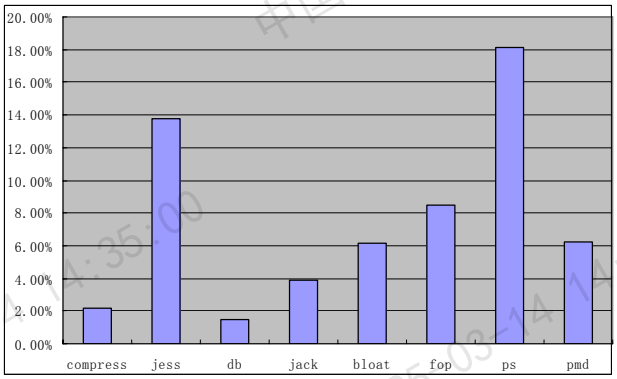

图 4.12 预取带来的性能提升

4.4.4 局部性优化及其效果

为了对对象的局部性进行改进，我们实现了[CL98]中的算法。该算法通过使用 Object Access Buffer 中记录的对象访问 trace 来构建对象亲缘关系图，其中图中的节点就是对象，节点间的边带有权重，表示的是对象之间的亲缘关系。其基本思想是，使用一个窗口从 Object Access Buffer 的顶端滑动到底端，每滑动一个记录项，就将落在窗口中的对象两两之间的边的权重增加 1。当窗口滑动完一遍的时候，对象的亲缘关系图就构建好了。这样，边的权重越大，则表示在对象的访问 trace 中，该边所连接的两个对象被相邻访问的频率越大，也就是这两个对象之间具有更好的空间局部性。因此，它们更应该被放在内存中相邻的位置。

在拷贝垃圾收集的过程中，算法首先按照权重的大小，深度优先的遍历亲缘图中的

对象，并将这些对象拷贝到目标地址空间中。该算法尽量地使权重大的边所连接的两个对象被放在相邻的内存地址上。当处理完亲缘图中的对象之后，该算法再按照正常的收集方法将堆中剩余的存活对象拷贝到目标地址空间中。

我们在分代垃圾收集中实现了该算法。在我们的分代垃圾收集中，对年老的分代使用的是标记缩并的收集方法。收集中存在两个改变对象布局的地方：一是将对象从年轻分代拷贝到年老分代的过程，即次级收集，二是将年老分代中的对象进行缩并的过程，即主收集。我们分别将局部性优化的算法应用在这两个过程中，但是并没有取得明显的效果。经过分析，我们总结了如下几个原因：

- 对于次级收集来说，如果对象的生命周期行为与弱分代假设十分吻合，则每次收集时存活对象的数量都很小，那么对这些极小量的存活对象进行布局的改变并不能明显地影响整个运行时系统的局部性。
- 由前一节中对分代收集的静态遍历序的比较可以发现，对于大部分的应用来说，Java 程序并没有表现出其对象局部性对收集时拷贝序的依赖。
- 对于主收集来说，标记缩并算法能够保持对象的分配序，这已经能够带来非常好的对象局部性了，因此，在这种情况下，再去改变对象的布局很难取得明显的效果。
- 我们通过分析垃圾收集时所产生的对象亲缘关系图，发现大部分的对象之间的亲缘关系非常小，而且经常是相同的，这种情况严重地影响了该局部性优化算法的有效性。如果把这些亲缘关系小的对象从图中过滤掉，则会发现真正具有较好的空间局部性的对象非常少。这些对象很可能由于被经常地访问而已经被缓存在 cache 中了。

4.4 Local memory 对内存系统性能的影响

先前的一些研究工作表明，与相同容量的 cache 组织结构相比，local memory（一些研究者也将其称为 scratchpad memory）具有更小的面积和更低的功耗，并且具有更少的访问时间[Banakar02]。这主要是因为 local memory 不需要额外的资源来存储 cache 行的 tag，也不需要对 tag 进行比较以确定数据的有效性。Local memory 占用了一段连续的地址空间，它的访问方式与内存相同。

Java 运行时环境的一个特点就是对于客户端的应用来说，大部分的对象都具有很短的生命周期。对于分代垃圾收集来说，所有的对象都被分配在年轻的分代中，这些对象中的大部分仅仅被访问了几次之后就死去了。考虑这些对象的亲缘关系并没有太大的意义，因为这些对象的访问模式并没有很多的重现，因此，很难从改变这些对象的布局中得到好处。对年轻分代中对象的访问更像是对一个连续的线性数组的顺序访问。对象的访问行为并不具备好的时间局部性，而空间局部性却相对较好，因此，如果能够更好的利用对象的空间局部性，应该能够得到更好的效果。在 cache 的组织结构中，cache 行被用来利用数据的空间局部性，所以，我们可以合理地期望在这种情况下，cache 行的大小越大，所能得到的性能好处越多。然而，在通用处理器的 cache 组织结构中，cache 行

的大小并不能够任意的增大。

结合 local memory 的硬件好处和 Java 运行时环境的对象行为特点，我们很自然地想到去尝试使用 local memory 来存储这些短生命周期的对象。Local memory 在本质上其实就是一个较大的 cache 行，它可以被用来更好地利用对象的空间局部性。使用 local memory 与 cache 结合的系统，我们能够利用 local memory 来处理空间局部性较好但时间局部性却不太好的对象数据，从而让 cache 能够更加专注于处理时间局部性好的对象数据，以获得更优的性能。

我们使用 MIPS 版本的 JDK1.5 和 SimOS 来进行有关 local memory 的实验。我们扩展了 SimOS 的体系结构，增加了一个可以配置地址空间和容量大小的 local memory 模块。该 local memory 模块与 cache 同时存在，当访存操作所访问的地址在 local memory 所映射的地址空间的话，则该次访问将直接读写 local memory 中的数据，而不会经过 cache 和内存的存储层次。同时，我们扩展了 linux-mips 的 sysmips( )系统调用，Java 虚拟机通过该系统调用来对 local memory 进行配置。

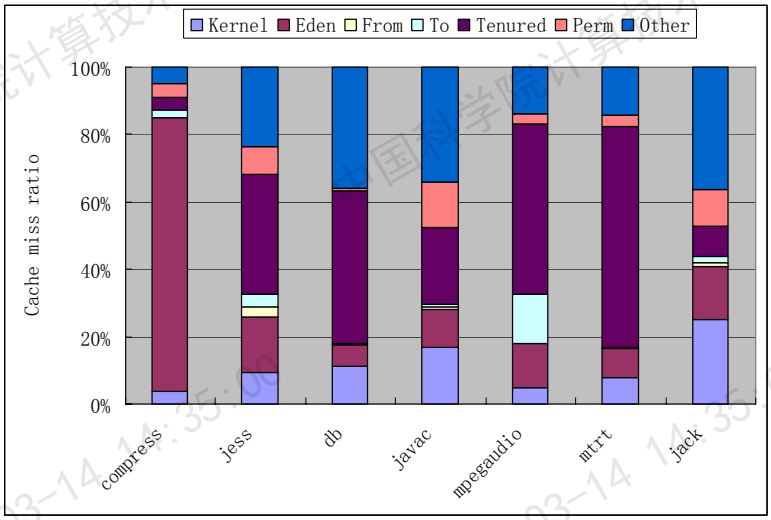

图 4.13 使用 JDK1.5 运行 SPECjvm98 时堆中不同区域的 cache 失效次数的比例

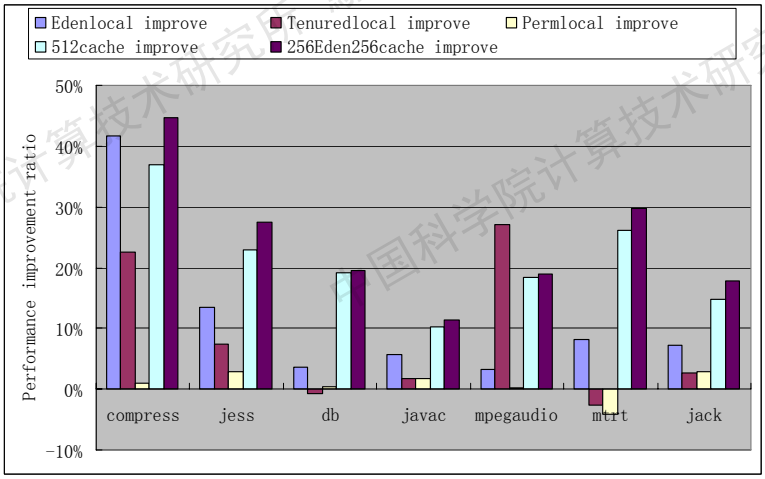

图 4.14 Local memory 对内存系统性能的影响

上一章已经描述过 JDK1.5 中堆空间的区域划分，本章就不再重复讨论了。图 4.13 显示了在使用 JDK1.5 来运行 SPECjvm98 中的基准测试程序时，堆中不同区域的 cache 失效次数的比例。其中“Kernel”是指内核空间中的 cache 失效，“Other”则是除了图例中其它区域之外的所有 cache 失效（主要是访问虚拟机内部数据而引起的 cache 失效）。数据 Cache 的大小被配置为 64KB。

由于在真实的硬件中，local memory 并不能被实现得非常大，而且年轻的分代也不应该比 local memory 大很多，因为这样会大大降低 local memory 的效果，因此，我们将 Eden 区的大小配置为 512KB。图中的数据表明对于大部分的应用来说，cache 失效主要发生在 Eden 和 Tenured 区域中。From 和 To 区域作为起过渡作用的对象老化区域，只引入了可以忽略的 cache 失效。

我们尝试将 local memory 分别映射到 Eden，Tenured 和 Perm 区域的低地址段来比较相应的效果。图 4.14 显示了实验的结果数据。图中显示的是 local memory 和 cache 的各种配置所带来的性能改进，其中用来比较的性能基准值是数据 cache 为 64KB 时的性能。图中不同配置的具体含义如下：

Edenlocal：512KB local memory 被映射到 Eden 区，数据 cache 大小为 64KB；

Tenuredlocal：512KB local memory 被映射到 Tenured 区的最低端，数据 cache 大小为 64KB；

Permlocal：512KB local memory 被映射到 Perm 区的最低端，数据 cache 大小为 64KB；

512cache：无 local memory，数据 cache 大小为 512KB；

256Eden256cache：256KB local memory 被映射到 Eden 区的最低端，数据 cache 大小为 256KB；

从图中可以看到，Edenlocal 能够得到较大的性能提高，特别是对于 compress 来说，因为 compress 的绝大部分的 cache 失效都发生在 Eden 区中。对于除 mpegaudio 以外的所有基准测试程序来说，Edenlocal 都比 Tenuredlocal 取得了更多的性能改进。其中的一个原因是 Tenured 区域相对来说是很大的，也就是说，对象的访问被分布到了很广泛的区域中，而真正落在 local memory 所覆盖的地址范围的访问就很少了，因此，不能得到明显的好处。Mpegaudio 是一个例外，这主要是由于 mpegaudio 只分配很少的对象，并且对象的寿命较长，它们都被提升到年老的分代，并且仅仅占据了 Tenured 空间中很小一部分的低端区域，因此，将 local memory 映射到 Tenured 区域的最低端已经足以覆盖大部分的对象访问，从而取得了较大的性能改进。

图 4.14 中还对 local memory 和 cache 相结合的结构与相同容量的 cache-only 的结构对性能的影响进行了比较。结果表明，尽管与 Edenlocal 相比，512cache 能够带来更好的性能，但是 256Eden256cache 却总是能够比 512cache 得到更多的性能改进。这说明使用 local memory 来缓存短生命周期对象的技术仍然是很有希望的。

4.5 本章小结

本章的工作从多个方面讨论了垃圾收集算法对应用程序的对象局部性的影响。我们对拷贝收集的对象遍历序进行了比较，结果表明，大部分 Java 应用程序的对象局部性对此并不敏感。在比较敏感的个别程序中，对象遍历序对程序局部性的影响主要来自于分代内不同分区中的拷贝收集。标记缩并由于保持了对象的分配序而在大部分情况下产生了较好的空间局部性。

我们设计并实现了一个结合预取机制和对象局部性优化的软件框架。预取技术主要用来对数组的访问进行优化，而局部性优化算法则主要针对于普通的对象。实验结果表明，对数组访问的简单的预取能够取得较为明显的效果，而在年轻分代的拷贝中和年老分代的标记缩并中改变对象的布局却并没有明显的改善。我们分析并总结了其中的原因。

我们调查研究了 local memory 对 Java 运行时环境中的内存系统性能的影响。实验结果表明，local memory 与 cache 结合的结构比单纯的 cache-only 的结构具有更优性能。

中国科学院计算技术研究所 鄢玺 2025-03-14 14:35:00

中国科学院计算技术研究所 鄢玺 2025-03-14 14:35:00

第五章 基于长生命周期对象的混合垃圾收集

5.1 引言

在垃圾收集系统中存在着许多内在的动态特性，因此，在设计一个垃圾收集系统的时候，人们通常会面临着许多重要的决策。

1．因该使用哪种分配算法；

2．应该在什么时候启动垃圾收集；

3．如果垃圾收集被启动，应该使用什么样的算法；

4．什么时候堆应该被扩展以及如何扩展；

5．应该如何组织堆的结构，分代，分区……

通过对前人工作的总结，我们可以把垃圾收集的动态机制大致地划分为 3 类。基于参数、基于算法和基于年龄的动态机制。

基于参数的动态机制主要考虑的是如何动态地改变堆的大小以及如何根据某种机制来控制垃圾收集发起的时机。垃圾收集对于运行时系统来说似乎总是会带来开销，人们也许会认为，完全不进行垃圾收集会使运行时系统达到最好的性能。然而，事实并非如此，垃圾收集一个重要贡献就是控制了应用程序的 footprint。如果应用程序所访问的虚地址空间超过了实际的物理内存，那么缺页故障就会频繁地发生，程序的性能就会严重下降。垃圾收集通过调整堆的大小来缓和这种恶劣的情况，因此这种动态机制通常是考虑如何有效地利用内存资源[AA90, Cooper92, Brecht01, Yang04, Zhang06]。另一方面，既然垃圾收集是必需的，那么尽可能地减小垃圾收集所带来的开销就自然地成为了人们研究垃圾收集的一个重要目标。控制垃圾收集发起的时机能够给减小垃圾收集的开销带来一定的帮助。对于拷贝垃圾收集来说，收集的开销与存活对象的数量成比例，因此，理想的收集时机应该是存活对象的数量较少的时刻[Buytaert05]。

基于算法的动态机制主要是考虑如何利用某种算法的优点来弥补另一种算法的缺点。垃圾收集系统的性能是和应用及资源环境密切相关的，已经有大量的先前的研究表明，没有一种垃圾收集算法能够让任何类型的应用在任何环境下都取得最好的性能。每种垃圾收集算法都有一些内在的优缺点使得它更适合于某些应用，却不太适合于另外一些类型的应用，我们的经验研究也说明了这一点。这意味着，设计一个能够在不同的垃圾收集算法之间动态切换的系统是非常有必要的。

然而，动态切换算法的切换策略的确定却不是一件十分容易的事情。一些研究者们通过离线的 profile 的方法来确定在什么情况下使用什么垃圾收集算法会比较好，并且将这种最优的搭配静态的绑定。这种方法对于一些静态编译的系统来说是有一定用处的，但是它的效果也依赖于训练输入集的选择。如果系统的性能只是对某一些输入集会比较

好，而对另外一些输入集却会比较差，则该系统的性能也可能是不稳定的。特别是对于像 Java 虚拟机这样的动态的运行时环境来说，这种离线的 profile 的方法也很难被广泛地应用。而且这种方法通常更侧重于对表面现象的观察，缺少对内在的本质问题的深入分析。另外一些研究者经过对某些垃圾收集算法的行为特点的研究和比较，把一些算法进行组合，使这些算法能够随着运行环境的变化而动态地切换，从而互相弥补对方的缺陷，以达到更好的性能。这种做法倾向于解决更本质的问题，一般具有更好的普适性。

另外，评估一个垃圾收集系统性能的另一个重要因素是该系统所使用的算法对对象的局部性所造成的影响。这个因素增加了制定算法切换策略的困难，因为收集器对局部性的影响是很难量化的。

基于年龄的动态机制主要是针对对象的年龄分布的特点，对不同年龄的对象进行不同的处理。分代垃圾收集是针对对象年龄问题的一个经典的垃圾收集算法。简单的分代垃圾收集并没有太多的动态的机制，然而基于年龄的动态机制大都是在某种分代垃圾收集的框架上实现的。

分代垃圾收集的提出是基于弱分代假设的成立，即大多数的对象都在年轻的时候死亡，少数的对象会存活更长的时间。在一个只有 2 个分代的分代垃圾收集中，如果没有对提升阈值的控制，即每次次级收集都提升所有存活的对象，则该算法通过次级收集只能将对象粗略地按照年龄分隔在两个分代中。这样，在年老的分代中也会存在一些很年轻的对象，这些对象可能会很快地死亡并成为占位垃圾，而占位垃圾的出现会降低分代垃圾收集的效率。

某些研究者们提出了一些控制提升阈值的机制，他们的目标是要让最年轻的对象有一定的时间去死亡，而不是在次级收集中立刻被提升到年老的分代中。对于某些应用来说，这种做法能够在一定程度上减少占位垃圾的出现，但是那些实际上寿命很长的对象也要在年轻的分代中被重复复制若干次后才会被提升，因此这种做法也不是完全没有代价的。另外，这些控制提升阈值的机制大都是静态的，也就是采用固定的提升阈值。由于应用具有动态的特性，因此静态地提升阈值对于某些应用来说很可能也不是十分地有效。

如果弱分代假设能够很好地成立的话，分代垃圾收集就可以从次级收集中得到相当多的好处，因为能够在次级收集中存活的对象将会非常少。然而，如果情况并不是这样，则分代垃圾收集的性能就会受到一定的影响，那些长生命周期的对象也要经过次级收集才能够被提升到年老的分代中。如果我们可以预先地知道某些对象就是长生命周期的对象，那么这些长生命周期的对象就可以直接在年老的分代中被分配，而不用经过提升。这种做法有两个好处：

1. 由于一部分长生命周期的对象被过滤，一次次级收集中会有更少的对象被提升，因此一次次级收集的效率会提高，并且次级收集所带来的暂停时间会减少。
2. 由于一部分长生命周期的对象被直接分配在年老的分代中，这使得次级收集发生的频率会有一定程度的下降。

前面所讨论的 3 类动态机制相互之间都是正交的（orthogonal），即每一类机制都可以很自然地和其他的机制同时被应用在垃圾收集系统中。基于参数的机制相对来说比较独立，特别是通过控制堆的大小来充分利用物理内存资源的策略，一般来说可以被应用在大多数的场合中，而并不是十分地依赖于收集的算法和对象生命周期的特点。

基于算法的动态机制和基于年龄的动态机制也能够被共同地应用在同一个垃圾收集系统中。有不少的动态切换收集算法的策略都是针对于非分代的垃圾收集的，比如标记清扫和节点复制的混合[Lang87]以及标记清扫和标记缩并的混合[Printezis01]等等。这种基于算法的动态机制通常能够很自然地被应用在分代垃圾收集的年老的分代中。然而，在这样的收集系统中，基于算法的切换并不依赖于对象年龄的信息。而本章的研究工作，提出了一种新的动态机制，它利用对象的生命周期信息来指导算法的切换，从而使垃圾收集器能够尽可能地避免对长生命周期的对象进行重复的操作，提高了垃圾收集系统的性能。实验结果表明，与经典的分代垃圾收集相比，我们的收集算法能够使 SPECjbb2005 的 Throughput 分值提高 3%~25%。

本章的组织结构如下：5.2 节介绍了基于对象年龄特性的相关工作；5.3 节讨论了本章研究的目标；5.4 节描述了评价垃圾收集系统性能的标准；5.5 节对服务器端的基准测试程序 SPECjbb2005 的行为特点进行了分析；5.6 节介绍了基于长生命周期对象的混合垃圾收集算法；5.7 节讨论了对 5.6 节中的算法所进行的改进；5.8 节对本章的工作进行了总结。

5.2 基于对象年龄特性的相关工作

5.2.1 控制提升的阈值

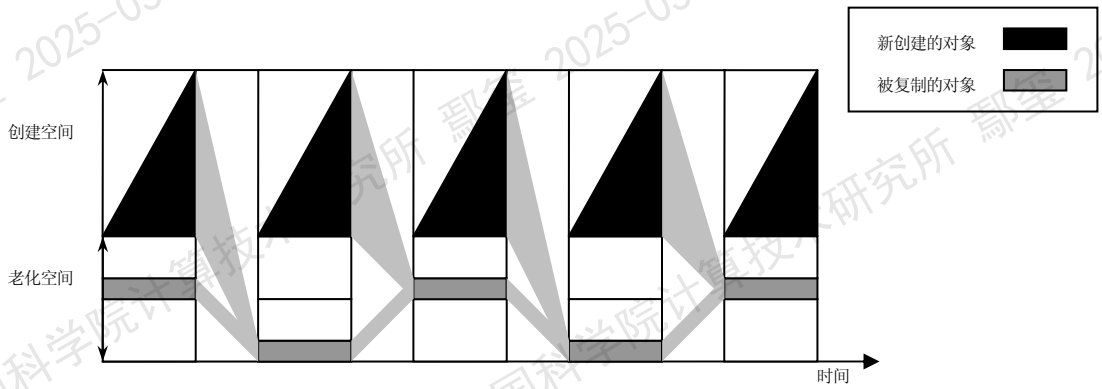

图 5.1 在 nursery 中使用分离的创建空间

为了控制提升的阈值，Ungar 提出了一个比较巧妙的技术。他把年轻的分代划分为两个子空间：创建空间和老化空间[Ungar84]。老化空间由 2 个半空间组成。新的对象只能够在创建空间中被分配。当创建空间耗尽的时候，次级收集被发起，创建空间和老化

空间中的存活对象被一起复制到老化空间的 to-space 中。已经进入老化空间的对象将会在组成老化空间的 2 个半空间之中被来回复制一定的次数，然后被提升到年老的分代中。图 5.1 描述了这一过程。如果应用的行为遵循弱分代假设，则每次次级收集中存活的对象的数量会很少，因此组成老化空间的这 2 个半空间也可以保持得相当小。但是，Ungar 的这种方案需要有空间来存放对象的年龄，即对象在老化空间中被复制的次数。一般来说，对象头的元数据结构（meta-data）是最适合存放对象年龄的地方。然而，这些元数据中是否有保存对象年龄的位置取决于运行时环境对对象模型的定义。据我们调查，SUN 的 JDK1.5 中所使用的分代垃圾收集就使用了 Ungar 的这种设计，并且在他们的虚拟机的对象模型中，也保留了记录对象年龄的空间。但是，apache harmony 的 drlvm 虚拟机的对象模型中却没有为记录对象的年龄预留空间。

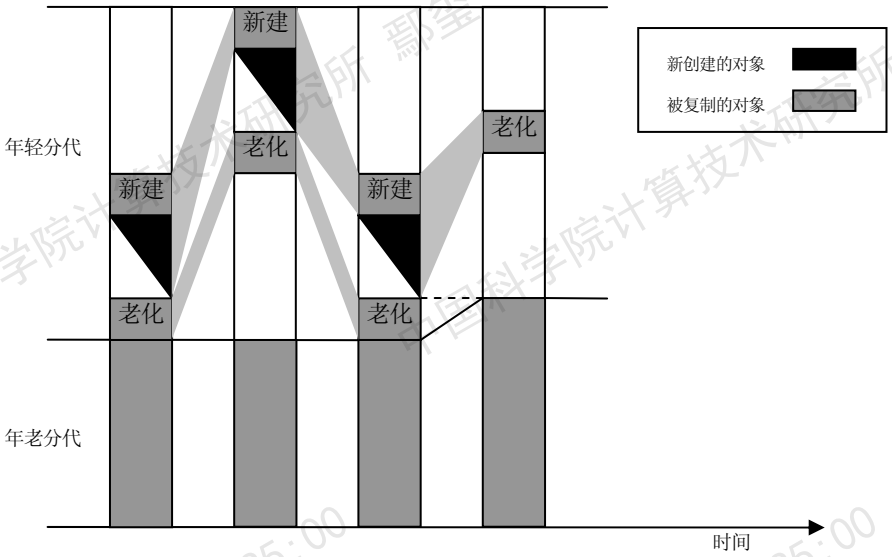

图 5.2 Shaw 的分“桶”方案

为了避免在每个对象内保存年龄，Shaw 提出了一个方案。他把年轻的分代进一步细分成 2 块或多块区域，这些区域被称为“桶”[Shaw88]。如图 5.2 所示，年轻的分代被分为新建桶和老化桶。在经过 n 次收集之后，新建桶中所有的存活对象被提升到老化桶中，老化桶中的存活对象则被提升到年老的分代中。这种方案保证了任何到达年老分代的对象都已经幸存过了 n+1~2n 次收集。

在 Shaw 的方案里，年老的分代位于内存的底部，向上增长。年轻的分代则紧随其上，并且与年老的分代相连。年轻的分代被分为两个半区，新建桶占据了半区的顶部，老化桶则占据了半区的底部。只有在垃圾收集将新建桶中的对象向上复制，也就是远离年老的分代的时候，才能进行提升。此时，老化桶正好位于年老的分代的上方，并与之相连。这时，只需要简单地向上移动两个分代之间的边界，就能立刻提升老化桶中所有的对象。

Wilson 和 Moher 结合了 Ungar 的创建空间和 Shaw 的年龄记录技术[WM89]。为了降低 Shaw 的方案在控制提升时的复杂性，他们的方案使用了更加简单的 2 桶机制。年轻

的分代仍然是被划分为 2 个桶，其中创建空间是一个桶，老化空间是一个桶。在一次次级收集的时候，创建空间中的存活对象被复制到老化空间中，而老化空间中的存活对象则被提升到下一个分代中。由于这两种复制操作是同时进行的，因此老化空间也必须被组织成两个半区的形式，其中一个空闲的半区用来存放从创建空间复制过来的存活对象。采用这种组织方式，这个系统中提升的年龄阈值是 2 次收集。Wilson 认为，高于 2 的提升阈值只会带来越来越小的回报。

另外，Wilson 的算法还允许在创建空间设置一个“水位线”。比这个水位线年老的对象将能够被直接提升到下一个分代中去。通过这种机制，Wilson 的收集器允许一些对象尽早地被提升。

5.2.2 基于年龄的（age-based）垃圾收集

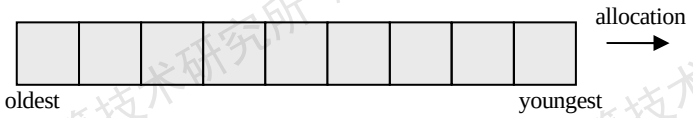

图 5.3 按照对象年龄排序的堆的视图

Stefanovic et al. 对分代式垃圾收集进行了概括和抽象，提出了一种基于年龄的垃圾收集的抽象模型[Stefanovic99]。在他们的模型中，对象按照年龄被线性地排放。如图 5.3 所示，最左边的对象具有最大的年龄，最右边的对象具有最小的年龄，新分配的对象被排放在最右边。当垃圾收集发生的时候，堆被分为两个区域：收集区和余留区。收集区是垃圾收集真正关心的区域——收集器会去遍历收集区中的对象，从而检查该区中对象的存活性。而对于余留区，收集器则假设该区中的对象都是存活的，而不再去关心它们。根据收集时对象的存活性，Stefanovic 的方案把收集区进一步分为存活集合和垃圾集合。

他们的方案考虑了两种年龄的概念：真实年龄和新生年龄。基于真实年龄的垃圾收集方案不会改变对象在堆中的相对顺序。对象从被创建起，就被赋予了永久的年龄。如果垃圾收集会把存活的对象重新移动到堆的最右边，即赋予了对象新生的年龄，就好像存活的对象是新分配的一样，那么这种收集方案就是基于新生年龄的。

根据他们的抽象，他们把垃圾收集方案分为以下几种：

1. True youngest-first collection
2. Generational youngest-first collection
3. True oldest-first collection
4. Deferred older-first collection
5. Deferred younger-first collection
6. Renewal-oldest-first collection

他们的实验结果表明，在大部分情况下，Deferred older-first(DOF)的收集方案会取得相对较好的性能。这主要是因为该方案在遵循弱分代假设的同时，也考虑到了最年轻的对象不太可能已经死去的事实。图 5.4 示意了 DOF 的收集过程。

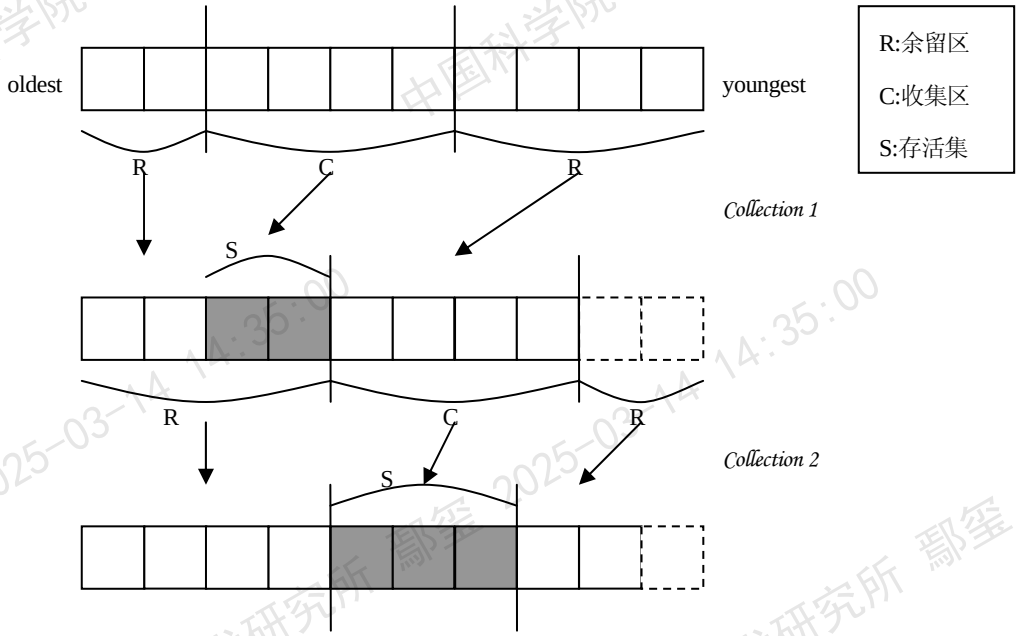

图 5.4 Deferred older-first 收集方案

5.2.3 对象的预提升技术（pretenuring）

预提升是一种通过将长生命周期的对象直接分配在垃圾收集很少发生的区域里，从而减少次级收集的拷贝开销的一种技术。

Ungar 和 Jackson 在 Smalltalk 语言系统中的一个 2 代的垃圾收集器中通过 profile 来识别长生命周期的对象[Ungar88, Ungar92]。他们的系统中的年老的分代是从来都不会被收集的。对象的分配不会发生在年老的分代中，只有在年轻代中存活过若干次次级收集的对象才会被拷贝到年老的分代中。他们的系统会把长生命周期的对象保存在年轻的分代中，重复地收集而不急于提升它们，以尽量避免占位垃圾的产生。他们也注意到存在一些永远不死（immortal）的对象，不过由于这些对象在他们的系统中并不重要，因此没有对它们采取特殊的措施。

Cheng et al. 在 ML 语言系统的一个分代垃圾收集器中对预提升和对象生命周期预测技术进行了评估[Cheng98]。与 Ungar 和 Jackson 相似，他们也把堆分为 2 个区域：固定大小的 nursery 和一个年老的分代。他们使用一种离线的 profile 的方法，并且在垃圾收集的时候搜集 profile 信息，然后根据这个信息来生成预提升的建议（advice）。然而他们所生成的预提升建议依赖于他们所使用的垃圾收集器和特定的配置。

Blackburn et al. 延续并改进了 Cheng 的工作[Blackburn01]。他们也使用了一种离线的 profile 的方法，不过他们的 profile 更加详细。他们记录了堆中对象分配、指针改变和对象死亡的精确信息，因此他们的 profile 需要更长的时间。根据他们的细粒度的 profile 信息，他们能够产生与垃圾收集算法及配置无关的预提升建议。他们在研究中发现，对于 Java 程序来说，使用位于分配点的对象生命周期信息来进行预测是非常准确的。另外，由于他们所使用的 JikeRVM 虚拟机平台本身也是用 Java 语言实现的，因此他们对对象

生命周期的 profile 可以被应用在虚拟机上。这样他们能够仅通过对虚拟机的方法产生预提升建议来改进性能，而无需对每个应用都进行 profile。当然，如果这样做的话，也会失去对 Java 应用程序进行预提升所带来的良机。

这种预提升的技术旨在预测长生命周期的对象，并且与本章所提出的动态技术是正交的。该技术可以和我们的动态技术同时使用，以增进其效果。另外，如果预提升技术对对象生命周期的预测不够准确的话，我们的动态混合垃圾收集技术也不会受到影响。

5.3 研究的目标——服务器端的应用

随着互联网时代的到来，人们对于部署高性能服务器的需求已经变得越来越强烈。现在，几乎任何一家具有一定规模的公司都需要在互联网中部署能够对任意数量的客户提供任意时间的服务的服务器。客户机/服务器模型已经成为互联网中最为主流的应用模式。

在服务器端的应用中，新技术的出现是相当快的，用户对新功能的需求也在不断地增加，而且服务器端的软件规模一般也是相当庞大的。基于这些原因，服务器端的应用程序的开发，通常应该采用面向对象、可移植和类型安全的语言系统。这种语言系统通常具有代码可重用及更容易进行模块化的编程等优点从而使服务器应用的快速开发成为可能。Java 语言正是一种具有这些特点的编程语言，它的类型安全和自动内存管理等优点，使得它被广泛地部署在许多现有的服务器平台上。

然而，与客户端的应用程序相比，服务器端的应用程序更容易受到垃圾收集系统的影响。这主要是因为以下两个原因：

1. 服务器端的应用程序通常需要运行在多线程的环境中，而且线程的数量会非常多。这意味着，服务器端的应用程序对对象分配的需求是相当巨大的。这种巨大的对内存资源的需求，经常会导致更加频繁的垃圾收集。
2. 服务器端的运行时系统一般会使用比较大的堆空间，因为服务器应用的工作集（working set）通常会比较大。在管理这种比较大的堆空间的时候，许多垃圾收集算法都有可能会带来很大的开销，同时造成很大的延迟。

另外，服务器端的应用程序比客户端的应用程序更需要一些动态机制的支持。一个高性能的服务器系统应该能够提供一个持续的执行环境，在这个执行环境中，多个不同的应用和代码组件会被用户动态地装载[HP, IBMa, BEA]。在这样的运行环境中，任何静态的方法和离线的 profile 机制都不再有效。如果在服务器系统运行的过程中，它所使用的垃圾收集影响了系统的性能，缺少动态的调整机制将会使该系统无法从这种糟糕的情势中恢复出来。

而对于客户端的应用程序来说，采用离线的 profile 的方法来指导对垃圾收集系统的调整通常都是可能的，而且是比较有效的。另一方面，由于客户端的程序的工作集较小，因此运行时系统通常在使用相对较小的堆的时候就能够极大地降低垃圾收集所带来的时

间开销。在这种情况下，垃圾收集对运行时系统性能的影响主要来自于它对程序的局部性的影响。然而，利用程序局部性的信息来指导垃圾收集的动态调整，相对来说是更加困难的。

基于以上这些原因，本章的研究工作主要关注于对服务器类型的应用的优化。

5.3.1 经典垃圾收集算法在服务器类应用中存在的问题

在本小节中，我们主要讨论以下几种经典的垃圾收集算法：标记缩并，基于半区的节点复制和简单的分代垃圾收集。

经典的标记缩并收集算法主要包括 4 个阶段：

1. 从根集出发，遍历并标记所有存活对象。
2. 计算存活对象将要被移动到的目标地址，并将目标地址作为迁移地址保存在对象头中。
3. 遍历所有存活对象的引用域，并根据引用域所指向的对象的迁移地址来更新引用域的值。
4. 将所有存活的对象拷贝到目标地址中。

其中第 2 个阶段的时间复杂度是和堆的大小成比例的。由于在服务器系统中，堆空间都被配置得非常大。因此，一次全堆的标记缩并收集将会带来非常大的时间开销。更糟糕的是，如果应用的工作集也很大的话，一次标记缩并能够回收的自由空间也相对减小，这使得收集会发生的更频繁，从而导致严重的性能问题。

基于半区的节点复制的收集开销同存活对象的数量成比例，因此，如果每次垃圾收集过程中，堆中的存活对象的数量很少的话，该算法应该能够获得不错的效率。但是，在服务器类型的应用中，情况往往并不是这样。随着应用线程的增多，应用的工作集也会相应地增大，节点复制的垃圾收集算法的效率会急剧地下降。由于该算法能够有效使用的内存空间仅仅是堆空间的一半，因此，当堆很大的时候，被浪费的空间也就随之变得很大了。另外，由于节点复制算法不能够区别对象的年龄，那些长期存活的对象会被重复地拷贝。

分代垃圾收集能够通过将长期存活的对象提升到年老的分代中而缓解节点复制垃圾收集所遇到的问题。与前两种收集算法相比，它具有非常明显的优势。理论上来说，如果弱分代假设成立，次级收集的效率将会非常高，然而实际情况并非一定如此。占位垃圾的产生会使次级收集错误地将一些已死的对象看成是存活的对象，这种现象将会降低弱分代假设的效果。这一方面使得次级收集要多做一些没有必要的拷贝工作，从而降低了次级收集的效率，另一方面也使得年老的分代会更快地被填满，从而导致开销很大的主收集的发起。因此，简单的分代垃圾收集并不能获得最理想的性能。

5.4 垃圾收集系统性能的评价标准

评价一个垃圾收集系统的好坏，主要可以通过以下 3 个标准来衡量：

1．垃圾收集的效率；

2．垃圾收集的内存布局所带来的局部性；

3．垃圾收集所导致的延迟，即暂停时间；

在一个服务器系统中，考虑垃圾收集的内存布局所产生的局部性是非常困难的。首先，服务器系统必须提供一个长时间的持续的运行环境，一个服务器的应用可能需要连续运行几天、几个月甚至几年的时间，而且这个环境也是在不断地动态地改变的。因此，采用离线的 profile 机制不太可能准确地得到应用的局部性信息。对于服务器的系统来说，动态地发现应用程序的局部性的规律似乎是一种更加可行的方法。然而，动态地分析局部性的技术通常都比较复杂，而且会带来比较大的时间开销。

另外，服务器应用的行为往往不具备太好的局部性。考虑一个简单的事务处理型的数据服务器模型，服务器维护一个大型的数据仓库，并且不断地响应由客户端应用发来的事务处理请求。这些事务处理主要包括对数据仓库的查询、计算及修改等操作。客户端应用的行为通常具有一定的随机性，他们想要查询或处理的信息都是不确定的，这意味着服务器应用对数据仓库的访问方式可能是毫无规律的。而对于表示事务处理请求的对象来说，它们的寿命一般都非常短，因此对这些对象的访问通常没有太多的可重复性，局部性也较差。从这种角度上来说，应用本身具有较差的局部性行为，从而降低了内存布局对局部性的影响。另一方面，由于服务器应用对内存系统的压力比较大，因此会招致更频繁的垃圾收集。在这种情况下，垃圾收集自身的开销成为影响应用性能的一个主要因素，而不同收集算法所导致的对象局部性的差距对系统的影响就显得没有那么重要了。基于以上这些原因，本章的研究工作在一定程度上忽略了不同垃圾收集算法对应用局部性的影响。

在服务器类型的应用中，响应时间是衡量该系统好坏的另一个重要的指标。垃圾收集所产生的暂停时间在很大程度上影响着应用的响应时间。然而这个问题可以通过渐进和并发的垃圾收集技术来予以解决。这些技术与本章所讨论的技术是正交的，因此，本章并不讨论如何减小垃圾收集暂停时间的技术，而主要关注垃圾收集系统对服务器应用的整体性能的影响。

下面，我们来详细讨论一下有关垃圾收集效率的问题。

定义 1：单次垃圾收集效率是指该次垃圾收集中单位时间内所回收的有效内存空间，有效内存空间是指收集结束之后，能够被用户程序所分配并使用的内存空间。即：

$$E_{gc} = S_e / T_{gc}$$

其中 S_e 是该次垃圾收集所回收的有效内存空间，T_{gc} 是该次垃圾收集的时间。

定义 2：一段运行时间之内的平均垃圾收集效率是该段运行时间内垃圾收集所回收的总的有效内存空间除以该段运行时间内总的垃圾收集时间，即：

$$E_{gc_avg} = S_{e_total} / T_{gc_total}$$

其中：如果总共发生了 n 次垃圾收集，

$$S_{e_total} = S_{e_1} + S_{e_2} + ... + S_{e_n}$$

$$T_{gc_total} = T_{gc_1} + T_{gc_2} + ... + T_{gc_n}$$

在一个垃圾收集系统中，假设运行的时间无限长，则 mutator 所消耗的总的内存数量和垃圾收集所回收的总的内存数量可以近似地认为是相等的。于是我们可以得到：

$$E_{gc_avg} \times T_{gc_total} = V_{mutator} \times T_{mutator} \Rightarrow$$
$$E_{gc_avg} / V_{mutator} = T_{mutator} / T_{gc_total}$$

其中：$V_{mutator}$ 是 mutator 的平均消耗内存的速度，$T_{mutator}$ 是 mutator 的总的运行时间。如果不考虑垃圾收集对应用的局部性行为的影响，对于固定的应用来说，$V_{mutator}$ 和 $T_{mutator}$ 是两个不变量，因此，垃圾收集的平均效率决定了垃圾收集的总时间开销。平均效率越高则垃圾收集的时间开销越小。

由此可见，在理想的情况下，垃圾收集的平均效率是衡量垃圾收集算法性能的一个非常好的指标。

5.5 服务器类型基准测试程序——SPECjbb2005 的行为特点分析

在服务器类型的应用中，对象的年龄特点往往并不是十分地符合弱分代假设。尽管存在大量的对象会在非常年轻的时候死亡，但是仍然会存在不少的对象，它们的生命周期会非常长，甚至永远不会死去（immortal）。服务器端的应用通常需要响应大量的来自客户端应用的事务处理请求，与这些事务处理请求所对应的对象通常具有非常短的生命周期，它们在请求发生的时候被创建，在请求得到响应之后便消亡。

然而，由于服务器端的应用必需具有提供持久服务的能力，因此，服务器系统中通常存在许多具有长生命周期的对象。例如数据仓库所对应的对象，它们主要是为了给客户端应用提供数据服务；以及许多中间件（middleware）所对应的对象，这些对象主要是为客户端应用提供某种能力服务，包括某种计算任务或功能性任务等等；另外，还有一些表示系统类的对象，它们将在服务器启动以后被装载并且会存活相当长的时间[IBMb]。

由于这些长生命周期的对象的存在，经典的非分代的垃圾收集算法将会面临严重的问题。这些长生命周期的对象将会被反复地标记或复制，从而极大地影响了垃圾收集系统的性能。尽管分代的垃圾收集算法能够在一定程度上缓解这一问题，但是这些大量的长期存活的对象会在年老的分代中堆积。这使得年老的分代会更快地被填满，从而会更

加频繁地引发对全堆的主收集。

SPECjbb2005（Java Server Benchmark）[SPECjbb05]是由美国标准性能评估公司开发的一种用于评估服务器端 Java 性能的 SPEC 基准测试程序。同上一代的 SPECjbb2000 一样，SPECjbb2005 能够通过模拟一个基于三层模型的客户机/服务器系统来评估服务器端 Java 的性能。该基准测试程序能够覆盖到 Java 虚拟机，即时编译器，垃圾收集，线程模型及操作系统的某些方面的实现。SPECjbb2005 提供了一个新的增强的工作集负载。该工作负载的实现采用了更加面向对象的风格，从而更能够反映真实世界中的应用的设计方式。同时它还引入了一些新的特性，例如 XML 处理和 BigDecimal 计算，因此，该基准测试程序能够更加真实地反映出当今的各类应用。

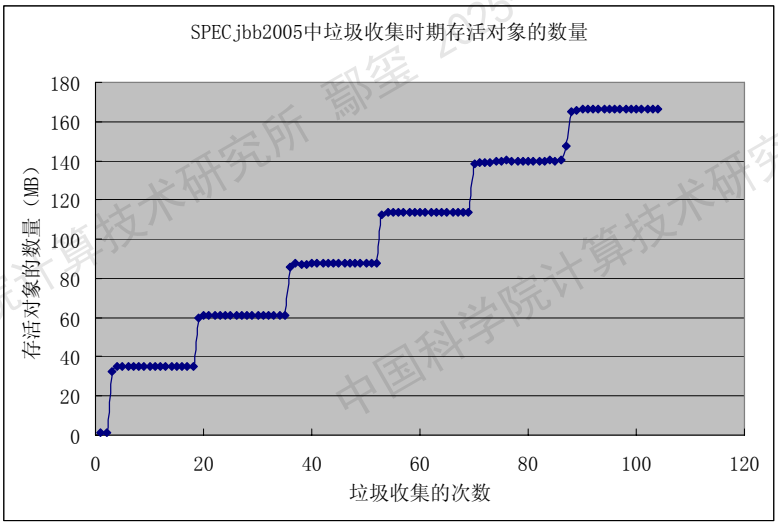

图 5.5 SPECjbb2005 中垃圾收集时存活对象的数量

图 5.5 显示了 SPECjbb2005 在运行过程中，每次垃圾收集时所发现的存活对象的数量。我们使用了标记缩并的收集算法来产生该图中的数据。从图中可以看出，存活对象的数量的增加呈阶梯状，并且呈现出明显的相（phase）。这和 SPECjbb2005 的运行过程是相吻合的。SPECjbb2005 首先装载数据仓库（warehouse）和具有商业逻辑的中间件，然后启动客户端的线程来产生事务处理请求。每次存活对象数量的剧增都发生在装载新的数据仓库的时候。在两次数据仓库的载入点之间的存活对象的数量基本保持不变。这意味着在这个过程中，对象的分配率和死亡率基本相同。

可能存在两种情况能够导致产生相同的对象分配率和死亡率：

1. 堆中的对象有明显的年龄分别，年老的对象持续地保持存活的状态，而年轻的对象则以相同的速度被快速地创建和死亡；
2. 堆中的对象没有明显的年龄分别，所有对象的寿命基本相同，最年老的对象以和最年轻的对象的分配速度相同的速度死亡；

然而，在实验中，我们发现导致 SPECjbb2005 的这种对象生存行为的原因是上述的第 1 种情况。

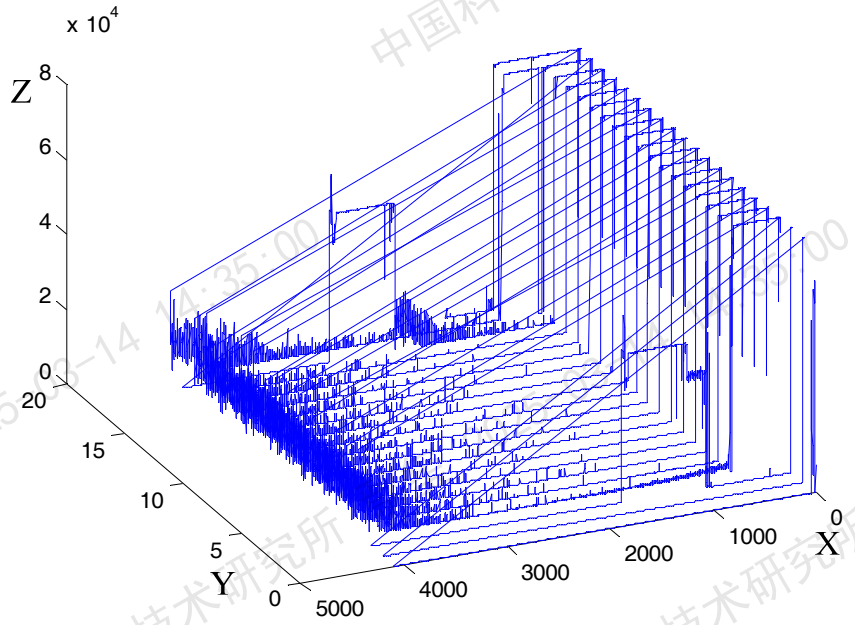

图 5.6 SPECjbb2005 中垃圾收集时存活对象在堆中的分布

图 5.6 显示了 SPECjbb2005 在垃圾收集时存活对象在堆中的分布状态。与图 5.5 一样，该图中的数据也是用对全堆的标记缩并收集算法产生的，所使用的堆的大小为 256MB。在我们的垃圾收集系统中，堆采用分块的组织形式，其中每个块的大小为 64KB。图 5.6 中 X 轴是按照地址顺序排放的块的编号，块的编号越小，则该块就处在越低的地址空间；Y 轴是垃圾收集的次数；Z 轴则表示的是存活对象的大小，单位是字节。图中(x, y, z)点表示在第 y 次垃圾收集的时候，堆中第 x 个块中的存活对象的大小为 z 字节。由于整个运行过程中产生的数据量巨大，因此，我们只显示了前 20 次垃圾收集所产生的数据。在这段过程中，SPECjbb2005 只装载了一个 warehouse。其它运行阶段的行为与该阶段类似，因此不会影响对程序行为的分析。

由于图 5.6 中的存活对象的分布数据是在标记阶段后，缩并阶段前产生的，因此，从该图中可以看出，在程序的运行过程中，存在着大量的长时间存活的对象被堆积在堆的底部。而在堆的顶部，一部分最年轻的对象仍然保持着存活的状态，而且越靠近顶部存活对象的数量越多，这种现象是和我们的直觉相吻合的。在堆的中间部分，很少的存活对象被稀疏地分布在一些块中，而其余的对象则在经历了相对较长的一段时间后死亡。

该图中的结果数据验证了我们的假想。那些被堆积在堆的底部的长生命周期的对象就是与数据仓库和中间件逻辑相对应的对象，而那些用来表示事务处理请求的对象的生命周期则非常短，并且它们的死亡率和分配率基本相同。

5.6 基于长生命周期对象的混合垃圾收集算法

5.6.1 算法概述

针对服务器应用的特点，我们设计并实现了一种新型的混合垃圾收集算法。在一个灵活的软件框架中，我们结合了标记缩并、基于半区的节点复制和分代的垃圾收集算法的特点，在应用运行的过程中，通过对这些基本算法的动态切换，来获得更高的性能。图 5.7 示意了该算法及其堆空间的组织结构。

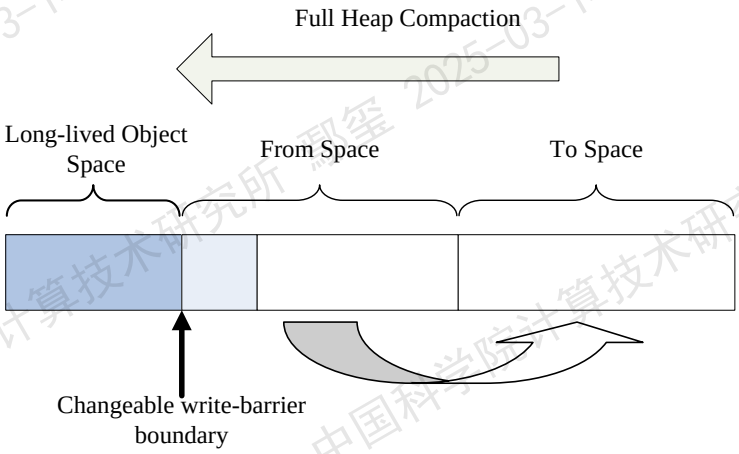

图 5.7 基于长生命周期的混合垃圾收集器

该混合垃圾收集器的主要目标是识别长生命周期的对象，并将其与短生命周期的对象隔离开来，从而避免对长生命周期对象的重复操作，因此我们的算法被称为 LLH 算法（Long-Lived-object-based Hybrid Garbage Collection）。对于短生命周期的对象区域，我们灵活的软件框架使得对该区域的管理可以采用任何的收集算法，在我们初始的实现中，我们采用了一个基于半区的节点复制算法——LLH-SS。

在 LLH 算法中，堆被分为两个主要的区域：LLOS（Long-Lived Object Space，长生命周期对象空间）和 SLOS（Short-Lived Object Space，短生命周期对象空间）。开始的时候，LLOS 并不存在，整个堆空间都由 SLOS 构成。在运行的过程中，算法使用节点复制的模式进行工作，并预测长生命周期对象出现的时机。当 LLH 收集器预测到有大量长生命周期的对象出现的时候，它会动态地切换到标记缩并的模式，从而将堆中的存活对象缩并到堆的最低端。这些位于堆的最低端的对象所占用的空间就形成了 LLOS，而高于 LLOS 的空间就成为新的 SLOS。为了在之后的垃圾收集中不用去重复地操作 LLOS 中的对象，LLH 收集器必须在 LLOS 和 SLOS 之间建立写拦截的机制，并将从 LLOS 中的对象指向 SLOS 中对象的指针记录在记忆集中。接下来，LLH 收集器再次切换回节点复制的算法以管理新的 SLOS，直到下一次预测到有大量新的长生命周期对象出现的时侯，再切换回标记缩并算法来重新整理 LLOS 和 SLOS。

通过这样的设计，我们可以得到以下几个好处：

1. 对于服务器类型的应用程序来说，长生命周期的对象通常是在程序的相（phase）边界成簇地出现，因此，开销较大的标记缩并并不会在事务处理密集的程序相中发生，从而避免了对客户端的事务处理请求的干扰。如果程序相的转变并不频繁的话，LLH-SS 算法的时间复杂度近似地和存活对象的数量呈比例。这一点，也是 LLH 算法与分代垃圾收集算法相比所具有的一个明显优势：在事务处理密集的程序相中，分代垃圾收集将不可避免地引起开销较大的全堆的主收集。
2. 虽然我们的实现中也采用了分代的机制，但是我们的设计不会产生大量的占位垃圾。因为，写拦截建立在 LLOS 对象和 SLOS 对象之间，而 LLOS 中的长生命周期对象几乎一直存活在堆中，因此它们所指向的对象本身就应该是存活的对象，从而不会产生占位垃圾。

5.6.2 LLOS 中短生命周期对象的清除及对长生命周期对象的预测

LLH 算法试图通过全堆的缩并将长生命周期的对象堆积在堆的底部，然而，由于并没有严格地区分对象的年龄，因此仍然会有少量的短生命周期的对象会被同时缩并到 LLOS 中。如果在做完缩并后立刻切换到使用记忆集的节点复制算法，则那些被缩并到 LLOS 中的短生命周期对象将会成为占位垃圾。这些占位垃圾是非常可怕的，在我们的实验中，它们会让 SLOS 中的存活对象越来越多，从而很快就将 SLOS 中的自由空间耗尽。

我们采用了一种动态机制来解决这个问题。以某一次对 SLOS 的垃圾收集为初始状态，假设初始垃圾收集结束的时候，堆中存活对象的大小为 $live_{base}$；相邻的两次垃圾收集中存活对象大小的差值为$\triangle live$；某次垃圾收集中存活对象的大小与 $live_{base}$ 的差值为$\triangle live_{with_base}$，则当如下某种情况发生的时候，我们猜测 LLOS 中可能出现了占位垃圾：

1. $0 < T_{\triangle} \leqslant \triangle live$，$T_{\triangle}$为某一个阈值；
2. $0 < \triangle live < T_{\triangle}$，并且这种现象持续了 n 次垃圾收集；
3. $0 < T_{\triangle} \leqslant \triangle live_{with_base}$；

如果以上某种情况发生，LLH 收集算法将改变工作的模式。对 SLOS 的收集仍然是使用节点复制的算法，不同的是，该模式并不通过记忆集来遍历 SLOS 中的对象，而是通过根集遍历整个堆中的活对象，对于 LLOS 中的活对象，则仅仅使用标记，而不移动对象的位置。利用这种不使用记忆集的节点复制算法，我们能够争取一定的时间，让 LLOS 中的短生命周期的对象死去。与使用记忆集的模式相比，这种模式的时间开销要大一些，但是由于仅仅是标记而不移动 LLOS 中的对象，所以仍然比缩并的时间开销要小得多。另外，由于 LLOS 中出现占位垃圾的频率非常低，而且，通常情况下，在经过一次不使用记忆集的节点复制模式之后，绝大部分的占位垃圾就已经被消除了，因此，这种模式只是偶尔才需要被使用。

接下来，我们再来讨论一下发生一次不使用记忆集的节点复制之后可能出现的情况：

1. 如果在发生该模式的收集之后，存活对象的大小没有得到控制，即存活对象的大小持续增长，则这种增长并不是由占位垃圾引起，而很可能是由于程序正运行在准备数据的阶段，因此产生了大量的长生命周期的对象。对于这种情况，我们的策略是一直保持该模式的收集（遍历整个堆中的对象）直到存活对象的大小不再增长，也就是趋于一个稳定的状态，然后再切换到缩并的模式将这些新增的存活对象缩并到堆的底部。
2. 如果存活对象的大小得到了控制并且有了较明显的下降，则我们认为之前存活对象的增加是由于占位垃圾引起的，并且该问题已经被消除，因此就切换到使用记忆集的节点复制模式继续运行。
3. 如果存活对象的大小虽然得到了控制，但是没有明显的下降，则之前存活对象的增加可能也不是由占位垃圾引起。但是由于情况没有恶化，之前增加的存活对象还并不值得引起一次标记缩并，则我们的策略引入一种累计的机制来检测是否这种缓慢的间断性的增加会持续的发生并产生不能忍受的结果。具体的做法是累计这种存活对象大小的增量，并将其控制在一个给定的阈值之内。如果累计的增量超过阈值，则 LLH 算法切换到标记缩并的模式，如果没有，则仍然恢复到使用记忆集的节点复制模式。

在 LLH 算法的切换机制中，有几个地方需要更新 $live_{base}$：

1. 发生过一次不使用记忆集的节点复制之后，$live_{base}$ 被更新成当前 SLOS 中的存活对象的大小；
2. 发生过一次全堆的标记缩并之后，$live_{base}$ 被更新成当前 SLOS 中的存活对象的大小；
3. 如果在某次垃圾收集之后，存活对象的大小低于 $live_{base}$，则 $live_{base}$ 被更新成当前的存活对象的大小；

Long-lived 的对象尽管生命周期很长，但还是有可能会成簇地死亡。为了能够发现这种情况，LLH 算法需要以一个较低的频率以一定的时间间隔去引发不使用记忆集的节点复制收集。如果与上一次遍历全堆的收集（包括不使用记忆集的节点复制和全堆的标记缩并）相比，LLOS 中的存活对象的大小减少了很多，则该算法通过对 LLOS 的缩并来释放出这些可用的自由空间。

当然，控制提升的阈值也是一种区分长生命周期对象和短生命周期对象的方法。然而，我们的设计中并没有采用这种方法，因为在长生命周期对象和短生命周期对象之间，没有一种可以严格区分的明确的界限，即我们并不知道具有多长年龄的对象就是长生命周期的对象，而具有多短的对象就是短生命周期的对象。因此，即使使用控制提升阈值的方法，也只能是一种预测机制。这种预测势必会导致错误地将一些短生命周期的对象放入长生命周期对象的存储区域，所以也就不可避免地需要用到我们所提出的清除 LLOS 中占位垃圾的机制。而我们的设计直接针对长生命周期对象所带来的问题，即堆的占用率的增加，因此更加简单而有效。

5.6.3 实验结果分析

我们的 LLH-SS 算法实现在 Harmony 的虚拟机平台上，Harmony 平台是 Apache 软件基金会的一个开源 Java 虚拟机项目。我们使用的系统平台是 Intel PentiumD 3.0GHz 双核处理器，Fedora Core 4 发行版，其 Linux 内核版本为 2.6.11。由于我们的研究目标是优化服务器端的垃圾收集系统的性能，所以我们选择了 SPECjbb2005 作为我们的基准测试程序，其分值是用 Troughput 来衡量的，即每秒钟完成的事物处理的操作数，bops（business operations per second）。Throughput 值越大表示性能越好。

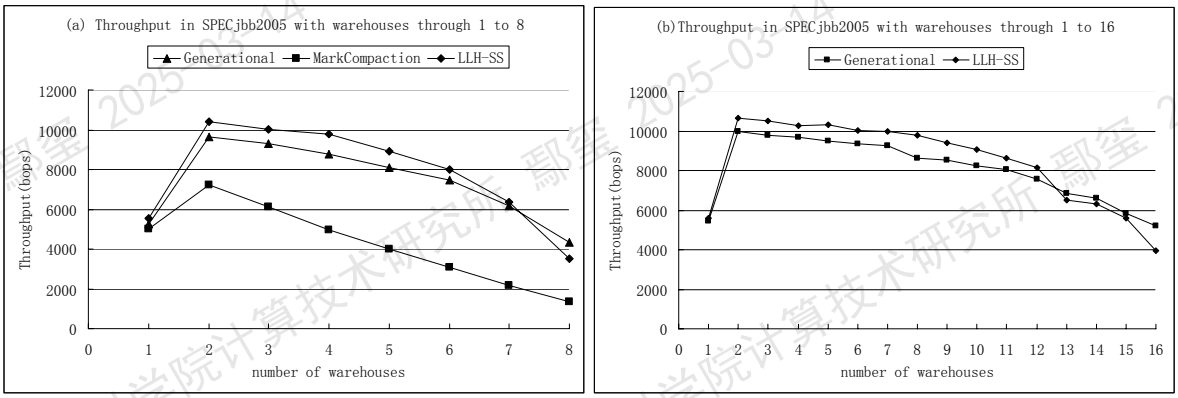

图 5.8 SPECjbb2005 的性能比较

图 5.8 显示了 LLH-SS 算法在运行 SPECjbb2005 时所获得的分数。其中左图比较了分代垃圾收集[1]（Generational）、标记缩并（MarkCompaction）和 LLH-SS 算法。运行时所使用的堆的大小为 256MB，装载的 warehouse 的个数是从 1 到 8，分代垃圾收集的 nursery 大小是 16MB。从图中可以看出，标记缩并垃圾收集的性能远远低于分代垃圾收集和 LLH-SS 算法。这主要是由于巨大的堆空间和大量的长期存活的对象的存在使得全堆的标记缩并开销非常大，因此，严重地影响了收集算法的效率，从而导致了极低的性能。当 warehouse 的个数少于 6 个的时候，我们可以看到标记缩并算法和另外两种算法的性能差距随着 warehouse 的增多而不断增加，这意味着存活对象的增加使得标记缩并收集的相对性能更差。当 warehouse 继续增加的时候，分代垃圾收集与标记缩并收集的性能差距有所减少，这主要是因为，当堆的占用率很大的时候，年老的分代的自由空间也非常少了，因此，年老的分代会很快被填满，从而导致了频繁的主收集的发生。

右图显示了在堆为 512MB，warehouse 从 1 到 16 的性能分数。由于标记缩并与另两种算法有明显的差距，因此右图中就不再比较标记缩并算法了。从左图和右图中我们都可以看到，当 warehouse 的数量较小的时候，LLH-SS 算法与分代垃圾收集相比具有较为明显的优势。这主要是由于以下几个原因。当 warehouse 的数量较小的时候，堆中存活对象的大小也较小，即堆的占用率小。由于 LLH-SS 可以有效地将长生命周期的对象和短生命周期的对象分隔开，因而在大部分的收集时间中不需要重复地处理长生命周期的

[1]在分代垃圾收集的实现中，对全堆的主收集使用的是标记缩并的算法。

对象。同时，由于 SLOS 大小相对于 SLOS 中真正的存活对象来说是非常大的，并且，LLH-SS 中的动态机制可以有效地避免 LLOS 中的占位垃圾的出现，因此，LLH-SS 的收集效率非常高。而对分代垃圾收集来说，由于占位垃圾的出现，使得分代垃圾收集的次级收集无法足够高效地运行，而且，分代垃圾收集还会受到主收集的影响，而 LLH-SS 算法则能够在事务处理密集的程序相中，避免对全堆的标记缩并。然而，当 warehouse 的数量很多的时候，LLH-SS 算法和分代垃圾收集的性能曲线开始出现了交叉。随着 warehouse 的增加，SLOS 的空间相对于 LLOS 越来越小，因此对 SLOS 空间的收集效率也越来越低。同时，这种性能的降低并不是线性的，而是下降的越来越快。这主要是由于随着 SLOS 空间的减小，SLOS 中的短生命周期的对象也来不及死亡，从而导致 SLOS 中每次收集时幸存下来的活对象也有所增加。我们知道垃圾收集的效率是和回收的空间成正比，和收集的时间成反比，而对于对 SLOS 空间的收集来说，也就是和幸存的活对象的大小成反比。因此，LLH-SS 的性能曲线并非线性的。

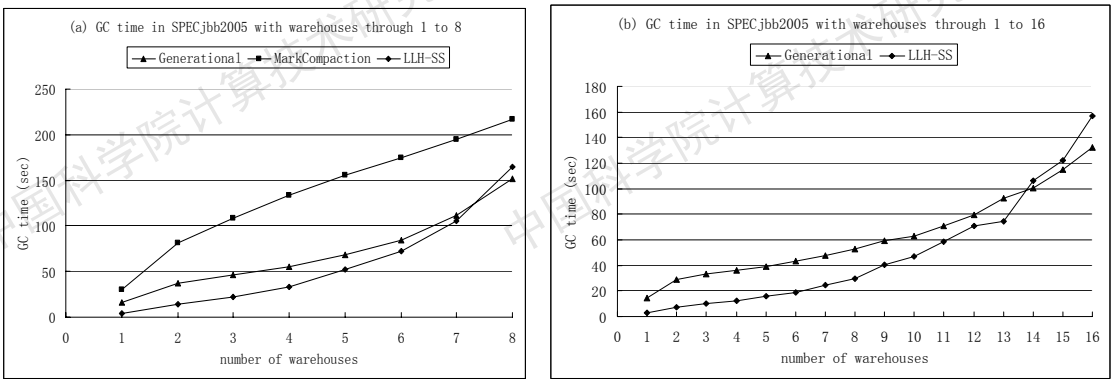

图 5.9 SPECjbb2005 中垃圾收集的时间开销

图 5.9 显示了 SPECjbb2005 运行过程中垃圾收集的时间开销。从图中可以看出，垃圾收集的时间曲线和 SPECjbb2005 的 Throughput 的性能曲线十分的吻合。图中的数据说明，垃圾收集确实在很大程度上影响了 SPECjbb2005 的性能，并且验证了我们之前的讨论中所进行的分析。

前面的讨论中，我们知道分代垃圾收集中可能存在的问题主要有两个方面：占位垃圾和主收集。下面我们再来评估一下占位垃圾所造成的影响。

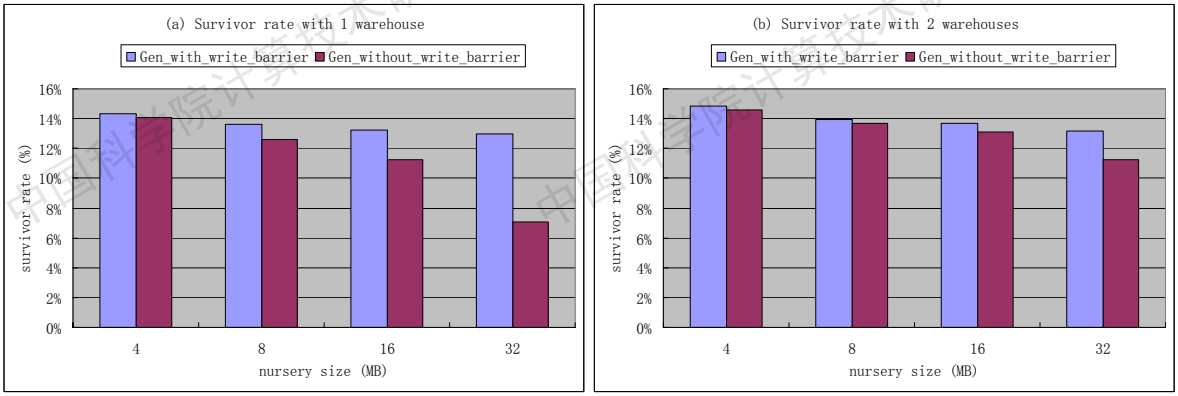

图 5.10 SPECjbb2005 Nursery 中对象存活率的比较

图 5.10 比较了分代垃圾收集的次级收集中对象的存活率[1]。Gen_with_write_barrier 就是普通的分代垃圾收集，它使用了记忆集来遍历 nursery 中的存活对象。Gen_without_write_barrier 是分代垃圾收集的一个变种。它在遍历 nursery 中存活对象的时候，并不使用记忆集中的内容，而是从根集出发，遍历整个堆中的对象，对于年老分代中的对象，则仅仅进行标记。这样，在 Gen_without_write_barrier 的收集中将不会受到占位垃圾的影响。图 5.10(a)示意了在只运行了 1 个 warehouse 的时候，两种收集的 nursery 中的对象存活率。可以看到，使用记忆集的分代垃圾收集由于受到了占位垃圾的影响，其对象存活率会偏高，而且，随着 nursery 大小的逐渐增加，这种差距越来越明显，即占位垃圾的影响越来越大。对于普通的分代垃圾收集来说，nursery 中的对象存活率基本不会随着 nursery 大小的变化而变化，这意味着，增加 nursery 的大小对于减小占位垃圾的影响没有太大好处。

图 5.10(b)显示了在运行了 2 个 warehouse 的时候，两种收集的 nursery 中的对象存活率。可以看到，尽管普通的分代垃圾收集仍然受到占位垃圾的影响，但是真正存活的对象也有所增加，从而在一定程度上减小了占位垃圾所带来的影响。因此，当 warehouse 增多的时候，影响分代垃圾收集效率的主要因素还是主收集所带来的开销。

5.7 LLH 算法的改进

在上一节的分析中，我们可以看出 LLH-SS 算法在大部分情况下由于避免了重复地操作长生命周期的对象而取得了较高的垃圾收集效率，从而使服务器端的应用的性能有所提高。然而，当工作负载增加到一定的程度，SLOS 空间逐渐缩小，而 SLOS 中的存活对象相对增加，收集的效率急剧地下降。由于基于半区的节点复制算法所回收的有效内存空间是半区的大小减去收集中存活对象的大小，因此，该算法更加依赖于 SLOS 空间的大小。

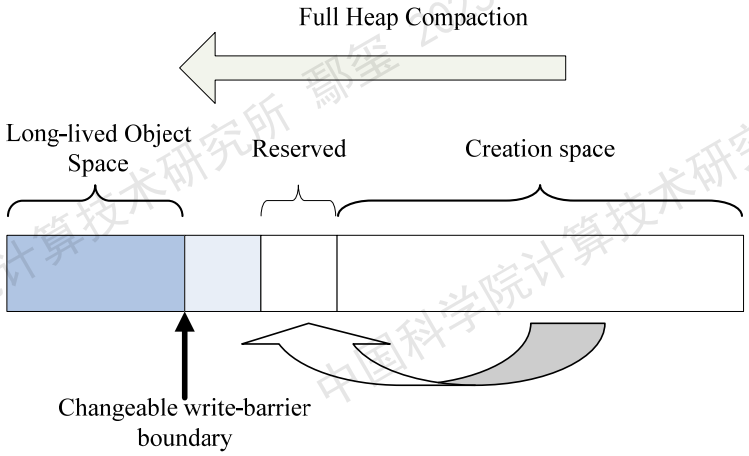

图 5.11 基于 Appel-like 改进的混合垃圾收集

[1] 存活率是指存活对象的大小占 nursery 大小的百分比。

为了更高效地利用 SLOS 空间，我们对 LLH-SS 的算法进行了改进。我们利用了一种类似于 Appel 风格的分代垃圾收集（Appel-like）[Appel89b]的收集算法来提高对 SLOS 空间的收集效率，我们称这种算法为 LLH-Appel-like 算法。算法的结构如图 5.11 所示。

LLH-Appel-like 算法将 SLOS 组织成创建空间、预留空间和堆积空间。初始的时候，整个 SLOS 被分为创建空间和预留空间，当创建空间被分配满的时候，对 SLOS 的收集被发起。创建空间中的存活对象被复制到预留空间中。这些预留空间中的存活对象被缩并地排放，同时构成了堆积空间。然后，预留空间向堆的顶部滑动，并且和堆积空间相连，同时，创建空间被缩小。然后，应用又重新在创建空间中分配对象。这个过程持续进行，直到堆积空间的大小加上预留空间的大小超过了 SLOS 空间的一半，这个时候，收集器将堆积空间中的存活对象复制到创建空间中，然后再整块滑动到 SLOS 的底部，构成新的堆积空间。之所以要在这个时机对堆积空间进行整理，是因为要保证创建空间中能够容纳堆积空间中存活对象的最大可能的大小。

在 LLH-Appel-like 算法中，每次回收的有效空间大小为 SLOS 的大小减去堆积空间和预留空间的大小的总和。大部分情况下，回收的有效空间是大于 SLOS 的一半空间的，因此，收集的效率比 LLH-SS 要高得多。但是在整理堆积空间的时候，开销会稍微大一些，因为存活的对象被移动了两次。与 Appel 风格的分代垃圾收集不同的是，在 LLH-Appel-like 算法中，堆积空间和创建空间之间并没有使用记忆集，因此，堆积空间中的对象在收集中是需要被标记的。

另外，在 LLH-Appel-like 算法中，为了尽可能地提高收集的效率，预留空间应该尽可能地小。但是，如果预留空间不足以容纳创建空间中的存活对象，则必须发生对创建空间的缩并，这时的开销就会比较大。为了避免发生这种缩并，我们将预留空间设计成动态可变的大小，保持预留空间与上一次复制到预留空间中的存活对象大小的比例。在我们的实验中，以这种方式控制预留空间的大小，对 SPECjbb2005 来说，能够有效地避免创建空间中缩并的发生。

图 5.12 比较了 LLH-Appel-like、LLH-SS 和普通的分代垃圾收集算法在运行 SPECjbb2005 时的性能。由图中可以看出，LLH-Appel-like 算法与其它两种算法相比具有更优的性能，而且 LLH-Appel-like 与 LLH-SS 的性能差距也随着 warehouse 的增加而增大。这和我们的直觉是相符合的：当 warehouse 很少的时候，SLOS 空间还比较大，因此，SS 算法的效率和 Appel-like 算法的效率相差不多；而当 warehouse 增多时，SLOS 空间越来越小，SS 算法由于对空间压力更加敏感而导致更差的性能。图 5.13 比较了这几种算法的垃圾收集时间开销，其结果与图 5.12 相吻合，垃圾收集确实在很大程度上影响了 SPECjbb2005 的运行时性能。

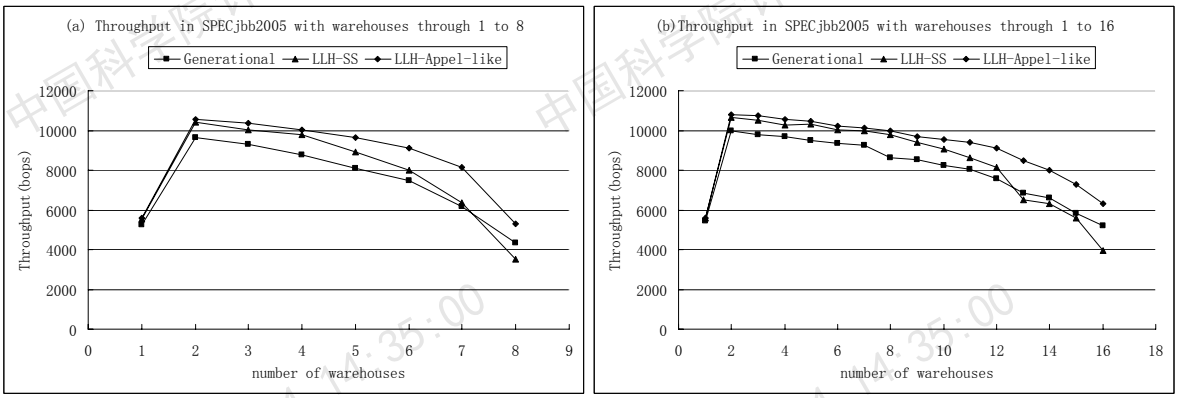

图 5.12 SPECjbb2005 性能比较（with LLH-Appel-like）

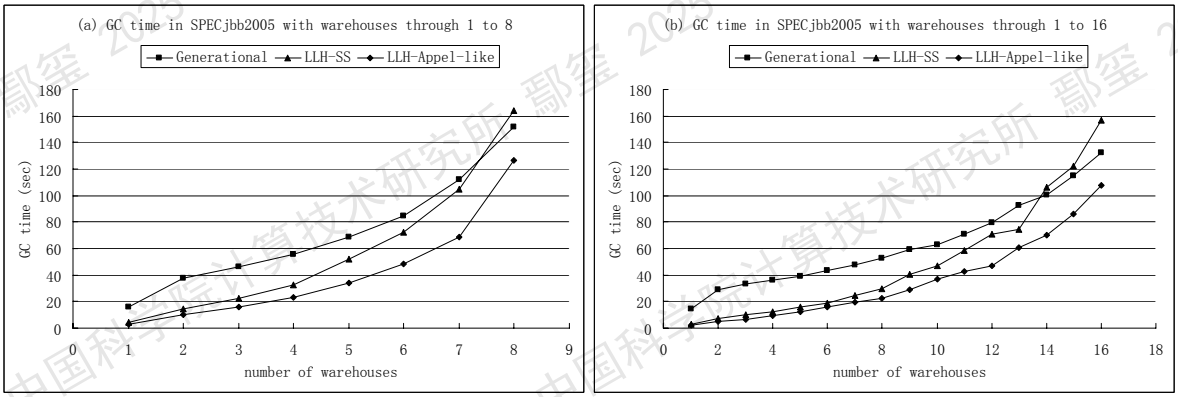

图 5.13 SPECjbb2005 中垃圾收集的时间开销（with LLH-Appel-like）

与普通的分代垃圾收集算法相比，LLH-Appel-like 算法已经在所有已测量的环境中取得了更优的性能。然而，我们依然可以看到，当 warehouse 较大的时候，LLH-Appel-like 收集算法的性能下降的速度是越来越快的。这是因为，SLOS 的占用率的增多使得 LLH-Appel-like 收集算法中对堆积空间的整理变得非常频繁。当 warehouse 的个数为测量时的最大值的时候，几乎每两次 SLOS 的收集中，就有一次需要对堆积空间进行整理。因此，从理论上说，LLH-Appel-like 算法也并不是在任何情况下都能取得最优的性能的。一个更优的组合可能是当 warehouse 很多的时候，将 SLOS 也组织成分代的模式，

5.8 本章小结

本章的工作针对服务器类型应用的特点提出了一种新的基于长生命周期对象的混合垃圾收集算法。该算法组合了若干经典的垃圾收集算法，在一个灵活可配置的垃圾收集软件框架中实现了这些经典算法的灵活的动态切换。本章所提出的算法通过将长生命周期的对象与短生命周期的对象分隔开来，来避免对长生命周期对象的重复操作，从而提高了垃圾收集系统的性能。

本章的工作表明，在一个应用的行为和工作负载都在动态改变的环境中，垃圾收集必需能够根据当前的运行时情况动态的改变自己的配置或者改变所使用的算法，否则静

态的垃圾收集将无法使恶劣的情况得到适当的缓解，从而严重地影响应用的性能。

本章的研究结果表明，LLH-SS 算法和分代垃圾收集算法的性能曲线有一个交叉点。这意味着，即使是一个具有一定动态特性的混合垃圾收集，仍然不能够在所有的情况下取得最优的性能。因此，如果能够在这两种收集算法之间切换，应该能够取得更好的效果。在本章的讨论中，我们可以知道，理想的情况下，垃圾收集的平均效率是一个很好的评价标准。我们可以周期的测量不同收集算法的平均效率来决定选择哪个算法会更好。尽管严格的理想情况是不存在的，我们仍然可以通过降低一些要求来得到近似的理想情况。事实上，当垃圾收集的效率较低，并且不同算法的垃圾收集效率差别又比较明显的时候，忽略对象的不同布局所带来的局部性的影响通常也是合理的。

某些研究者正在试图寻找一些通用的动态切换策略来适应所有的应用环境。然而这种做法要想成功是相当困难的。通过本章的工作，我们更加主张另外一种设计方法。一个灵活可切换的动态框架是必需的，在此基础上，针对一定范围的应用的需求，通过对这类应用的行为的分析，寻找更加适合的动态机制，通常能够取得更好地效果。

中国科学院计算技术研究所 鄢玺 2025-03-14 14:35:00

中国科学院计算技术研究所 鄢玺 2025-03-14 14:35:00

第六章 基于 Task-pushing 的并行 GC 标记算法

6.1 引言

随着共享内存的多核和多处理器系统被越来越广泛地部署，现代运行时系统中并行和并发的垃圾收集技术正在变得越来越重要。一个被良好开发的并行垃圾收集器能够有效地减少垃圾收集所带来的暂停时间，改进运行时系统的可缩放性，从而获得更好的性能。自从 Halstead 设计了第一个并行垃圾收集[Halstead85]之后，有大量的算法被提出以并行化垃圾收集的过程[IT93, Endo97, Flood01, Cheng01, Attanasio01, Ossia02, KP06]。现今，大部分商用的 Java 虚拟机系统中都存在着一个或者多个并行的垃圾收集算法的实现。

并行垃圾收集的一个主要任务就是为可用的垃圾收集线程合理的划分收集的工作。在一个“stop-the-world”的垃圾收集中，收集过程包括以下 3 个主要的阶段：

1. 枚举（enumerating）根引用，构造根集合；
2. 发现存活的对象；
3. 回收不再被使用的垃圾内存；

对于第一个任务的并行化，可以以一种简单的方式来实现：当可用内存资源耗尽的时候，每个用户线程在垃圾收集线程开始工作之前，都独立地去枚举并构造自己局部的根引用集合。直到所有的根引用集合都被构造完毕之后，所有的用户线程才停止下来，垃圾收集线程才真正开始工作。

对于一个拷贝垃圾收集来说，任务 2 和任务 3 通常是一起完成的。拷贝收集在遍历对象的同时就将对象复制到目标空间中，并且将所有的引用都更新到了对象所在的新的位置。于是，当遍历完成之后，旧的空间就可以整体地被回收了。由于在拷贝垃圾收集中存在对对象的局部性进行优化的良机，因此，一些研究者也将拷贝收集的并行化和对象的拷贝序结合起来进行考虑[IT93, Siegwart06]。

在一个非拷贝的垃圾收集中，比如标记清扫和标记缩并收集，任务 2 和任务 3 通常是在分开的阶段中实现，其中任务 2 就是标记阶段，而任务 3 则是清扫或缩并阶段。一旦标记阶段通过 mark-bit-table 这样的辅助数据结构或者对象头中的元数据结构识别出堆中所有的存活对象之后，任务 3 的并行化相对来说就比较容易了。例如，清扫过程可以通过按照垃圾收集线程把堆划分为若干个区域来实现其并行化。因为每个收集线程可以将局部区域的垃圾回收到局部的自由链表中，好的负载平衡和低的同步开销的获得将是非常直接的[Attanasio01]。

对于缩并过程来说，一旦要被移动的对象的目标地址能够被计算出来，缩并过程便可以很好地被并行化。与清扫过程不同的是，一些高效的并行缩并算法是按照目标空间

的区域来划分任务负载的。并行缩并需要解决的一个比较困难的问题是在并行缩并的过程中保持对象的相对顺序并且尽可能地减少遍历堆的次数[Abuaiadh04, KP06]。

当堆空间很大并且存活的对象很多的时候，对于一个非拷贝的 stop-the-world 的垃圾收集来说，标记过程可能成为垃圾收集所带来的暂停时间的主要贡献者。这种情况在服务器端的应用中经常会出现。在多处理器的系统中并行化标记的过程能够缓解这个问题。然而，并行化的效率是影响服务器应用最终所能获得的性能的关键因素。

串行的标记过程通常需要使用一个被称为 mark-stack 的辅助的栈结构。在扫描一个对象的引用域的过程中，如果发现某个引用域所引用的对象还没有被标记过，则对该对象的引用就被放入 mark-stack 中。从三色抽象的概念来理解，mark-stack 中实际上就存放了指向灰色对象的引用。

整个标记过程可以被看作是对 mark-stack 中的元素操作的叠代（iteration）。在每一次叠代中，垃圾收集线程将栈顶元素弹出 mark-stack，并扫描该元素所引用的对象，如果该对象中的某个引用域所引用的对象还未被标记，则标记该引用域所引用的对象，并将指向这个被标记的对象的引用再压入 mark-stack 中。由于标记过程的这种完美的叠代属性，人们可能会从直觉上认为对标记过程的并行化方法是很简单的。一个很直接的并行化方法是让所有的垃圾收集线程使用一个全局共享的 mark-stack。但是这种共享的 mark-stack 需要密集的同步访问，这通常会在处理器个数较多的时候带来很大的开销并且导致很差的可缩放性。

一些改进已经被提出以减少同步的开销同时获得较好负载平衡[Endo97, Flood01, Cheng01, Ossia02]。这些工作主要是想努力地找到一种灵活的任务划分机制，同时尽可能地减少对同步操作的使用，比如，尽可能地避免对锁的使用，尽量采用原子指令来完成必需的同步操作。然而，由于动态的负载平衡势必会存在某种形式的共享，因此要想完全地避免同步的操作将是非常困难的。

我们相信，如果同步的操作能够被完全地避免，那么标记过程的并行化将会被进一步改进。在本章的工作中，我们提出了一种新的并行标记技术来达到这个目标，我们称这种技术为“task-pushing”。在任务分配和平衡的方式上，task-pushing 不同于先前的研究。它利用了 CSP（communicating sequential process）的思想来协调不同的垃圾收集线程，因此能够获得 CSP 在同步和负载平衡上所具有的优势。在 task-pushing 算法中，每个垃圾收集线程都会主动地去发现正处在空闲状态的收集线程，然后通过一个通信管道将新的任务传送给空闲的收集线程。在先前的研究中，任务的分配要么是通过访问一个全局共享的任务池（task-pool）要么是通过从其它收集线程的任务链表中偷取未被执行的任务来实现。这些传统的协调线程的方法不能够避免对同步原语的依赖。

本章工作的贡献包括：

1. 我们为非拷贝的垃圾收集设计并实现了一个无需任何同步原语支持的基于 CSP 风格的并行标记算法；
2. 为了使这种 CSP 风格的设计能够高效的实现，我们开发了一个高性能的队列数

据结构。该队列能够允许两个线程的同时访问而无需原子操作的支持；

3. 我们详细地研究了队列和任务分配机制对负载平衡的影响；
4. 我们在真实的硬件平台上对这种并行标记算法进行了评估。而且，我们的评估将标记阶段从垃圾收集的其它阶段中分离出来。这一点很重要，因为使用这样的评估方法，我们能够更清晰地了解到标记阶段的行为特点，而不会受到其它阶段的行为的影响。

我们在 Harmony 的平台上实现了 task-pushing 的算法（Harmony 平台是 Apache 软件基金会的一个开源 Java 虚拟机项目），并在一个 16-way Intel Xeon multiprocessor 硬件平台上对 task-pushing 算法进行了评估。我们也实现了 Flood 的优化的 work-stealing 算法并进行了比较。实验结果表明，对于 pseudojbb 和 GCOld 这两个服务器端的基准测试程序来说，task-pushing 在 16 个处理器上能够获得 8.3 的加速比。而且，与 work-stealing 技术相比，task-pushing 所带来的性能好处会随着处理器个数的增加而变得越来越大。

本章的组织如下：6.2 节介绍了并行标记算法的相关工作；6.3 节介绍了 CSP 的并行编程思想，描述了一种通信队列的实现；6.4 节描述了 task-pushing 算法的设计和工作原理；6.5 节讨论了 task-pushing 算法中与负载平衡相关的几个问题；6.6 节对 task-pushing 算法进行了实验评估；6.7 节对本章的工作进行了总结。

6.2 并行标记算法的相关工作

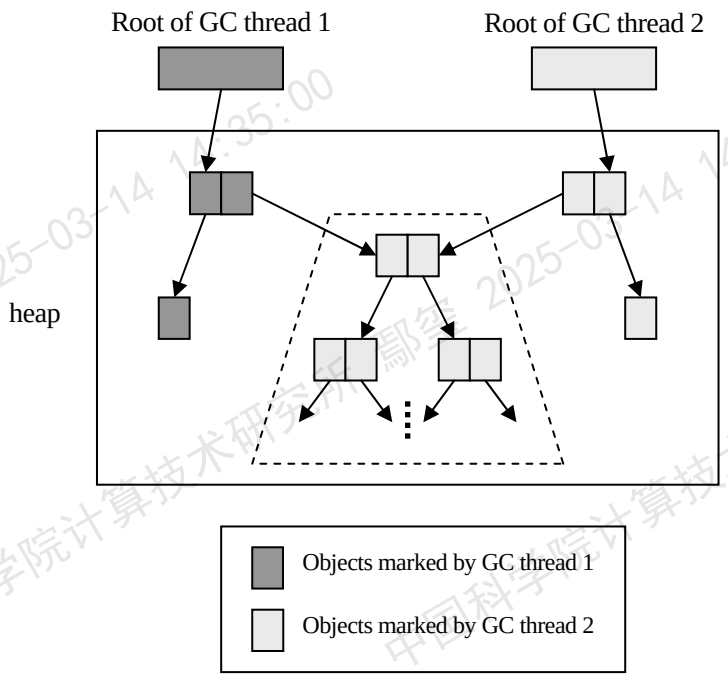

图 6.1 简单的算法所导致的负载不平衡

Endo et al. 设计了一个并行的标记清扫收集算法[Endo97]。在他们的算法中每个垃圾收集线程都拥有一个局部的 mark-stack。初始的时候，根集合以某种方式被划分给每个收集线程，这些线程随后从自己局部的根集合出发，把根集合所指向的对象的引用放

入自己局部的 mark-stack 中。然后，每个收集线程便开始了构成标记算法的叠代过程。

在 Endo 的收集算法中，标记位是使用辅助的 mark-bit-table 来实现的，该数据结构将标记位组织成位图的形式。该位图中每一位都与堆中的一个字相对应，因为对象是字对齐的，因此堆中的每一个字都有可能是某个对象的起始位置。当一个对象被标记的时候，与该对象起始位置相对应的 mark-bit-table 中的那一位将被置成 1，表示该对象已经被标记过。然而，这个操作必需是原子的，因为不同的垃圾收集线程可能会同时将 mark-bit-table 中的某个字的不同位置位。当所有可达的对象都被标记过后，标记过程结束。在 Endo 的算法的初始实现中，他们使用锁的机制来实现标记操作的原子性。

Endo 发现，这种简单的做法会导致严重的负载不平衡。图 6.1 显示了可能导致这种负载平衡问题的一个例子。当一个大的数据结构通过一个根对象而被垃圾收集线程所共享的时候，只有一个垃圾收集线程能够成功地标记该根对象，并且只有该线程能够继续遍历这个数据结构。在我们的实验中，这种现象几乎总是会发生。

为了解决这个负载平衡的问题，Endo 设计了一种基于 work-stealing 的技术。除了局部的 mark-stack，每个垃圾收集线程还需要维护一个被称为 stealable-mark-queue 的附加的数据结构。标记任务[1]通过该数据结构进行交换。图 6.2 示意了该过程。

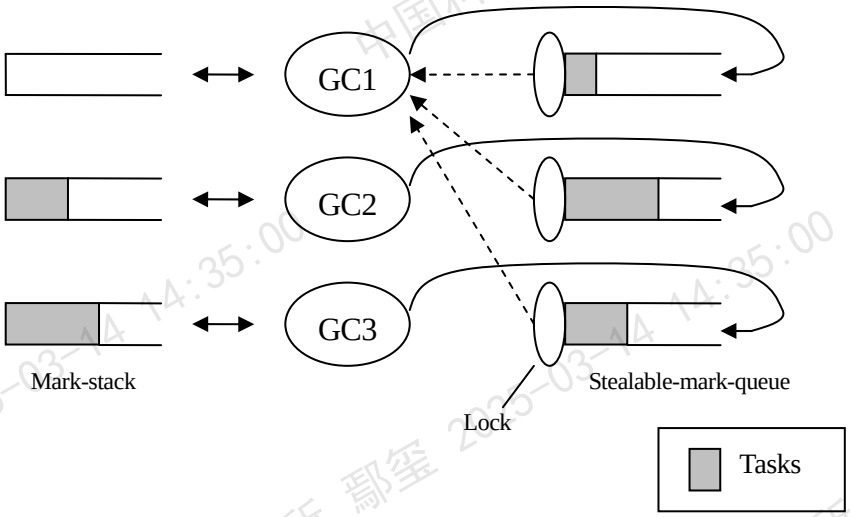

图 6.2 使用 Stealable-mark-queue 实现的动态负载平衡机制

在标记的过程中，每一个收集线程都会周期地检查自己的 stealable-mark-queue。如果该队列为空，收集线程就把自己局部的 mark-stack 中的所有任务都放到该队列中。当某个收集线程空闲了，即该线程的 mark-stack 为空，它就尝试着从 stealable-mark-queue 中获取新的任务。收集线程首先检查自己的 stealable-mark-queue，然后再检查其它线程的，直到它发现一个非空的队列。当它发现一个非空的 stealable-mark-queue，它就把该队列中一半的任务偷取出来并将它们放入自己局部的 mark-stack 中。因为几个收集线程可能同时变为空闲，因此，从 stealable-mark-queue 中偷取任务的操作必需进行同步。Endo

[1]在标记算法的术语中，任务是指 mark-stack 中的数据项，也可以理解成指向对象的引用。

使用了锁的机制来实现这种同步。当所有的 mark-stack 和 stealable-mark-queue 都变为空的时候，标记过程就结束了。

Endo 发现收集线程经常会在获取 stealable-mark-queue 的锁的操作上花去大量的时间。因此他们改进了获取锁的操作序列。他们使用一种 try-lock-then-steal 的序列来替换 lock-then-steal 的序列。当获取锁的操作失败了的时候，收集线程只是简单地放弃对获取该锁的尝试，并立刻去检查下一个 stealable-mark-queue。另外，他们发现使用锁的机制来保证标记操作的原子性会带来较大的开销，于是他们利用原子的 CAS 指令来替换锁的机制。

Flood et al.扩展了 Endo 的并行标记算法[Flood01]。他们依然是使用 work-stealing 的技术来实现动态的负载平衡。与 Endo 的工作不同的是，Flood 使用了一个更高效的 work-stealing-queue 的实现，该实现是基于 Arora et al.提出的一个非阻塞（non-blocking）的双端队列数据结构[Arora98]。每一个垃圾收集线程都拥有自己的一个工作队列（即 work-stealing-queue），该队列有 3 个基本的操作：

- PushBottom 将一个数据元素放入到队列的底部；
- PopBottom 将一个数据元素从队列的底部弹出；
- PopTop 将一个数据元素从队列的顶部弹出；

PushBottom 和 PopBottom 是局部操作，通常不需要同步。PopTop 操作被用来从其它的收集线程中偷取任务。

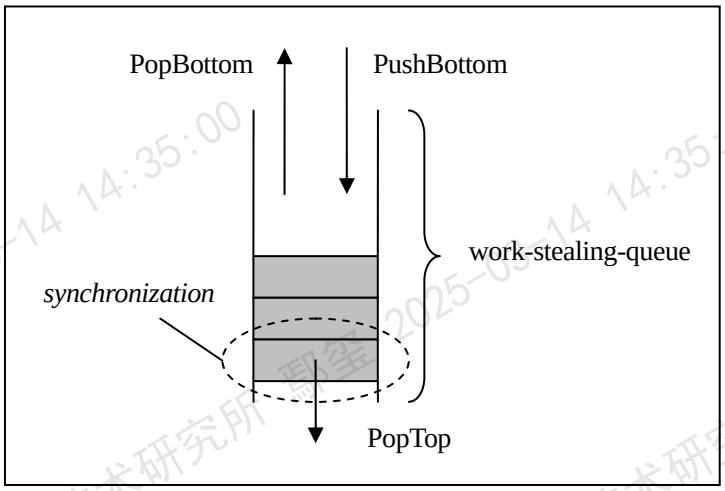

图 6.3 Arora 的 work-stealing-queue

初始的时候，根集合以某种方式被划分给每个垃圾收集线程，然后收集线程通过 PushBottom 操作将自己局部的根集合中的任务放入到自己的工作队列当中。接下来，每个收集线程使用 PopBottom 操作将任务弹出并扫描该任务所引用的对象，然后将产生的新任务通过 PushBottom 操作再放入工作队列中。当一个收集线程不能够从自己的工作队列中获取新任务的时候，它就通过 PopTop 操作从其它收集线程的工作队列中偷取一个新的任务。有两个地方必需使用同步：

1. 当使用 PopTop 操作从其它线程的工作队列中偷取任务的时候；

2. 当 PopBottom 操作在获取工作队列中的最后一个元素的时候；

图 6.3 示意了 work-stealing-queue 及对该队列的三种操作。

尽管 Endo 和 Flood 都通过使用一些非常巧妙设计来减小同步操作的开销，但是他们始终还是不能够避免同步操作的出现，因此，当处理器的个数很多的时候，他们的算法仍然可能由于频繁的同步操作而使可缩放性受到影响。

6.3 CSP 和高性能的通信队列

先前的学者们在并行的GC标记算法的研究中不能够成功地避免对同步原语的依赖，主要是因为它们都使用某种全局共享的数据结构来实现动态的负载平衡。然而，要想使对这种全局共享的数据结构的操作能够保持正确性，同时又不依赖于同步原语的支持，是相当困难的。Arora et al.所提出的 work-stealing-queue 的设计[Arora98]已经是我们所知道的对同步原语依赖最少的数据结构了。

Hoare 提出了一种被称为通信顺序进程（Communicating Sequential Processes, CSP）的概念[Hoare78, Hoare04]，它实际上是一种并行编程范式。在并行编程的领域里，存在着多种机制以解决各种具体的同步问题。然而，Hoare 尝试着寻找一种单一的简单的机制来解决所有并行编程中可能遇到的各种同步问题。

Hoare 认为输入、输出和并发都应该被看作是程序设计中的基本原语，通信顺序进程的并行的组合是一种最基础的程序结构化方法。对于一些持续运行的并且和它们所在的环境不停地进行交互的计算机系统来说，人们通常可以很容易地把它分解为若干的子系统，这些子系统能够并发的运行，并且能够在相互之间以及和环境之间进行交互。在 CSP 的编程模型中，进程之间的共享都是通过进程间的通信来完成的，而通信则是通过某种管道来实现的。

在对 CSP 编程模型的观察和研究中，我们发现，该模型并不依赖于一个全局的共享数据结构，唯一的共享数据结构就是两个进程之间进行通信的管道。因此，我们相信，如果能够找到一种通信管道的高效实现（即无需同步原语的支持），并且把 CSP 风格的编程模型应用在并行的 GC 标记算法的设计中，我们就能够避免并行标记算法中的同步原语，从而获得更高的可缩放性。

6.3.1 高性能的通信队列

在真实的硬件平台中，CSP 风格的计算模型的性能在很大程度上依赖于进程间的通信管道的高效实现。我们开发了一个高性能的队列数据结构来实现进程间的通信管道。由于现今的商用硬件平台并没有为队列的实现提供特殊的硬件支持，因此，我们使用了一种巧妙的设计以使对该队列的操作无需原子指令的支持。

这个通信队列的主要设计思想非常简单。我们使用单读单写（Single-Reader-Single-Writer, SRSW）的队列作为构成队列机制的基本单元，并且利用硬件的 cache 一致性协

议来保证对该队列访问的正确性。多个读者和多个写者的队列可以由单读单写的子队列组合而成。

一个 SRSW 队列只能够由一个读者和一个写者进行访问。一个读者线程能够通过 dequeue 操作将队列中的一个数据元素读取出来，而一个写者线程能够通过 enqueue 操作将一个数据元素写入该队列中。在我们的设计中，SRSW 队列中的所有的数据项都是按字对齐（word-aligned）的，这样，对某个数据项的 load 和 store 操作就可以保证是原子的。现代大多数的多处理器平台都对这种字对齐的内存访问的内在原子性提供支持。由于对象的引用本身就是字对齐的，因此对于并行的 GC 标记算法来说，这种按字对齐的要求并不会带来实际的问题。

```
struct queue {
    int head;
    int tail;
    void* entry[queue_size];
};
```

(a) Data structure of SRSW queue. It has a *head* and a *tail* pointer, and an array of queue entries with size of *queue_size*

Note: entry[queue_size] elements are word-aligned in most existing architectures

```
void* dequeue(queue* que){
    void* data;
    int head;

d1: head = que->head;
d2: data = que->entry[head];
d3: if( data == NULL )
d4:     return NULL;

d5: que->entry[head] = NULL;
d6: que->head = (head+1)% queue_size;
d7: return data;
}
```

(b) Dequeue implementation. Return the entry data if the queue is not empty

```
bool enqueue(queue* que, void* data){
    void* old;
    int tail;

e1: tail = que->tail;
e2: old = que->entry[tail];
e3: if( old != NULL )
e4:     return FALSE;

e5: que->entry[tail] = data;
e6: que->tail = (tail+1) % queue_size;
e7: return TRUE;
}
```

(c) Enqueue implementation. Return TRUE if data is successfully enqueued

图 6.4 SRSW 队列的实现

```
struct queue {
    int head;
    char pad1[LINE_SIZE – INT_SIZE]
    int tail;
    char pad2[LINE_SIZE – INT_SIZE]
    void* entry[queue_size];
};
```

(a) Enforcing cache layout by programmer padding the data structure with extra bytes: *pad1* and *pad2* array. The count of padding bytes is the difference of cache line size (LINE_SIZE) from the size of an integer (INT_SIZE).

```
struct queue {
    int head __attribute((aligned(LINE_SIZE)));
    int tail __attribute((aligned(LINE_SIZE)));
    void* entry[queue_size] __attribute((aligned(LINE_SIZE)));
};
```

(b) Enforcing cache layout by using compiler directives. Here given is an example for GCC compiler. LINE_SIZE is the cache line size in byte.

图 6.5 使用 cache 优化后的 SRSW 队列数据结构

SRSW 队列使用值 NULL（或者是其它的任何值，只要该值不是有效的任务 id[1]就可以）来表示一个空的数据项，任何值不为 NULL 的数据项都应该持有一个有效的任务 id。当读者线程使用 dequeue 操作从队列中取出一个数据元素之后，它会立刻将该数据元素所在的数据项置为 NULL；在写者线程通过 enqueue 操作将一个数据元素写入队列中的

[1] 在并行 GC 标记算法中，任务 id 就是对象的引用。

某个数据项之前，它会检查当前正要被写入的数据项的值是否为 NULL。图 6.4 给出了 SRSW 队列实现的伪代码。

注意到在 SRSW 队列中，head 指针只会被读者线程所访问，而 tail 指针只会被写者线程所访问，因此，dequeue 和 enqueue 的操作可以通过将 head 指针和 tail 指针放在不同的 cache 行来进行改进，从而避免了对 head 指针和 tail 指针的访问所带来的假共享。图 6.5 给出了改进过后的 SRSW 队列的数据结构。

下面，我们给出了 SRSW 队列设计的正确性的证明。

证明：我们使用图 6.4 来描述证明的过程。要证明对 SRSW 队列操作的正确性，我们只需要证明从语义上讲 dequeue 和 enqueue 的操作相互之间是具有原子性的，也就是说代码序列 d1~d7 和代码序列 e1~e7 的工作行为就好像在任何情况下它们都不会交叠一样。我们仅仅需要考虑和共享数据相关的读取（d2 或 e2）、检查（d3 或 e3）和写入（d5 或 e5）操作，因为其它那些对非共享数据的操作与我们想要证明的原子性并不相关。

如果 head 指针和 tail 指针指向队列中不同的数据项，也就是说读者线程和写者线程将会访问不同的数据项，则 dequeue 和 enqueue 操作相互之间显然是原子的。因此，我们只需要考虑 head 指针和 tail 指针指向队列中相同数据项的情况。在这种情况下，我们再来考虑 2 种子情况：

1. dequeue 和 enqueue 操作并没有交叠，则这两种操作显然具有语义上的原子性；
2. dequeue 操作和 enqueue 操作互相交叠，也就是说，d2 发生在 e2 和 e5 之间或者 e2 发生在 d2 和 d5 之间。在这两种指令序列下，读者线程和写者线程将会读取相同数据项的值。因为它们对该数据项的检查（d3 和 e3）使用相反的条件（NULL 或者非 NULL），因此只有一个线程会发现条件为 FALSE，并且继续后面的写入操作（d5 或 e5）。于是，这两个线程的操作序列看起来就好像是没有交叠一样；

证毕■

在 CSP-style 的并行 GC 标记算法中，我们使用 SRSW 队列作为线程之间唯一的通信队列。

6.4 Task-pushing 并行 GC 标记算法

6.4.1 初始的划分区域的设计

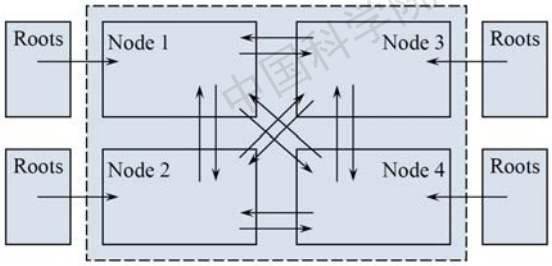

图 6.6 划分为 4 个区域的堆结构

有了 CSP 的并行编程模型和高性能的通信队列，我们尝试将 CSP 的编程思想应用到并行标记算法的实现中来。在初始的设计中，为了达到更好的负载平衡，我们期望将堆按照垃圾收集线程的个数划分成相应的区域，每一个收集线程只需要去遍历和标记属于自己区域的存活对象。这种堆的组织结构如图 6.6 所示。

为了实现这种划分区域的并行标记，一种方法是使用写拦截将所有区域间的引用都记录下来。但是这会导致写拦截的开销非常大，因为区域间的指针可能非常多，特别是当处理器的个数较大的时候情况会更加严重，因此这种方法并不可取。

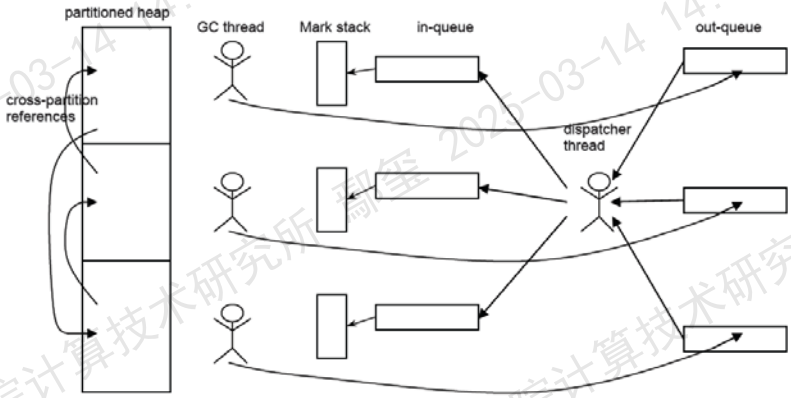

图 6.7 具有 CSP-style 的划分区域的并行标记算法

我们采用了另外一种方法来实现这种划分区域的并行标记。如图 6.7 所示，每个收集线程都有自己局部的 mark-stack。另外，每个收集线程都有一个 in-queue 和 out-queue，它们的实现都是 SRSW 队列。垃圾收集开始的时候，每个收集线程从自己局部的根集合出发去遍历和标记属于自己区域的存活对象。当遍历到不属于自己区域的存活对象的时候就将该对象的引用放入自己的 out-queue 中。另外有一个 dispatcher 线程不停地轮询所有收集线程的 out-queue 的状态。它将 out-queue 中的对象引用取出，并判断该对象属于哪一个线程所管辖的区域，然后将该对象的引用放入相应线程的 in-queue 中。当某个收集线程的 mark-stack 为空的时候，它就会尝试从自己的 in-queue 中取出新的任务来执行，如果 in-queue 中也没有新的任务，就只能够等待。

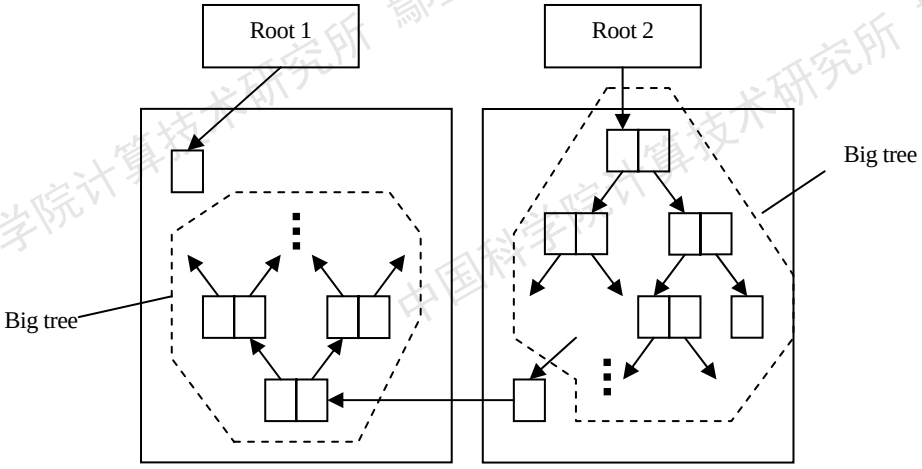

图 6.8 对存活对象的遍历依赖于其它线程

在这个设计中存在几个问题。首先，区域的平均划分并不等价于任务的平均划分。在标记过程开始之前，存活对象在堆中的分布是没有被确定的，我们很难做到对任务的公平划分。如果只是平均地划分区域，则某个区域中的存活对象的密度可能远大于其它区域，因此，就会导致很差的负载平衡。

其次，由于不同垃圾收集线程的处理速度有所不同，而且通信队列的长度都是有限的，因此，无论是 in-queue 还是 out-queue 都有可能被填满。当这种情况发生的时候，某些垃圾收集线程就会被迫等待，从而影响了负载的平衡。

另外，尽管某个收集线程所管辖的区域中存在大量的存活对象，但是，这些对象的根源可能位于其它收集线程的区域中，因此，该线程对这些对象的遍历依赖其它收集线程能够尽快地将这些对象的根引用通过通信队列传递过来，否则，该线程就只能够陷入等待的状态。图 6.8 示意了这种情况。

在实验中，我们发现，这种设计总是会导致某个垃圾收集线程将大量的时间花在对任务的等待上，因此导致了很差的可缩放性。

6.4.2 Task-pushing 算法概述

由于划分区域的并行标记算法在任务的分配上存在一定的问题，因此，我们需要一种更加灵活的任务分配机制，使得在避免同步原语的同时获得最大的负载平衡。Task-pushing 的技术能够使我们达到这个目的，本节描述了该算法的实现。

在 task-pushing 算法中，每个垃圾收集线程拥有自己局部的 mark-stack，它们通过均匀的划分根集合中的引用来获得自己的初始任务。在扫描对象的过程中，当发现一个对未被标记过的对象的引用时，一个新的任务被产生。缺省的情况下，这个新产生的任务总是被放到该收集线程的局部 mark-stack 中。在我们的实现中，标记位被实现在对象头的元数据结构中，并且由于标记操作是等幂（idempotent）的，即一个对象被多次标记和一次标记的效果是一样的，因此，对标记位的操作并不需要同步原语的支持来保证其正确性。这样，我们就有了一个简单的没有任何同步操作的并行标记算法，尽管该算法会导致很差的负载平衡。

并行化的下一步就是要允许不同的垃圾收集线程能够共享遍历的任务。在 task-pushing 算法中，我们使用 CSP 的通信模型来实现这种任务的共享。在基于 CSP 的数据流计算模型中，每一个垃圾收集线程可以被看作是一个顺序进程（sequential process），它们之间的任务共享通过相互的通信来完成。我们使用 SRSW 队列作为收集线程之间的通信队列。每个收集线程可以将自己未能及时处理的任务通过通信队列传送给其它的收集线程。这样，整个标记过程可以被表示成一个处理网络，如图 6.9 所示。

每个垃圾收集线程维护一个队列数组（即元素为队列的数组）。数组中每个队列对应着一个对等的收集线程。这些队列就是垃圾收集线程之间的通信管道。一个队列被标识为一个 2 元组<*i, j*>，表示收集线程 *i* 是该队列的写者，收集线程 *j* 是该队列的读者。队列<*i, j*>是线程 *i* 的输出队列，同时也是线程 *j* 的输入队列。线程 *i* 的所有的输出队列被表

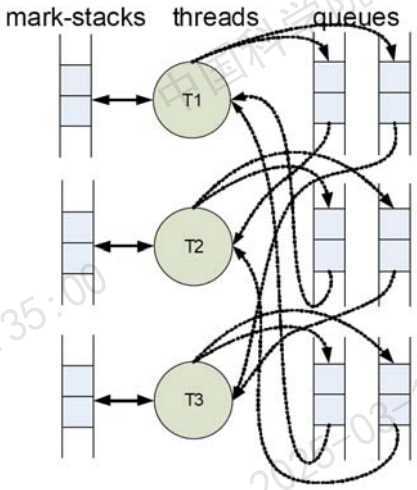

图 6.9 标记过程的处理网络

示为<*i*, *>，线程 *j* 的所有的输入队列被表示为<*, *j*>，其中星号“*”表示所有有效的线程索引号。

垃圾收集线程 *i* 优先的从自己的 mark-stack 中获取任务，直到 mark-stack 为空。此时，线程 i 检查自己的输入队列<*, *i*>，并尝试从中获取新的任务。在执行任务（即扫描对象）的过程中，收集线程 *i* 主动地将所有新产生的任务公平地分配给所有的收集线程（包括它自己）。在线程 *i* 看来所有的收集线程都是对等的。在具体的实现中，我们采用 Round-Robin 的轮转方式来决定当前产生的新任务应该分配给哪个线程。如果该任务应该分配给线程 *j*，则线程 *i* 检查队列<*i, j*>是否已满。如果队列<*i, j*>中仍然有空余的数据项，则将该任务放入队列<*i, j*>。如果队列<*i, j*>已满或者该任务本应该分配给线程 *i* 自己，则线程 *i* 就将该任务放入自己的 mark-stack 中。

在队列<*i, j*>已满的情况下，收集线程 *j* 不可能正处于空闲的状态，因为即使线程 *j* 的 mark-stack 为空，它也能够立刻从它的输入队列<*i, j*>中获得新的任务。图 6.10 显示了 task-pushing 算法的伪代码。布尔变量 *Exit_Marking* 指示了整个标记阶段是否已经结束，*mark_stack[i]*是收集线程 *i* 的局部 mark-stack。

```
Thread i:
while( !Exit_Marking ) {
   while ((task = pop (mark_stack[i])) != NULL) {
      new_tasks = execute( task );
      foreach( task in new_tasks)   {
         if(j needs task-pushing and queue <i,j> has vacancy){
            enqueue( task, <i,j>);
            continue;
         }
         push(task, mark_stack[i]);
      }
   }
   if( queue <k,i> for any k has item ){
      task = dequeue( <k,i> );
      push(task, mark_stack[i] );
   }
}
```

图 6.10 task-pushing 的伪代码

在这样的处理网络中，一旦程序开始运行并进入稳定的状态，所有的线程都应该处于忙碌的状态。因此，只要标记过程能够正确地终止，我们就可以合理地期望该系统会获得很好的可缩放性。

6.4.3 标记过程的正确终止

标记过程的终止问题之所以存在，是因为一个收集线程不能够局部地决定是否它应该退出标记阶段。即使当前的 mark-stack 和所有的输入队列都为空也不意味着它不再会有新的任务需要去做，因为其它的收集线程可能很快就会把新的任务传递过来。

因此，我们开发了一个无需原子指令支持的终止检测机制。其伪代码如图 6.11 所示，其中的符号表示与图 6.10 相同。

***Thread 0*:**	***Thread i (i≠0)*:**

terminating = FALSE;
if(mark_stack[0] and <*,0> are empty)
  terminating = TRUE;
else
  goto NORMAL;
 	
for(all thread i)
  if(! no_work[i] || <i, *> not empty)
  goto NORMAL;
 	
if(terminating){
  Exit_Marking = TRUE;
  exit_mark_phase;
}
 	
goto NORMAL;
	no_work[i] = FALSE;
	if(mark_stack[i] and <*,i> are empty)
	  no_work[i] = TRUE;
	else
	  goto STOP_CHECK;
	
	while(<*, i> are empty){
	  if(Exit_Marking){
	  exit_mark_phase;
	  }
	}
	
	STOP_CHECK:
	no_work[i] = FALSE;
	terminating = FALSE;
	goto NORMAL;

图 6.11 并行标记过程的终止检测机制

我们可以任意指定一个垃圾收集线程作为终止检测线程，在图 6.11 中，线程 *0*（*Thread 0*）被指定来承担这个角色，它的指令序列与其它所有的收集线程（*Thread i*）的指令序列有所不同。一个全局的标志 *terminating* 被用来指示是否当前的终止检测结果是有效的。当线程 *0* 没有新的任务可做时，它把标志 *terminating* 置为 TRUE。当其它收集线程有新的任务可做时，该线程会把标志 *terminating* 置为 FALSE。线程 *i* 维护标志 *no_work[i]*，该标志指示是否线程 *i* 有任务可做。

当线程 *0* 没有任务可做的时候，它设置标志 *terminating* 为 TRUE，然后检查是否所有其它的线程也没有任务可做。假如检查的结果表明其它的线程都已经没有任务可做，线程 *0* 就再检查标志 *terminating*。如果标志 *terminating* 仍为 TRUE，则线程 *0* 通知其它的线程标记过程已经结束，然后退出标记阶段。假如线程 *i* 的所有输入队列都为空，则它只是简单地循环以等待新任务的到来。在循环中，它也可以接受线程 *0* 的通知并退出标记过程。否则，如果线程 *i* 能够从输入队列中获得新的任务，它就会将标志 *no_work[i]* 和标志 *terminating* 都置为 FALSE，然后返回到正常的执行路径上去。

下面我们将给出该终止检测算法的正确性证明。

证明：在终止检测的过程中，唯一可能出错的一个情况是检测线程已经检查过某个线程，并在当时发现该被检查的线程已经没有任务可做（标志 *no_work[i]*为 TRUE），而且该线程的所有输出队列均为空（<*i*, *>均为空），但是之后，该线程又被某个还未被检查的线程重新激活[1]，然而这个激活却未被检测线程所观察到。

假设线程 *0* 为检测线程，它按照线程索引号的升序排列来检查其它所有的线程。假设线程 *i* 会被线程 *j* 重新激活，并且 $0 < i < j$，线程 *j* 将会在线程 *i* 被检查之后才会被检查。由于线程 *i* 只有在将标志 *terminating* 置为 FALSE 之后才会从队列<*j*, *i*>中取出新的任务，并且线程 *0* 一定会在检查过线程 *i* 之后的某个时刻检查队列<*j*, *i*>是否为空，因此只有下面两个情况可能发生：

1. 如果对队列<*j*, *i*>的检查发生在线程 *i* 从队列<*j*, *i*>中取出任务之后，则标志 *terminating* 已经被置为 FALSE，终止检测过程将被中断；
2. 如果对队列<*j*, *i*>的检查发生在线程 *i* 从队列<*j*, *i*>中取出任务之前，则当检查队列<*j*, *i*>的时候，该队列不可能为空。于是，终止检测过程将被中断；

因此，这种重新激活总是会被检测线程观察到。

证毕■

6.5 Task-pushing 算法的负载平衡

在 task-pushing 算法中有一些和负载平衡相关的很微妙的设计考虑会影响所能获得的可缩放性。本节讨论两个有关的问题，并给出解决的方案。

6.5.1 任务的分配策略

按照我们的术语，在对象扫描过程中所发现的任何未被标记的对象的引用都被看作是新产生的任务。在 task-pushing 算法的初始设计中，由线程 *i* 新产生的任务通过两种方式被公平地分配给所有的收集线程：

1) 将任务放入队列<*i*, *>，假如这些队列有空余的数据项；
2) 将任务放入线程 *i* 的局部 mark-stack 中；

每一个收集线程都通过维护一个计数器以 Round-Robin 的方式来决定任务的分配。

这种简单的负载平衡策略存在一个问题。如果一个正在忙碌的收集线程所扫描的对象中，大部分的对象都没有非空的引用（即对象没有引用域或引用的值为 NULL），则该线程可能不能够立刻为正在空闲的线程产生新的任务。这时，将会有大量的对象被堆积在该线程的 mark-stack 中而不能够及时地被分配出去，我们将这种现象称为任务的沉积（task deposition）。

通过允许将任务从 mark-stack 的底部“漏”出，可以使 task-pushing 得到改进，我们

[1] 被重新激活是指线程又能够重新获得新的任务。

称这种技术为 task-dripping。当一个收集线程正在扫描一个对象却没能够产生新的任务，同时当前需要被分配任务的线程的输入队列中也有空余的数据项的时候，正在扫描对象的这个收集线程就从它的 mark-stack 的底部取出一个任务来分配给相应的收集线程。我们借用 Arora 的 work-stealing-queue [Arora98]的设计思想将 mark-stack 实现成一个双端队列。由于 mark-stack 是线程局部的，因此这个双端的 mark-stack 的实现仍然不需要同步原语的支持。

6.5.2 队列长度的选择

和 mark-stack 一样，SRSW 队列也会导致任务沉积的出现。由于对每个队列只有一个线程能够获取其中的任务，如果该队列中的任务也大都不能够导致新任务的产生，则任务沉积的现象就会在该队列中出现。

我们无法通过 dripping 的方法来解决队列中的任务沉积，因为 SRSW 队列不能够被设计成双端的形式，否则就必需引入同步原语。因此，我们通过减少队列的长度来缓和队列中的任务沉积。在极端的情况下，队列的长度可以为 1。这时，队列就退化成为两个线程之间的共享变量。然而，当队列的长度变得太短的时候，被削弱的缓冲效果可能会给可缩放性带来一定的损失。然而，我们相信，对于大部分的服务器端的应用来说，一个非常小的队列长度应该足以带来非常好的可缩放性。

另外，非常小的队列长度意味着非常少的内存需求。对于 N 个垃圾收集线程来说，task-pushing 算法需要维护 N*(N-1)个队列。短的队列长度意味着和堆的大小相比，所有队列的内存需求都是可以被忽略的。例如，当队列的长度为 2 时，16 个收集线程的通信队列所消耗的内存大小小于 2KB（1920 字节）。

由于队列长度可以非常小，这同时也意味着存在着把队列用硬件寄存器来实现的可能性。在未来多核的发展趋势中，处理器之间通过寄存器来相互通信是可能的。这也从另一个侧面说明，我们的这种基于 CSP 风格的数据流计算的 task-pushing 技术与多核体系结构的发展趋势是相吻合的。

6.6 实验评估

6.6.1 基准测试程序和实验平台

Task-pushing 算法主要是为大规模的并行计算机而设计的，因此我们选择了两个模拟了服务器端应用行为的基准测试程序来评估该算法：GCOld 和 pseudojbb。在我们的实验中，堆的大小都被配置为 320MB。

GCOld[GO]是一个合成的基准测试程序。它综合的建模了一系列的服务器应用的对象行为特点。该程序维护了一个指针数组，数组中每个指针都指向一个二叉树的根。

GCOld 的一次运行包括一个初始化阶段和一个具有稳定状态[1]的阶段。初始化阶段分配和初始化一些数据结构，而稳定阶段则由一些 step 组成。每个 step 分配一些寿命很短的对象并做一些改变对象连通关系的工作。每个 step 也会在一些寿命很长的树结构中分配一些节点并替换一些先前存在的树，使这些树结构变为不可达的垃圾结构。每个 step 还会对那些长寿命的树结构做一些指针修改的工作。GCOld 允许通过命令行参数来控制每个 step 的工作量。在我们的实验中，我们配置 GCOld 的存活数据量为 300MB，短寿命对象的分配率和长寿命对象的分配率的比率为 3:1，即每分配 1 个字节的长寿命对象就会分配 3 个字节的短寿命对象。

Pseudojbb 是 SPECjbb2000[SPECjbb00]的一个变种。与 SPECjbb2000 不同，pseudojbb 并不是测量单位时间所完成的操作的数量，而是给出固定的操作数量，并测量执行的时间。在我们的评估中，pseudojbb 被配置为运行 8 个 warehouse，并且每个 warehouse 处理 500000 个交易（transaction）。

我们所使用的硬件平台是一台 Unisys Enterprise Server ES7000 500 series，它拥有 16 个 Intel Xeon 处理器，每个处理器的主频为 3.0GHz，并配有 4MB 的 cache；操作系统平台是 Fedora Core release 3。我们的实验仅对标记阶段的行为进行了评估。

6.6.2 SRSW 队列长度的选择

我们测量了不同的通信队列的长度对 task-pushing 的可缩放性的影响。图 6.12 显示了测量的结果。

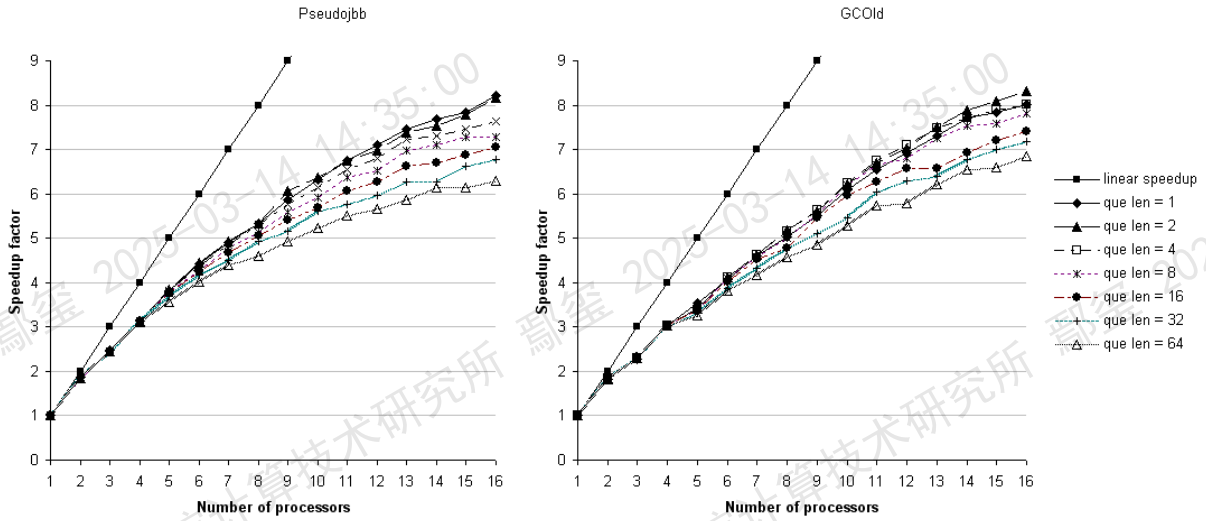

图 6.12 不同队列长度对 task-pushing 可缩放性的影响（任务分配策略为 hybrid task sharing）

图 6.12 显示了 task-pushing 在处理器个数从 1 到 16 的加速比曲线。每条曲线对应着一个队列的长度。从整体上看，这些曲线表明当队列的长度大于 2 时，队列越长，可缩放性越差。对于 pseudojbb 来说，最好的可缩放性在队列长度为 1 时获得，而对于 GCOld 来说，当处理器个数较多时，最好的可缩放性在队列长度为 2 时获得。这个结果和我们

[1] 这里的稳定状态是指稳定的堆占用率。

的分析是一致的：当队列长度较大时，任务沉积的现象会使可缩放性受到影响，而队列的缓冲效果在 task-pushing 算法中并不是关键问题。

6.6.3 任务分配策略

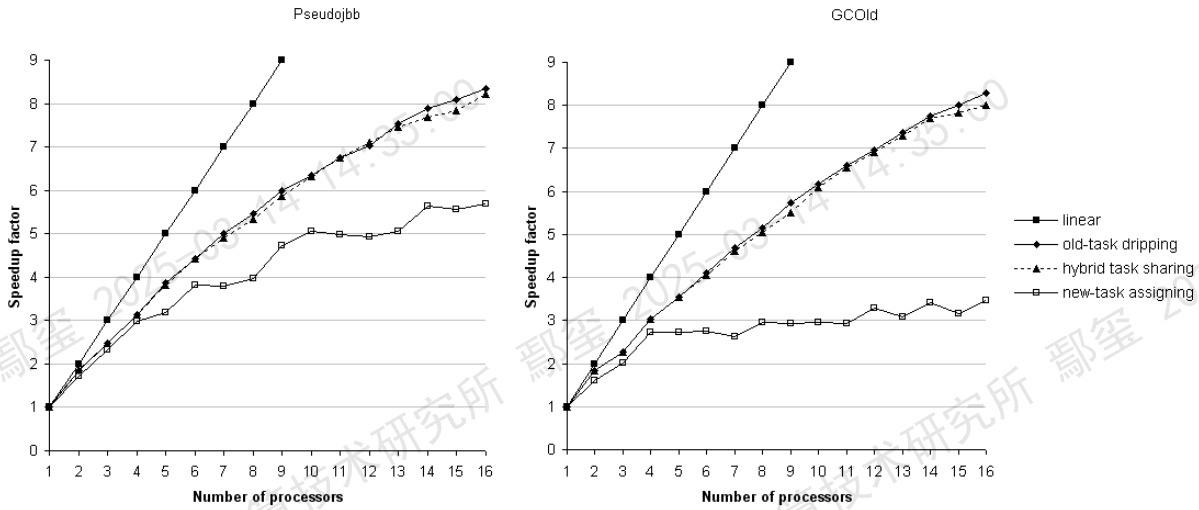

图 6.13 不同的任务分配策略对 task-pushing 的可缩放性的影响（队列长度为 1）

为了更好的理解任务分配策略对 task-pushing 的影响，我们实现了 3 种不同的策略：

New-task assigning：在扫描对象的过程中，只有新产生的任务才会被传递给其它的线程，即没有任何 dripping 的策略；

Old-task dripping：在分配任务的时候，收集线程总是从 mark-stack 的底部弹出任务并传递给其它的线程；

Hybrid task sharing：如果有新产生的任务，就将新产生的任务分配给其它线程；否则，就从 mark-stack 底部弹出任务并进行分配；

图 6.13 显示了不同的任务分配策略对 task-pushing 的可缩放性的影响。“new-task assigning”的策略显示出了颠簸的加速比曲线，表明该策略具有较差的负载平衡。另外两种策略由于采用了 task-dripping，都显示出了非常平滑的加速比曲线，而且“old-task dripping”策略似乎总是更好一些。这个结果和我们的直觉是一致的：与栈顶的对象引用相比，mark-stack 底部的对象引用通常代表着更多的任务，因为深度优先的堆遍历策略意味着越早被放入栈中的对象越可能会更靠近某个树结构的根。我们的实验结果表明，对于 task-pushing 算法来说，任务的 dripping 技术是非常关键的。

6.6.4 原子 CAS 指令的影响

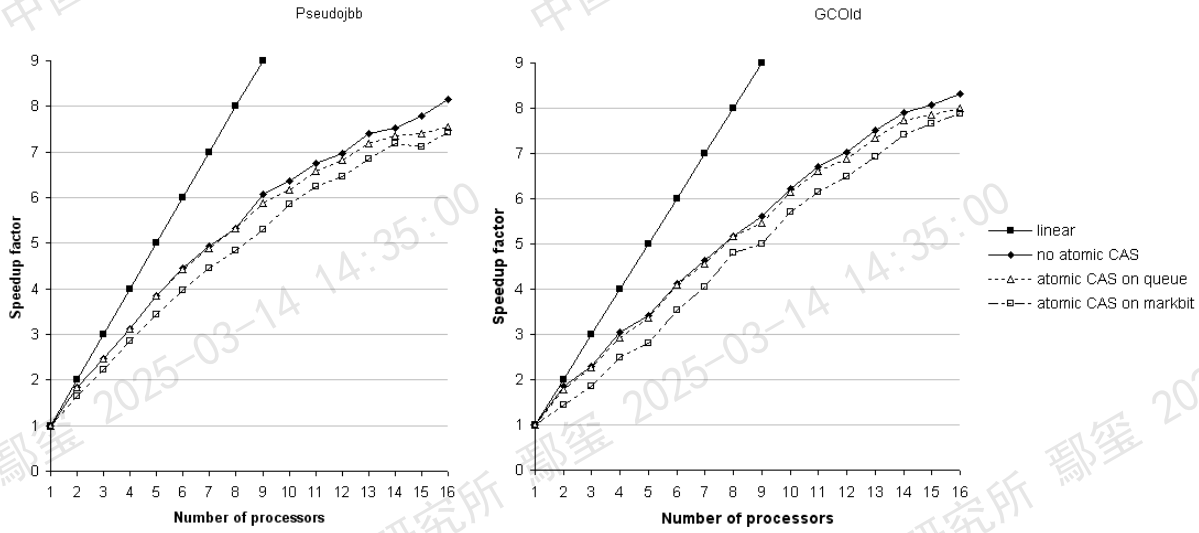

图 6.14 同步原语对可缩放性的影响（任务分配策略为 hybrid task sharing，队列长度为 2）

在传统的并行 GC 标记算法中，一般有两个地方需要依赖同步原语的支持：标记操作和对任务交换所使用的共享数据结构的访问。其中，对于 task-pushing 来说，任务交换的共享数据结构就是通信队列。尽管 task-pushing 算法已经避免了这两处的同步操作，但是为了了解同步原语给可缩放性带来的影响，我们还是对此进行了评估。

图 6.14 比较了 3 种同步方案：无原子 CAS 指令、在队列的访问上使用原子 CAS 指令和在标记操作上使用原子 CAS 指令。图中的数据表明，在标记操作上使用原子指令会导致较差的可缩放性。明显的差别在处理器个数为 2 的时候就开始出现。这主要是因为标记操作非常频繁，每一个存活对象都对应着一个标记操作，因此，每次操作上的小的开销可能会带来较大的性能问题。

与标记操作上的原子指令的影响相比，在队列上使用原子指令的结果要好一些。当处理器的个数较少的时候，在队列上使用原子指令和不使用原子指令的性能差不多。但是当处理器个数较多时，“atomic CAS on queue”的性能曲线随处理器个数的增加下降的相对较快。在处理器个数为 16 的时候，它的可缩放性和在标记操作上使用原子指令的可缩放性相当。这种现象是可以理解的：标记操作的总量是固定的，不会随处理器个数的增加而增加。而对通信队列的访问频率却会随着处理器个数的增加而增加。

另外，实际的同步操作的影响可能要比图 6.14 中所显示的更加严重，因为同步的开销不仅在于原子指令本身的成本，而且也包括了失败的尝试所带来的开销。但是在 task-pushing 中，虽然我们在队列的访问中增加了原子指令，但实际上并不会发生真正的数据竞争，因此不会出现失败的尝试。

6.6.5 Task-pushing vs. Work-stealing

为了比较 task-pushing 和 work-stealing 这两种并行技术，我们在 Harmony 的平台上

实现了基于[Flood01]的 work-stealing 并行 GC 标记算法。该算法是我们所知的引入同步原语最少、动态负载平衡最灵活的算法。

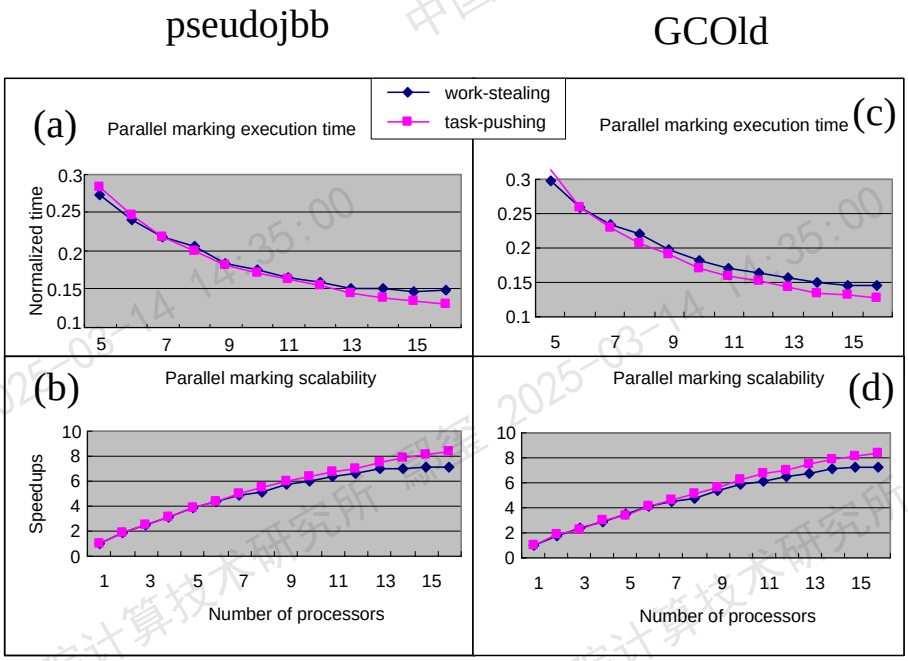

图 6.15 Task-pushing 与 work-stealing 的性能和可缩放性的比较

图 6.15 比较了这两种并行标记算法的执行时间和可缩放性。其中，task-pushing 具有如下配置：

1) 通信队列长度为 1；
2) 任务分配策略为“old-task dripping”；
3) 在标记操作和队列访问操作上均没有原子指令的支持；

被用来比较的 work-stealing 算法在标记操作上也没有使用原子指令。

图 6.15(a)和(c)比较了标记阶段的执行时间。纵坐标是相对于串行标记的执行时间进行了正规化后的时间值。图中只显示了部分的曲线（处理器个数为 5~16），因为当处理器个数小于 5 时，两种算法的执行时间没有明显的差别。Task-pushing 算法在处理器个数小于 7 的时候会稍微慢一点（因为 task-pushing 在产生新任务后需要一些额外的指令来判断该任务应该被分配给哪个线程），但之后便会随着处理器个数的增加而带来持续增加的更好的性能，即处理器个数越多，task-pushing 的相对性能就会越好。处理器个数较少时的性能的些微劣势被处理器较多时的高可缩放性所抵消了。

图 6.15(b)和(d)比较了两种算法的可缩放性。图中的数据结果表明，task-pushing 比 work-stealing 具有更好的可缩放性，并且它们之间的差距也随着处理器个数的增加而增大。有趣的是，我们发现 work-stealing 的加速比曲线在 15 个处理器的时候出现了较大的弯曲，而 task-pushing 的加速比则可以持续的增长。

6.7 本章小结

标记阶段是非拷贝（non-copy）垃圾收集的一个主要组成部分。当堆空间很大并且有大量的存活对象的时候，标记过程常常成为垃圾收集暂停时间的主要贡献者。并行标记能够有效地减少这种暂停时间。为了能够在大规模的多处理器平台上获得更高的可缩放性，同步操作和负载平衡必需被仔细的考虑和设计。我们开发了一个被称为 task-pushing 的并行 GC 标记算法来解决这两个主要问题。Task-pushing 应用 CSP 风格的计算来协调线程的活动。一个高性能的SRSW队列被设计出来以避免对同步原语的依赖，从而减少同步的开销。我们研究了不同的任务分配策略，并且发现 task-dripping 能够带来更好的可缩放性。

在一个 16-way 的机器上，我们使用两个 sever-kind 的基准测试程序对 task-pushing 和 work-stealing 技术进行了比较。实验结果表明，尽管对于单线程的执行来说，task-pushing 会有一些额外的操作，但是随着处理器个数的增加，task-pushing 能够带来持续增加的更优的性能。

中国科学院计算技术研究所 鄢銎 2025-03-14 14:35:00

中国科学院计算技术研究所 鄢玺 2025-03-14 14:35:00

第七章 总结及未来工作

Java 语言以其在软件工程上的优势而被广泛地应用在各个领域的软件开发中。Java 语言的一个重要的特性就是使用了自动内存管理，即垃圾收集。自动内存管理使程序员无需显式地调用分配内存和回收内存的接口，这大大减轻了程序员在进行软件开发时的负担，减少了引入内存错误的概率。垃圾收集技术已经经历了 40 多年的历史，它最早被应用在 Lisp 语言之中。然而，学术界和工业界对垃圾收集技术的讨论一直都没有停止过。这主要是因为垃圾收集不仅能给软件开发带来好处，同时也对应用程序的性能有着重要的影响。特别是近年来 Java 语言的广泛使用，使得研究者们对垃圾收集技术的讨论变得更加的热烈。

本文系统深入的分析和研究了 Java 运行时环境中的垃圾收集技术。主要的工作如下：

第一，设计和实现了一个支持灵活的算法动态切换的内存管理软件框架——RSMTk。该框架通过使用 Allocation Area 的概念来使不同的收集算法共用相同的对象分配函数，从而避免了在收集算法切换过程中由于分配函数的内联而引起的方法重编译。

比较了不同垃圾收集算法对应用的对象局部性的影响。比较了深度优先拷贝序，广度优先拷贝序和保持对象分配序的收集算法对对象局部性的影响。我们发现，保持对象分配序通常能够取得较优的对象局部性。另外，我们还比较分析了在拷贝垃圾收集中，分代内的不同分区间的拷贝序和不同分代间的拷贝序对对象局部性的影响。结果表明，分代内的不同分区间的对象拷贝序是影响对象局部性的主要因素。

调查了 on-chip 的 local memory 在 Java 应用中对内存系统性能的影响。结果表明，on-chip local memory 和 cache 相结合的系统能够比 cache-only 的系统具有更优的性能。

第二，分析了服务器端 Java 应用中的对象生命周期行为，发现其中存在着大量长生命周期的对象。设计并实现了一个基于长生命周期对象的混合垃圾收集器。该收集器利用几种经典收集算法的运行时动态切换及其相互的协作来减少垃圾收集对长生命周期对象的重复操作。我们使用 SPECjbb2005 基准测试程序对该算法进行了评估。实验结果表明，与经典的分代垃圾收集相比，我们的混合垃圾收集算法能够使 SPECjbb2005 的 Throughput 分值提高 3%~25%。

第三，设计并实现了一种新的并行垃圾收集标记算法——task-pushing。受到 CSP 思想的启发，task-pushing 通过将标记过程构造成一个处理网络来避免对同步原语的依赖同时获得良好的动态负载平衡。为了支持这种无需同步操作的并行标记算法，我们设计了一种高性能的通信队列，该队列支持单个读者和单个写者，利用现有硬件所支持的字对齐访问操作的原子性来避免对同步原语的使用。

在今后的研究工作中，针对以上三个方面的工作，我们可以进行如下进一步的研究：

第一，研究动态配置 on-chip local memory 的技术。由于程序的行为在动态地改变，

local memory 被映射到的地址空间也不应该是一成不变的。随着程序的相不断地发生变化，可能某些其它的地址空间的访问更加适合通过 local memory 来进行，这时就需要改变 local memory 的地址映射。这种动态的改变机制的研究有硬件和软件两个层次。硬件方面主要研究如何实现 local memory 的映射地址的改变，软件方面主要研究如何动态的获取程序行为信息来确定映射地址改变的策略。

第二，进一步研究 Java 应用中的对象行为特点，包括对象的生命周期和对象的大小。寻找针对这些行为特点的更加灵活有效的垃圾收集算法。

第三，尝试将 CSP 风格的并行算法应用在垃圾收集的其它阶段中，使得在整个的并行垃圾收集过程中都能够仅仅引入最少的同步。Task-pushing 算法所使用的队列的长度很小，这使得它们可以通过硬件的寄存器来实现，因此这种算法同时也可以被看作是寄存器网络的一种应用。今后的研究可以将该算法和多核体系结构的研究相结合，即研究多核中寄存器网络的实现和应用。另外，Task-pushing 从本质上并不仅仅是一个并行标记算法，而是一种并行编程方法，因此，未来的工作可以研究如何将 task-pushing 应用到其它的并行计算领域中。

中国科学院计算技术研究所 鄢玺 2025-03-14 14:35:00

参考文献

[AA90] R. Alonso and Andrew W. Appel. An advisor for flexible working sets. In *Proceedings of the 1990 ACM SIGMETRICS Conference on Measurement and Modeling of Computer Systems*, Boulder, May 1990, pages 153-162, ACM Press.

[Abuaiadh04] Diab Abuaiadh, Yoav Ossia, Erez Petrank, and Uri Silbershtein. An efficient parallel heap compaction algorithm. In *ACM Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA'04)*, ACM SIGPLAN Notices, Vancouver, October 2004. ACM Press.

[Agesen97] Ole Agesen and David L. Detlefs. Finding references in Java stacks. *OOPSLA'97 Garbage Collection and Memory Management Workshop*, Atlanta, GA, October 1997.

[Agesen98a] Ole Agesen, David L. Detlefs, and J. Eliot B. Moss. Garbage collection and local variable type-precision and liveness in Java virtual machines. In *Proceedings of the ACM SIGPLAN'98 Conference Programming Language Design and Implementation (PLDI)*, Montreal, Canada, June 1998, pages 269-279.

[Agesen98b] Ole Agesen. GC points in a threaded environment. *Technical Report 98-70*, SUN Microsystems, Inc. 1998.

[AHSS04] A. Adl-Tabatabai, R. Hudson, M. Serrano, and S. Subramoney. Prefetch injection based on hardware monitoring and object metadata. In *Proceedings of the ACM SIGPLAN Conference on Programming Languages Design and Implementation (PLDI'04)*, 2004, pages 267-276.

[Andre86] D. L. Andre. Paging in Lisp programs. Master's thesis, University of Maryland, College Park, Maryland. 1986.

[Andreasson02] E. Andreasson, F. Hoffmann, and O. Lindholm. To collect or not to collect? Machine learning for memory management. In *Proceedings of the Java Virtual Machine Research and Technology Symposium (JVM'02)*, August 2002.

[Appel87] Andrew W. Appel. Garbage collection can be faster than stack allocation. *Information Processing Letters*, 25(4), June 1987, pages 275-279.

[Appel89a] Andrew W. Appel. Runtime tags aren't necessary. *Lisp and Symbolic Computation*, 2, 1989, pages 153-162.

[Appel89b] A. W. Appel. Simple generational garbage collection and fast allocation. *Software Practice and Experience*, 19(2), 1989, pages 171-183.

[Arora98] Nimar S. Arora, Robert D. Blumofe, and C. Greg Plaxton. Thread scheduling for multiprogrammed multiprocessors. In *ACM Symposium on Parallel Algorithms and*

Architectures (SPAA), 1998, pages 119-129.

[Attanasio01] C. R. Attanasio, D. F. Bacon, A. Cocchi, and S. Smith. A comparative evaluation of parallel garbage collectors. In *Proceedings of the Fourteenth Annual Workshop on Languages and Compilers for Parallel Computing*, Cumberland Falls, Kentucky, August 2001, vol. 2624 of Lecture Notes in Computer Science, Springer-Verlag.

[Bacon00] David Bacon, Dick Attanasio, Han Lee, and Stephen Smith. Java without the coffee breaks: A nonintrusive multiprocessor garbage collector. In *Proceedings of SIGPLAN 2000 Conference on Programming Languages Design and Implementation*, ACM SIGPLAN Notices, Vancouver, June 2000. ACM Press.

[Badawy01] A. -H. Badawy, A. Aggarwal, D. Yeung, and C. -W. Tseng. Evaluating the impact of memory system performance on software prefetching and locality optimizations. In *2001 international Conference on Supercomputing (ICS'01)*, Sorrento, Italy, June 2001, pages 486-500. ACM Press.

[Baker78] Henry G. Baker. List processing in real-time on a serial computer. *Communications of the ACM*, 21(4), 1978, pages 280-294.

[Baker91] H. G. Baker. Cache-conscious copying collectors. In *OOPSLA/ECOOP'91 Workshop on Garbage Collection in Object-Oriented Systems*, 1991.

[Banakar02] R. Banakar, S. Steinke, B.-S. Lee, M. Balakrishnan, and P. Marwedel. Scratchpad memory: a design alternative for cache on-chip memory in embedded systems. In *Proceedings of the Tenth International Symposium on Hardware/Software Codesign*, 2002, pages 73–78.

[BEA] M. Rosen. BEA's enterprise platform. IDC white paper sponsored by BEA. http://www.bea.com/framework.jsp.

[Blackburn01] S. Blackburn, J. Moss, K. McKinley, and D. Stefanovic. Pretenuring for Java. In *Proceedings of the Conference on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA)*, Tampa, FL, October 2001.

[Blackburn02] S. M. Blackburn, R. E. Jones, K. S. McKinley, and J. E. B. Moss. Beltway: Getting around garbage collection gridlock. In *Proceedings of SIGPLAN 2002 Conference PLDI*, Berlin, Germany, June 2002, pages 153-164.

[Blackburn04a] S. M. Blackburn, P. Cheng, and K. S. McKinley. Myths and realities: The performance impact of garbage collection. In *SIGMETRICS*, NY, June 2004.

[Blackburn04b] S. M. Blackburn, P. Cheng, and K. S. McKinley. Oil and water? High performance garbage collection in Java with JMTk. In *Proceedings of the International Conference on Software Engineering*, Scotland, UK, May 2004, pages 137-146.

[Blau83] R. Blau. Paging on an object-oriented personal computer for Smalltalk. In *ACM SIGMETRICS Conference on Measurement and Modeling of Computer Systems*, Minneapolis, August 1983.

[Blelloch01] Guy E. Blelloch, Perry Cheng, and Phil Gibbons. Room synchronizations. In *ACM Symposium on Parallel Algorithms and Architecture*, July 2001, ACM Press.

[Boehm88] Hans-Juergen Boehm and Mark Weiser. Garbage collection in an uncooperative environment. *Software Practice and Experience*, 18(9), 1988, pages 807-820.

[Boehm92] Hans-Juergen Boehm and David R. Chase. A proposal for garbage-collector-safe C compilation. *Journal of C Language Translation*, 1992, pages 126-141.

[Boehm93] Hans-Juergen Boehm. Space efficient conservative garbage collection. In *Proceedings of ACM SIGPLAN Conference on Programming Languages Design and Implementation*, 1993, pages 197-206.

[Brecht01] Tim Brecht, Eshrat Arjomandi, Chang Li, and Hang Pham. Controlling garbage collection and heap growth to reduce the execution time of Java applications. In *ACM Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA'01)*, November 2001.

[Buytaert05] D. Buytaert, K. Venstermans, L. Eeckhout et. al. Garbage collection hints. In *Proceedings of HIPEAC'05*, Lecture Notes in Computer Science Volume 3793, Springer-Verlag, November 2005

[Calder98] Brad Calder, Chandra Krintz, Simmi John, and Todd Austin. Cache-conscious data placement. In *Proceedings of the Eighth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS VIII)*, October 1998.

[CDL99] T. M. Chilimbi, B. Davidson, and J. R. Larus. Cache-conscious structure definition. In *ACM SIGPLAN Conference on Programming Languages Design and Implementation*, Atlanta, GA, May 1999, pages 13-24.

[CH02] T. Chilimbi and M. Hirzel. Dynamic hot data stream prefetching for general purpose programs. In *Proceedings of ACM SIGPLAN Conference on Programming Languages Design and Implementation (PLDI)*, 2002.

[Chambers93] Craig Chambers. The Cecil language: Specification and rationale. *Technical Report TR-93-03-05*, University of Washington Seattle, March 1993.

[Chen06] Wen-ke Chen, Sanjay Bhansali, Trishul Chilimbi, Xiaofeng Gao, and Weihaw Chuang. Profile-guided proactive garbage collection for locality optimization. In *Proceedings of ACM SIGPLAN Conference on Programming Languages Design and Implementation (PLDI'06)*, Ottawa, Ontario, Canada, June, 2006.

[Cheney70] C. J. Cheney. A non-recursive list compacting algorithm. *Communications of the ACM*, 13(11), 1970, pages 677-678.

[Cheng98] P. Cheng, R. Harper, and P. Lee. Generational stack collection and profile-driven pretenuring. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, Montreal, Canada, June 1998, pages 162–173.

[Cheng01] P. Cheng and G. Blelloch. A parallel, real-time garbage collector. In *ACM SIGPLAN Conference on Programming Languages Design and Implementation*, Snowbird, UT, June 2001, pages 125-136.

[CL98] T. M. Chilimbi and J. R. Larus. Using generational garbage collection to implement cache-conscious data placement. In *ACM International Symposium on Memory Management*, Vancouver, BC, October 1998, pages 37-48.

[Clark77] Douglas W. Clark and C. Cordell Green. An empirical study of list structure in LISP. *Communications of the ACM*, 20(2), 1977, pages 78-87.

[Clark79] Douglas W. Clark. Measurements of dynamic list structure use in Lisp. *IEEE Transactions on Software Engineering*, 5(1), 1979, pages 51-59.

[Click05] Cliff Click, Gil Tene, and Michael Wolf. The pauseless GC algorithm. In *Proceedings of 1st International Conference on Virtual Execution Environments*, Chicago, June 2005.

[CM01] B. Cahoon and K. S. McKinley. Data flow analysis for software prefetching linked data structures in Java. In *Proceedings of the International Conference on Parallel Architectures and Compiler Techniques*, September 2001.

[Cohen83] Jacques Cohen and Alexandru Nicolau. Comparison of compacting algorithms for garbage collection. *ACM Transactions on Programming Language and Systems*, 5(4), 1983, pages 532-553.

[Collins60] George E. Collins. A method for overlapping and erasure of lists. *Communications of the ACM*, 3(12), 1960, pages 655-657.

[Cooper92] E. Cooper, S. Nettles, and I. Subramanian. Improving the performance of SML garbage collection using application-specific virtual memory management. In *Conference Record of the 1992 ACM Symposium on Lisp and Functional Programming*, San Francisco, CA, June 1992, pages 43-52.

[Courts88] R. Courts. Improving locality of reference in a garbage-collecting memory management system. *Communications of the ACM*, 31(9), 1988, pages 1128-1138.

[DeT90] John DeTreville. Experience with concurrent garbage collectors for Modula-2+. *Technical Report 64*, Digital Equipment Corporation Systems Research Center, Palo Alto, California, August 1990.

[DeTreville90] John DeTreville. Heap usage in the Topaz environment. *Technical Report 63*, DEC Systems Research Center, Palo Alto, CA, August 1990.

[Deutsch76] L. Peter Deutsch and Daniel G. Bobrow. An efficient incremental automatic garbage collector. *Communications of the ACM*, 19(9), 1976, pages 522-526.

[DH98] Sylvia Dieckman and Urs Holzle. A study of the allocation behaviour of the SPECjvm98 Java benchmarks. In Erik Jul, editor, *Proceedings of the 12th European Conference on Object-Oriented Programming, ECOOP'98*, volume 1445 of Lecture Notes in Computer

Science, Brussels, Belgium, 1998, pages 92-115, Springer-Verlag.

[Dijkstra76] Edsgar W. Dijkstra, Leslie Lamport, A. J. Martin, C. S. Scholten, and E. F. M. Steffens. On-the-fly garbage collection: An exercise in cooperation. *Lecture Notes in Computer Science*, No. 46. Springer-Verlag, New York, 1976.

[Ding05] C. Ding, C. Zhang, X. Shen, and M. Ogihara. Gated memory control for memory monitoring, leak detection and garbage collection. In *Proceedings of the 3rd ACM SIGPLAN Workshop on Memory System Performance*, Chicago, IL, June 2005.

[Doligez93] Damien Doligez and Xavier Leroy. A concurrent generational garbage collector for a multi-threaded implementation of ML. In *Conference Record of the Twentieth Annual ACM Symposium on Principles of Programming Languages*, ACM SIGPLAN Notices, January 1993, pages 113-123, ACM Press.

[Doligez94] Damien Doligez and Georges Conthier. Portable, unobtrusive garbage collection for multiprocessor systems. In *Conference Record of the Twenty-first Annual ACM Symposium on Principles of Programming Languages*, ACM SIGPLAN Notices, January 1994, ACM Press.

[Edelson90] Daniel R. Edelson. Dynamic storage reclamation in C++. Master's thesis, University of California at Santa Cruz, June 1990.

[Endo97] Toshio Endo, Kenjiro Taura, and Akinori Yonezawa. A scalable mark-sweep garbage collector on large-scale shared-memory machines. In *Proceedings of High Performance Networking and Computing (SC'97)*, 1997.

[Fenichel69] Robert R. Fenichel and Jerome C. Yochelson. A Lisp garbage collector for virtual memory computer systems. *Communications of the ACM*, 12(11), 1969, pages 611-612.

[Flood01] Christine Flood, Dave Detlefs, Nir Shavit, and Catherine Zhang. Parallel garbage collection for shared memory multiprocessors. In *Usenix Java Virtual Machine Research and Technology Symposium (JVM'01)*, Monterey, CA, April 2001.

[Foderaro81] John K. Foderaro and Richard J. Fateman. Characterization of VAX Macsyma. In *1981 ACM Symposium on Symbolic and Algebraic Computation*, Berkeley, CA, 1981, pages 14-19, ACM Press.

[FQ03] S. Fink, and F. Qian. Design, implementation and evaluation of adaptive recompilation with on-stack replacement. In *International Symposium on Code Generation and Optimization (CGO)*, March 2003.

[FT00] R. Fitzgerald, and D. Tarditi. The case for profile-directed selection of garbage collectors. In *Proceedings of the second International Symposium on Memory Management*, 2000, pages 111–120, ACM Press.

[Gabriel85] Richard P. Gabriel. Performance and evaluation of Lisp systems. *MIT Press Series in Computer Science*. MIT Press, Cambridge, MA, 1985.

[GO] Sun Microsystems Laboratories. GCOld Benchmark. http://www.experimentalstuff.com/Technologies/GCold/.

[Goldberg91] Benjamin Goldberg. Tag-free garbage collection for strongly typed programming languages. *ACM SIGPLAN Notices*, 26(6), 1991, pages 165-176.

[Gosling96] J. Gosling, Bill Joy, and G. Steele. The Java Language Specification. Addison-Wesley, 1996.

[Haddon67] B. K. Haddon and W. M. Waite. A compaction procedure for variable length storage elements. *Computer Journal*, 10, 1967, pages 162-165.

[Halstead85] Robert H. Halstead, Jr. Multilisp: A language for concurrent symbolic computation. *Transactions on Programming Languages and Systems (TOPLAS)*, 7(4), 1985, pages 501-538.

[Hayes91] Barry Hayes. Using key object opportunism to collect old objects. In A*CM Conference on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA'91)*, Phoenix, Arizona, USA, October 1991, pages 33-46.

[Hertz05] M. Hertz, Y. Feng, and E. D. Berger. Garbage collection without paging. In *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation*, New York, NY, USA, 2005, pages 143-153, ACM Press.

[Hoare78] C. A. R. Hoare. Communicating sequential processes. *Communications of the ACM* 21(8), 1978.

[Hoare04] C. A. R. Hoare. Communicating Sequential Processes. http://www.usingcsp.com/. June 21, 2004.

[HP] Hewlett-Packard Company. NonStop Server for Java Software. Project home page. http://nonstop.compaq.com/view.asp.

[HP02] John L. Hennessy, David A. Patterson. Computer Architecture: A Quantitative Approach. Morgan Kaufmann, San Mateo, CA. 3rd edition, 2002.

[Huang04] X. Huang, S. Blackburn, K. McKinley, J. Moss, Z. Wang, and P. Cheng. The garbage collection advantage: improving program locality. In *Proceedings of the ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA'04)*, 2004.

[IBMa] IBM Corporation. WebSphere software platform. Product home page. http://www-3.ibm.com/software/info1/websphere/index.jsp.

[IBMb] IBM Corporation. Persistent Reusable JVM. Project home page. http://www.haifa.il.ibm.com/projects/systems/rs/persistent.html.

[Inagaki03] T. Inagaki, T. Onodera, H. Komatsu, and T. Nakatani. Stride prefetching by dynamically inspecting objects. In *Proceedings of ACM SIGPLAN Conference on Programming Languages Design and Implementation (PLDI)*, 2003.

[IT93] Akira Imai and Evan Tick. Evaluation of parallel copying garbage collection on a shared-memory multiprocessor. *IEEE Transactions on Parallel and Distributed Systems*, 4(9), 1993.

[Jones96] Richard E. Jones. Garbage Collection: Algorithms for Automatic Dynamic Memory Management. Wiley, July 1996.

[Jonkers79] H. B. M. Jonkers. A fast garbage compaction algorithm. *Information Processing Letters*, 9(1), July 1979, pages 25-30.

[KH00] J. Kim and Y. Hsu. Memory system behavior of Java programs: Methodology and analysis. In *ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems*, June 2000, pages 264-274, ACM Press.

[KP06] Haim Kermany, Erez Petrank. The Compressor: concurrent, incremental, and parallel compaction. In *Proceedings of the 2006 ACM SIGPLAN Conference on Programming Languages Design and Implementation (PLDI 2006)*, 2006, pages 354-363.

[Kung77] H. T. Kung and S. W. Song. An efficient parallel garbage collection system and its correctness proof. In *IEEE Symposium on Foundations of Computer Science*, 1977, pages 120-131, IEEE Press.

[Lam92] M. S. Lam, P. R. Wilson, and T. G. Moher. Object type directed garbage collection to improve locality. In *Proceedings of International Workshop on Memory Management*, Volume 637 of Lecture Notes in Computer Science, Saint Malo, France, September 1992, pages 404-425, Springer-Verlag.

[Lang87] Bernard Lang and Francis Dupont. Incremental incrementally compacting garbage collection. In *SIGPLAN 1987 Symposium on Interpreters and Interpretive Techniques*, Saint Paul, Minnesota, June 1987, pages 253-263, ACM Press. Published as SIGPLAN Notices 22(7), July 1987.

[LH83] Henry Lieberman and Carl E. Hewitt. A real-time garbage collector based on the lifetimes of objects. *Communications of the ACM*, 26(6), 1983, pages 419-429.

[LM96] C.-K. Luk and T. C. Mowry. Compiler-based prefetching for recursive data structures. In *Proceedings of the 7th InternationalConference on Architectural Support for Programming Languages and Operating Systems*, October 1996, pages 222–233.

[LM99] C.-K. Luk and T. C. Mowry. Automatic compiler-inserted prefetching for pointer-based applications. *IEEE Transactions on Computers*, 48(2), 1999.

[LY96] Timothy Lindholm and Frank Yellin. The Java Virtual Machine Specification. The Java Series, Addison-Wesley, 1996.

[McCarthy60] John McCarthy. Recursive functions of symbolic expressions and their computation by machine. *Communications of the ACM*, 3, 1960, pages 184-195.

[Meyer88] Bertrand Meyer. Object-oriented Software Construction. Prentice-Hall, 1988.

[Moon84] David A. Moon. Garbage collection in a large Lisp system. In *LISP and Functional*

Programming (LFP), 1984

[Ossia02] Yoav Ossia, Ori Ben-Yitzhak, Irit Goft, Elliot K. Kolodner, Victor Leikehman, and Avi Owshanko. A parallel, incremental and concurrent GC for servers. In *Proceedings of the 2002 ACM SIGPLAN Conference on Programming Languages Design and Implementation*, ACM SIGPLAN Notices, Berlin, June 2002, ACM Press.

[Pepels88] E. J. H. Pepels, M.C.J.D. van Eekelen, and M. J. Plasmeijer. A cyclic reference counting algorithm and its proof. *Technical Report 88-10*, Computing Science Department, University of Nijmegen, 1988.

[Printezis01] T. Printezis. Hot-swapping between a mark&sweep and a mark&compact garbage collector in a generational environment. In *Usenix Java Virtual Machine Research and Technology Symposium*, Monterey, California, April 2001.

[Purify92] Pure Software, Los Altos, CA. Purify, 1992.

[RS99] A. Roth and G. Sohi. Effective jump pointer prefetching for linked data structures. In *International Symposium on Computer Architecture (ISCA)*, 1999.

[Salkild87] Jon D. Salkild. Implementation and analysis of two reference counting algorithms. Master's thesis, University College, London, 1987

[Saunders74] Robert A. Saunders. The LISP system for the Q-32 computer. In E. C. Berkeley and Daniel G. Bobrow, editor, *The Programming Language LISP: Its Operation and Applications*, Cambridge, MA, 1974, pages 220-231, Information International, Inc.

[Shaw87] Robert A. Shaw. Improving garbage collector performance in virtual memory. *Technical Report CSL-TR-87-323*, Stanford University, March 1987.

[Shaw88] Robert A. Shaw. Empirical Analysis of a Lisp System. PhD thesis, Stanford University, Palo Alto, California, February 1988.

[Shuf02] Y. Shuf, M. Gupta, H. Franke, A. Appel, and J. Singh. Creating and preserving locality of Java applications at allocation and garbage collection times. In *Proceedings of the ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA'02)*, 2002, pages 13-25.

[Siegwart06] David Siegwart, Martin Hirzel. Improving Locality with Parallel Hierarchical Copying GC. In *International Symposium on Memory management (ISMM)*, 2006.

[SM03] N. Sachindran and J. E. B. Moss. Mark-Copy: Fast copying GC with less space overhead. In *ACM Conference on Object-Oriented Programming Systems, Languages, and Applications*, Anahiem, CA, October 2003, pages 326-343.

[Soman04] S. Soman, C. Krintz, and D. F. Bacon. Dynamic selection of application-specific garbage collectors. In *ISMM'04, Proceedings of the 4th International Symposium on Memory Management*, New York, NY, USA, 2004, ACM Press.

[SPECjbb00] Standard Performance Evaluation Corporation. *SPECjbb2000 (Java Business*

Benchmark) Documentation, release 1.01 edition, 2001.

[SPECjbb05] Standard Performance Evaluation Corporation. *SPECjbb2005 (Java Server Benchmark) Documentation*, v1.05, 2006, http://www.spec.org/jbb2005/.

[Stamos82] J. W. Stamos. A large object-oriented virtual memory: Grouping strategies, measurements, and performance. *Technical Report SCG-82-2*, Xerox PARC, Palo Alto, CA, May 1982.

[Stamos84] J. W. Stamos. Static grouping of small objects to enhance performance of a paged virtual memory. *ACM Transactions on Computer Systems*, 2, 1984, pages 155-180.

[Steele75] Guy L. Steele. Multiprocessing compactifying garbage collection. *Communications of the ACM*, 18(9), 1975, pages 495-508.

[Stefanovic99] Darko Stefanovic, Kathryn S. McKinley, and J. Eliot B. Moss. Age-based garbage collection. In *Proceedings of the 1999 ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications, OOPSLA'99*, volume 34(10) of ACM SIGPLAN Notices, Denver, Colorado, USA, October 1999, pages 370-381, ACM Press.

[Ungar84] David M. Ungar. Generation scavenging: A non-disruptive high performance storage reclamation algorithm. *ACM SIGPLAN Notices*, 19(5), 1984, pages 157-167.

[Ungar88] D. Ungar and F. Jackson. Tenuring policies for generation-based storage reclamation. In N. K. Meyrowitz, editor, *Conference on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA'88)*, San Diego, California, September 1988, pages 1-17.

[Ungar92] D. Ungar and F. Jackson. An adaptive tenuring policy for generation scavengers. *ACM Transactions on Programming Languages and Systems (TOPLAS)*, 14(1), 1992, pages 1-27.

[Weizenbaum62] J. Weizenbaum. Knotted list structures. *Communications of the ACM*, 5(3), 1962, pages 161-165.

[WG98] Derek White and Alex Garthwaite. The GC interface in the EVM. *Technical Report TR-98-67*, Sun Microsystems Laboratories, 1998.

[White80] J. L. White. Address/Memory Management for Gigantic LISP Environment or, GC Considered Harmful. In *Proceedings of the 1980 ACM Conference on LISP and Functional Programming*, 1980, pages 119-127.

[Wilson91] Paul R. Wilson, Michael S. Lam, and Thomas G. Moher. Effective "static-graph" reorganization to improve locality in a garbage-collected system. In *Proceedings of ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 1991.

[Wilson94] Paul R. Wilson. Uniprocessor garbage collection techniques. *Technical report*, University of Texas, January 1994. Expanded version of the IWMM92 paper.

[WM89] Paul R. Wilson and Thomas G. Moher. Design of the opportunistic garbage collector. *ACM SIGPLAN Notices*, 24(10), 1989, pages 23-35.

[Yang04] T. Yang, M. Hertz, E. D. Berger, S. F. Kaplan, and J. E. B. Moss. Automatic heap sizing: taking real memory into account. In *Proceedings of the 4th International Symposium on*

Memory Management, New York, NY, USA, 2004, pages 61-72, ACM Press.

[Yuasa90] Taichi Yuasa. Real-time garbage collection on general-purpose machines. *Journal of Software and Systems*, 11(3), 1990, pages 181-198.

[Zhang06] Chengliang Zhang, Kirk Kelsey, Xipeng Shen, Chen Ding, Matthew Hertz, and Mitsunori Ogihara. Program-level adaptive memory management. In *Proceedings of International Symposium of Memory Management*, Ottawa, Ontario, Canada, June 2006.

[Zorn90] Benjamin Zorn. Comparing mark-and-sweep and stop-and-copy garbage collection. In *Conference Record of the 1990 ACM Symposium on LISP and Functional Programming*, 1990, pages 87-98.

[Zorn93] Benjamin Zorn. The measured cost of conservative garbage collection. *Software Practice and Experience*, 23(7), 1993, pages 733-756.

致 谢

首先要感谢我的导师胡伟武研究员。胡老师对我的学习和科研工作给予了指导和关怀，使我受益匪浅。胡老师给予了我参加工程项目的机会，使我在工程项目中积累了丰富的知识和经验，这无疑为我的博士论文工作的完成带来了莫大的帮助。从胡老师那里，我不仅学习到了学习知识和做研究的方法，更重要的是学习到了一个科研工作者应该具有的素质。胡老师身体力行的“做人，做事，做学问”的思想一直指引着我朝着正确的方向前进。在此，谨向胡老师表示崇高的敬意和衷心的感谢。

感谢王剑老师一直以来对我的关心和帮助。在生活上，王老师总是不遗余力地帮助我，照顾我；在学习和工作上，我也经常请教王老师，并且从与他的讨论中学习到了非常丰富的知识。

感谢“龙芯”课题组提供的良好的科研环境和气氛。感谢课题组的老师和同学们对我的帮助。感谢史岗和吴少刚老师在我们在兰靛厂进行龙芯产业化工作的时候对我的关心和照顾。感谢这么多年来和我一起在科研工作中奋斗的战友们，曾洪博，马可，黄琨，邹琼，朱晓静，马麟，杨祎，侯锐，陈云霁，袁楠……。感谢虽然已经离开计算所，但曾经给过我很大帮助的同学们，林伟，许先超，刘超，董俊峰，陈瑜……。在我攻读博士学位的这些日子里，是你们的帮助和鼓励使我坚持走到了今天。

感谢英特尔中国研究中心的李晓峰及他所带领的研究小组的全体成员。他们为我的博士论文的工作提供了大量的硬件和软件的支持。在他们的帮助下，我能够深入到自动内存管理的领域中。同时，与他们的交流和讨论也给我的博士论文的完成带来了巨大的帮助。

感谢我的女朋友吕炯。感谢她在我最困难的时候一直陪伴着我，给我鼓励和支持，给我信心和勇气。

最后，感谢我的父母。他们含辛茹苦地把我养育成人，供我读书，给我创造良好的学习机会和环境。我一定会以学习和工作中所取得的最好的成绩来报答他们。

中国科学院计算技术研究所 鄢玺 2025-03-14 14:35:00

中国科学院计算技术研究所 鄢玺 2025-03-14 14:35:00

作者简历

姓名：伍鸣　　性别：男　　出生日期：1978.10.19　　籍贯：山西永济

2002.9 – 2007.7　　中科院计算所计算机系统结构专业硕博连读生
1997.9 -- 2002.7　　中国科学技术大学计算机科学与技术系本科生

【攻读博士学位期间发表的论文】

[1]伍鸣，张福新，林伟，许先超，袁楠，王剑，龙芯 2 号处理器系统优化关键技术，计算机研究与发展，Vol.43, No.6, 2006.

[2]Ming Wu, Xiao-Feng Li. Task-pushing: a Scalable Parallel GC Marking Algorithm without Synchronization Operations. In Proceedings of IEEE International Parallel & Distributed Processing Symposium, 2007.

[3]伍鸣，齐骥，邹琼，基于长生命周期对象的混合垃圾收集，小型微型计算机系统（已录用）

[4]邹琼，伍鸣，胡伟武，基于插桩分析的 Java 虚拟机自适应预取优化框架，软件学报（已录用）

【攻读博士学位期间参加的科研项目】

[1] 国家高技术研究发展计划（八六三）重点项目“高性能通用CPU 芯片设计”(2002AA111100)，2002 年～2005 年
[2] 国家863 项目“龙芯2 号增强型处理器芯片设计”（2005AA110010）。
[3] 国家杰出青年基金项目“系统结构研究”（60325205）。

【攻读博士学位期间的获奖情况】

[1] 2005 年获中科院计算所所长优秀学生奖学金
[2] 获得2005-2006 学年中国科学院研究生院“三好学生”荣誉称号